

AY 2025/2026 S1

EE6427 Video Signal Processing

Part 4 AI Models & Architectures

Dr Yap Kim Hui

Room: S2-B2b-53

Tel: 6790 4339

Email: ekhyap@ntu.edu.sg



References

- Stanford Lecture Notes, CS231n: Convolutional Neural Networks for Visual Recognition.
- Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, <http://www.deeplearningbook.org>
- Pytorch Tutorial. <https://pytorch.org/tutorials/>
- University of Wisconsin Madison Lecture Notes, CS638. University of Michigan, EECS 498-007 / 598-005: Deep Learning for Computer Vision
- Simplilearn, Recurrent Neural Network Tutorial
- Shusen Wang, Transformer Model, Youtube Online Videos
- Jay Alammar, The Illustrated Transformer.

Part 4 Outline

- AI Models & Architectures
 - Convolutional Neural Network (CNN)
 - Recurrent Neural Network (RNN)
 - Long Short-Term Memory (LSTM)
 - Transformer

Section I

Convolutional Neural Network (CNN)

Section I Overview

- The section covers the following topics:
 - Introduction
 - Linear Classifier
 - Convolutional Neural Networks (CNNs)
 - CNN Training & Optimization
 - Well-Known CNN Architectures
 - Applications

Different Deep Neural Network (DNN) Architectures

Common DNN Architectures

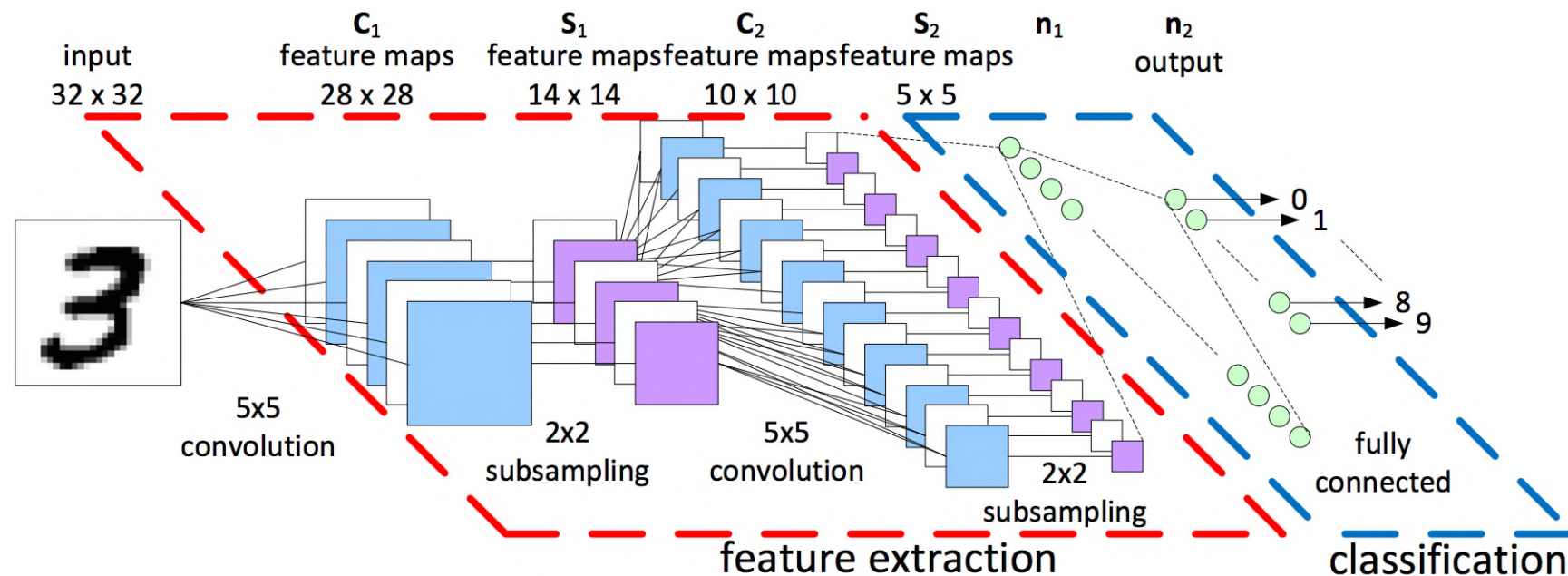
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs)
- Transformers
- Large Language Models (LLMs)
- Etc.

Why Different DNN Models?

- Different DNNs (tools) for different applications.
- Each DNN has its own unique features to address unique problem.

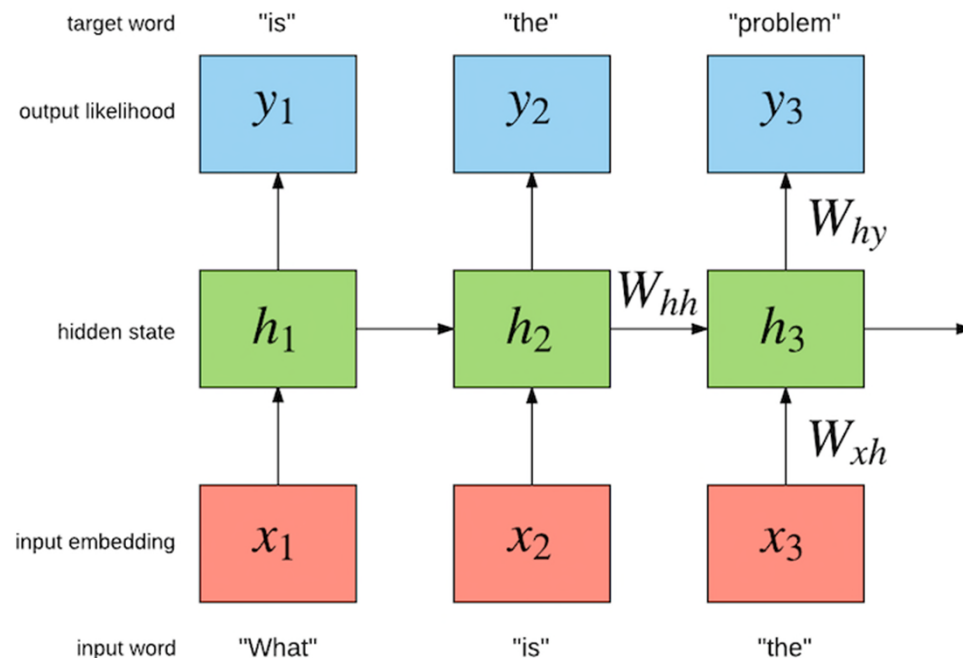
Convolutional Neural Network (CNN)

- Consist of deep layers to extract progressively higher-level abstraction features.
- Commonly used in classification and regression applications.



Recurrent Neural Network (RNN)

- A type of neural network that specializes in processing sequences.
- Commonly used in applications involving time-series and state-series prediction and modelling.
- Example applications: stock price prediction, language translation.



Transformer

- A type of network that uses attention mechanism to process input sequence in parallel.
- Good at modelling long-range dependency.
- Achieve state-of-the-art performance in many vision and NLP applications.

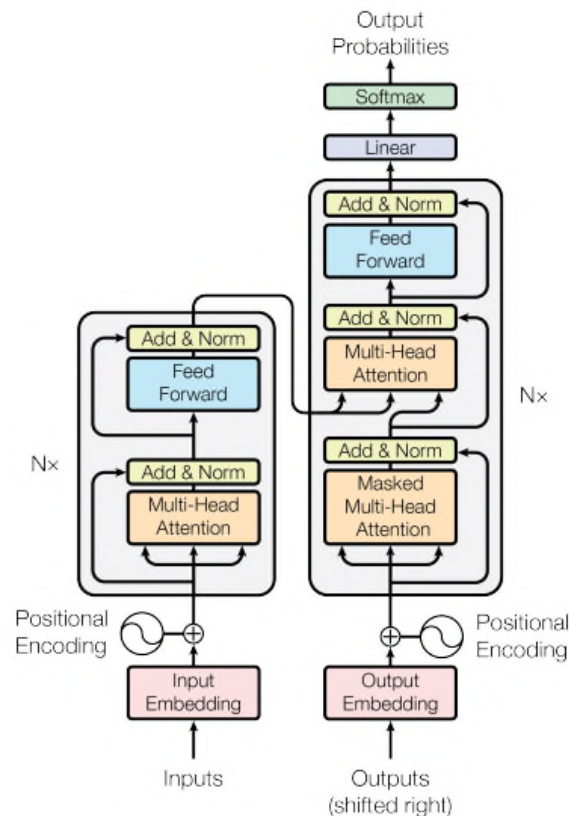
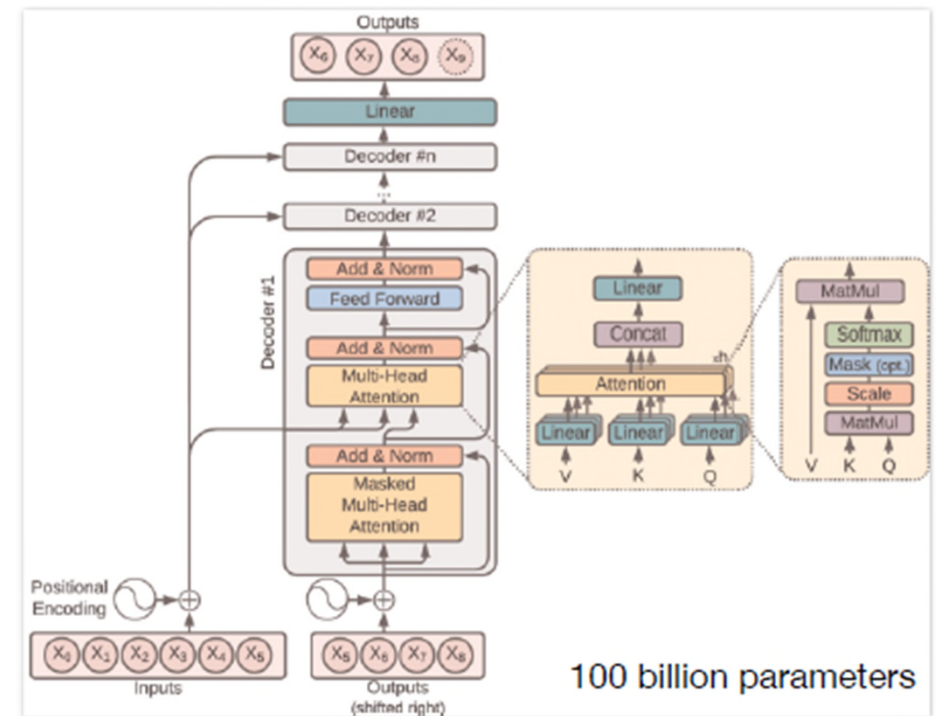
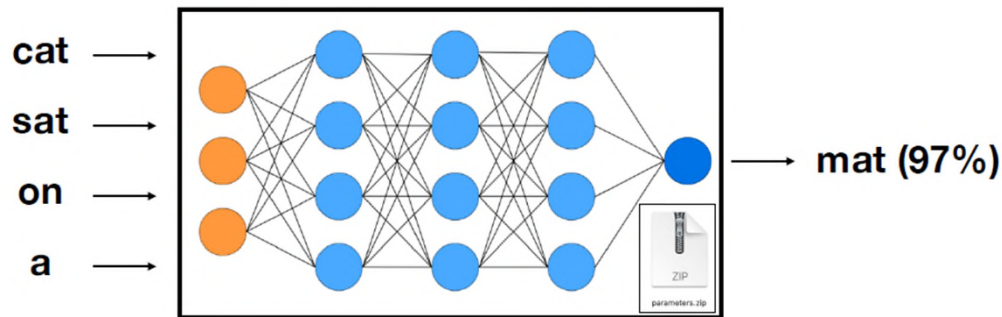


Figure 1: The Transformer - model architecture.

Large Language Models (LLMs)



Foundation Models (FMs)

- FMs are models that are trained on large scale broad data that can be adapted (finetuned) to a wide range of downstream tasks / applications.
- Examples: LLMs (e.g., GPT), Vision-Language Models (VLMs) (e.g., CLIP).

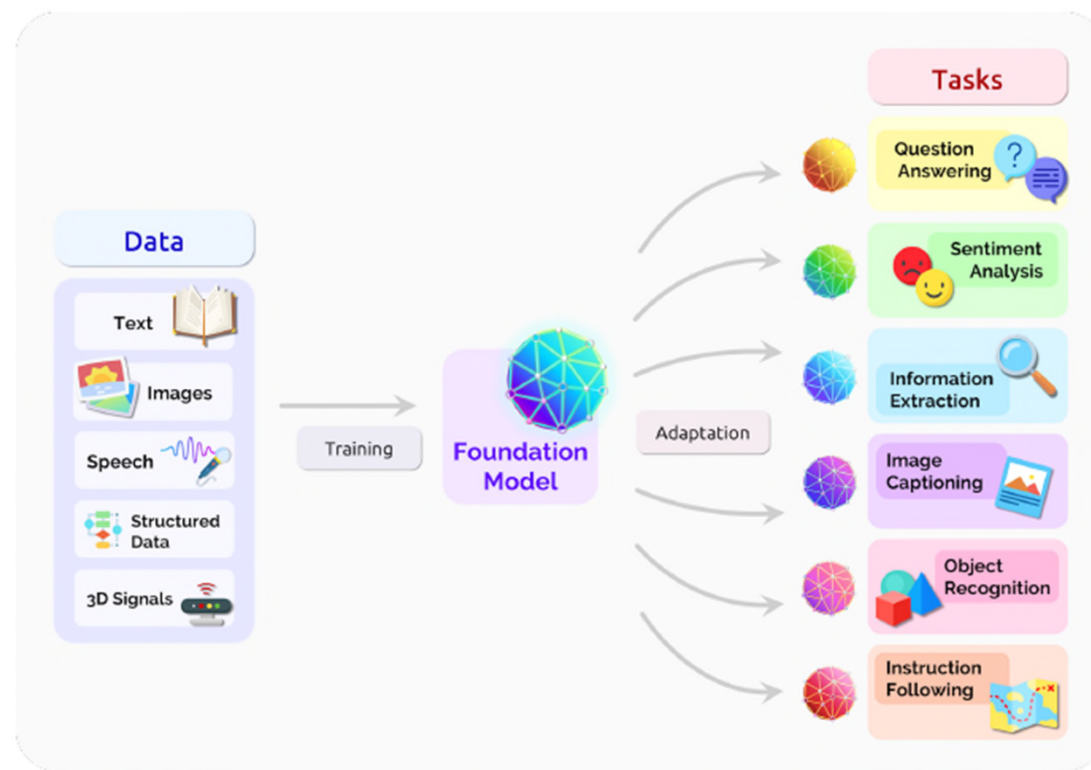
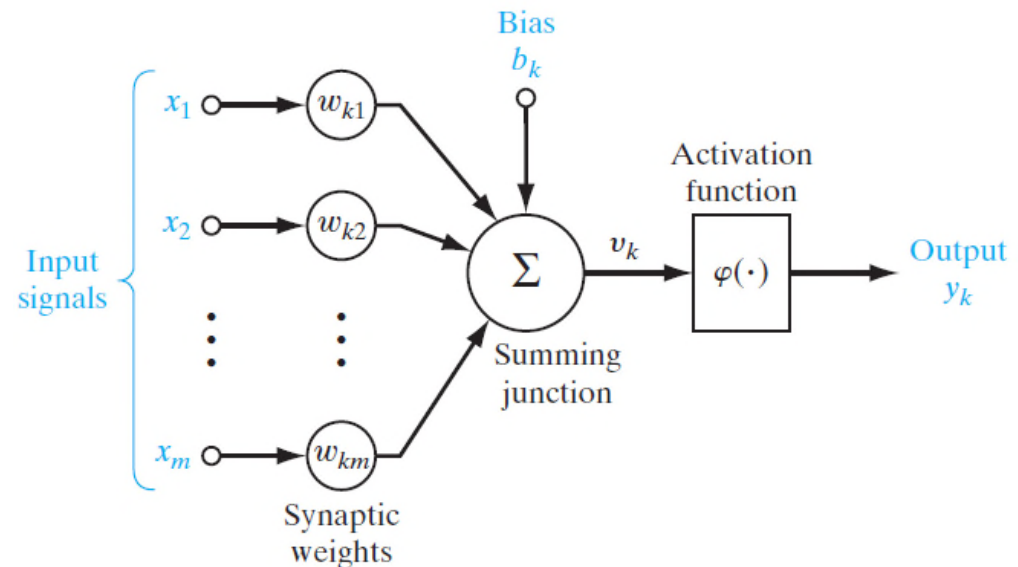
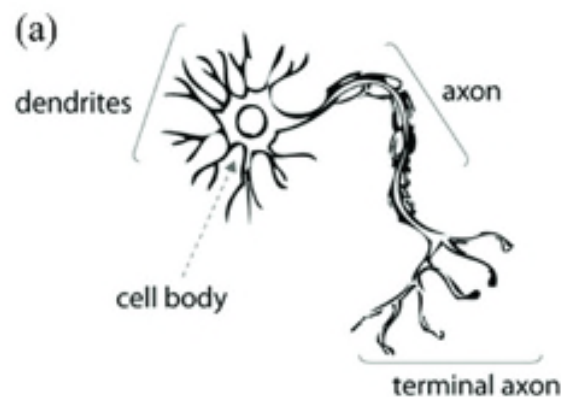


Fig. 2. A foundation model can centralize the information from all the data from various modalities. This one model can then be adapted to a wide range of downstream tasks.

Linear Classifiers

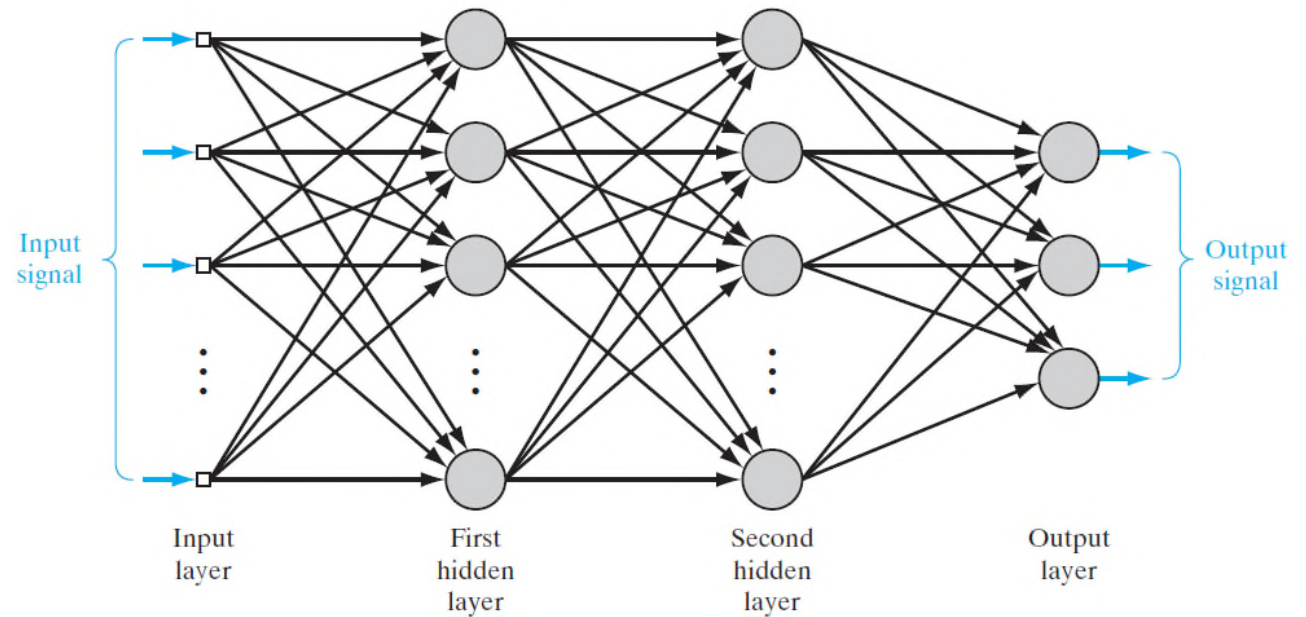
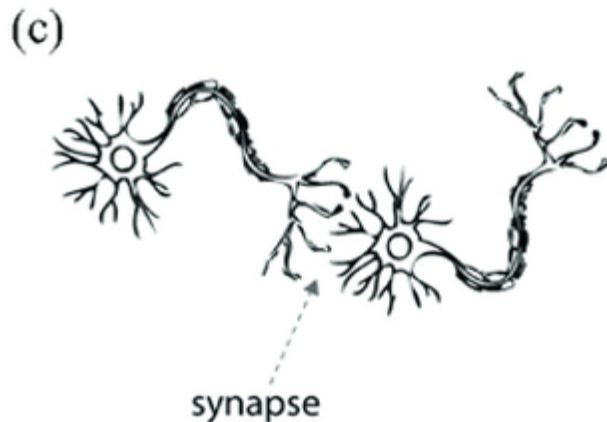
Biological Neural Network

- Multiple neurons interconnected together to form biological neural network.
- Perceptron model: artificial model of a biological neuron.



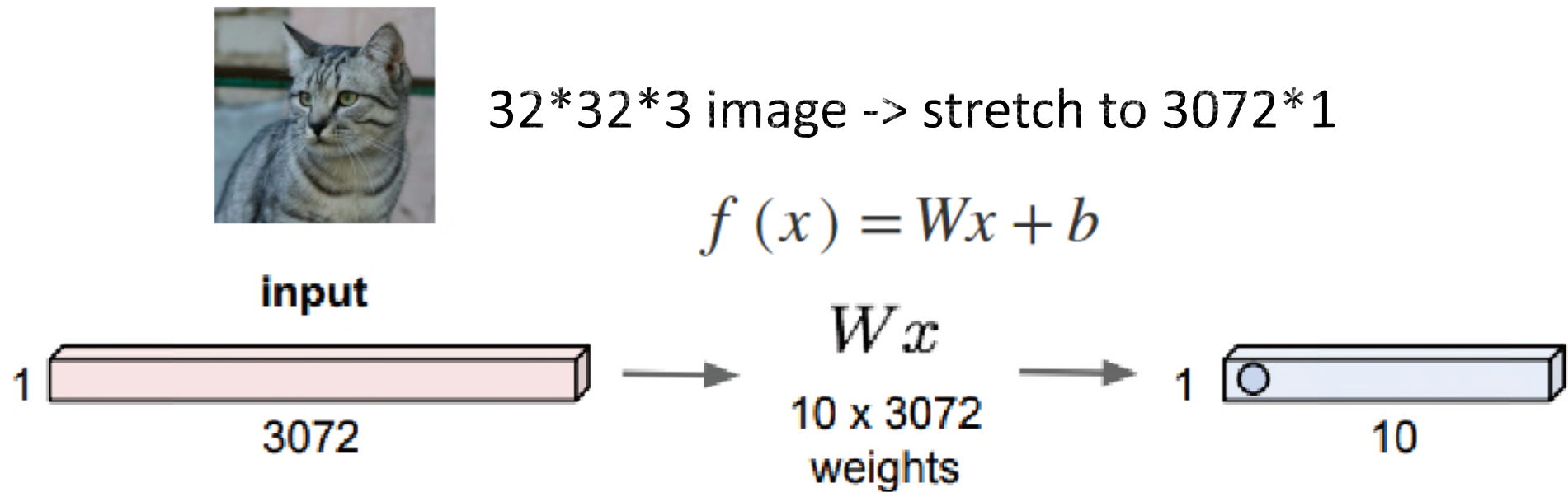
Multilayer Perceptron (MLP)

- MLP is consisted of interconnection of perceptrons.
- Also known as Feed-Forward (FF) or dense network.

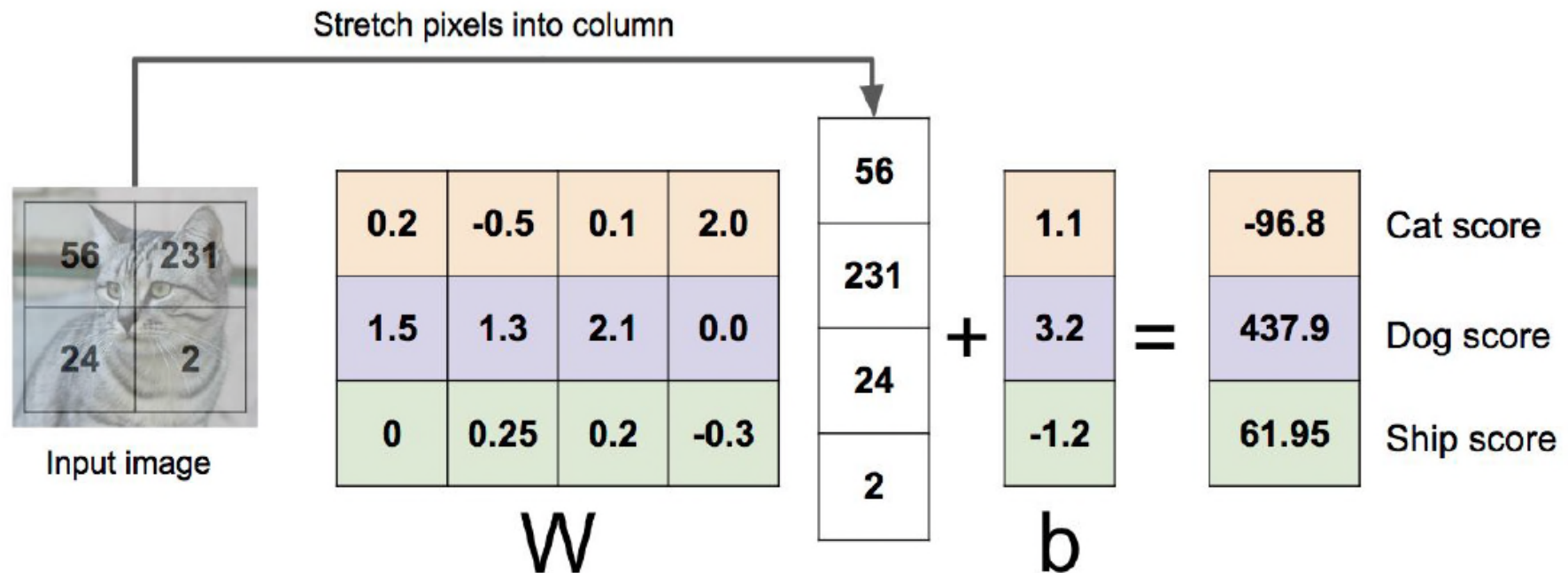


Linear Classifier

- Linear classifier
 - Make a classification decision based on a linear combination of input data
- E.g., linear classifier for 10 class labels (e.g., animal classes)



Algebraic Viewpoint



Loss Function / Error Function

- How do we determine **W** and **b**?
- We need a loss function (error measurement) which is a metric/distance between predicted values and output target values (teachers) during training.

Loss Function

- The loss function will depend on what type of problems we are addressing.
- Two common problems:
 - Regression
 - Target output is continuous value.
 - E.g., share price prediction, rain fall estimation, etc.
 - Classification
 - Predict the label/category of input.
 - Target output is discrete (from a set of possible labels).
 - E.g., cancer classification (binary), face recognition (multi-class).

Common Regression Loss Functions

- Square loss:

$$L(x, y) = \sum_i (y_i - f(x_i))^2$$

- Mean Square Error (MSE):

$$MSE = \frac{1}{N} \sum_i (y_i - f(x_i))^2$$

- Mean Absolute Error (MAE):

$$MAE = \frac{1}{N} \sum_i |y_i - f(x_i)|$$

x and y are the input data and target output value, $f(x)$ is the network predicted output.

Common Classification Loss Function

- Softmax loss:
 - Cross-entropy loss with softmax normalization.

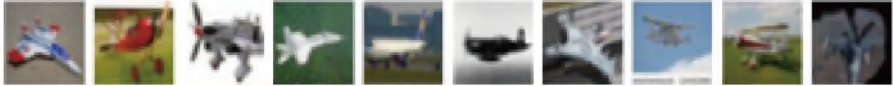
$$p_j = \frac{e^{z_j}}{\sum_k e^{z_k}}, \text{ where } z_j = f(x_j)$$

$$L = -\sum_j y_j \log_e p_j$$

Softmax Loss Example

- The output scores of a linear classifier for an input cat image are given as follows. What is the softmax loss for this training data?

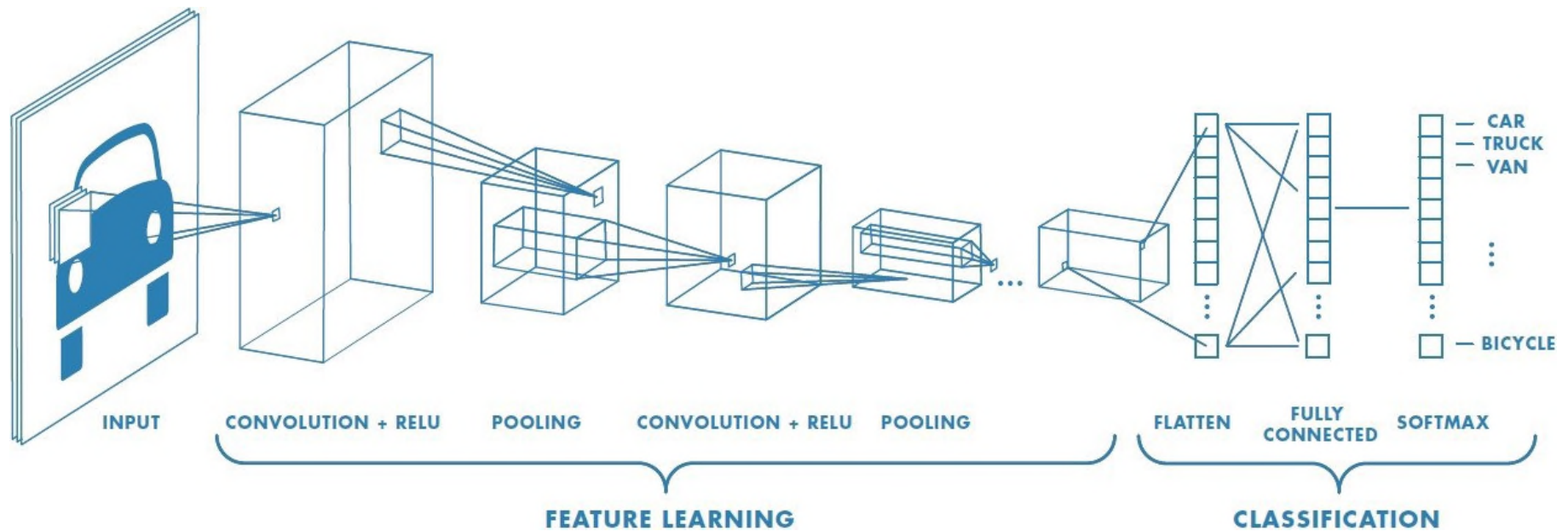
$$p_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

		Predicted output Scores (z)	Ground truth Output Labels (y)	Normalized probabilities (p)
airplane		0.7	0	0.109
automobile		0.1	0	0.060
bird		1.6	0	0.269
cat		2.2	1	0.489
deer		0.3	0	0.073

$$L = -\sum_j y_j \log p_j = 0.715 \quad (\text{Note: The log is natural log namely } \ln)$$

CNN Architecture

CNN Architecture



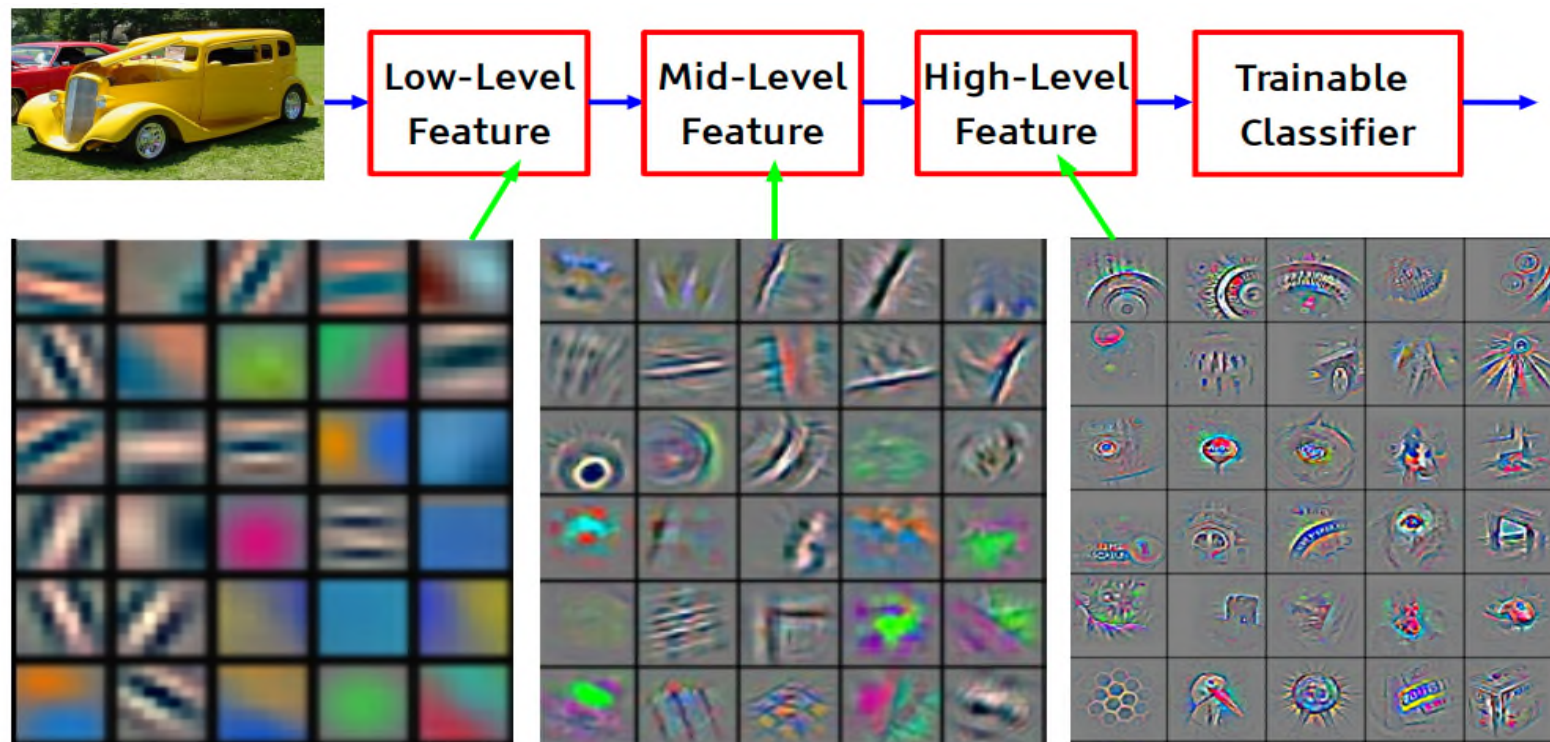
CNN Architecture

- Convolutional Layer
- Activation Function Layer
- Pooling Layer
- Fully-Connected (FC) Layer / Linear Layer
- Softmax Layer

Convolutional Layer

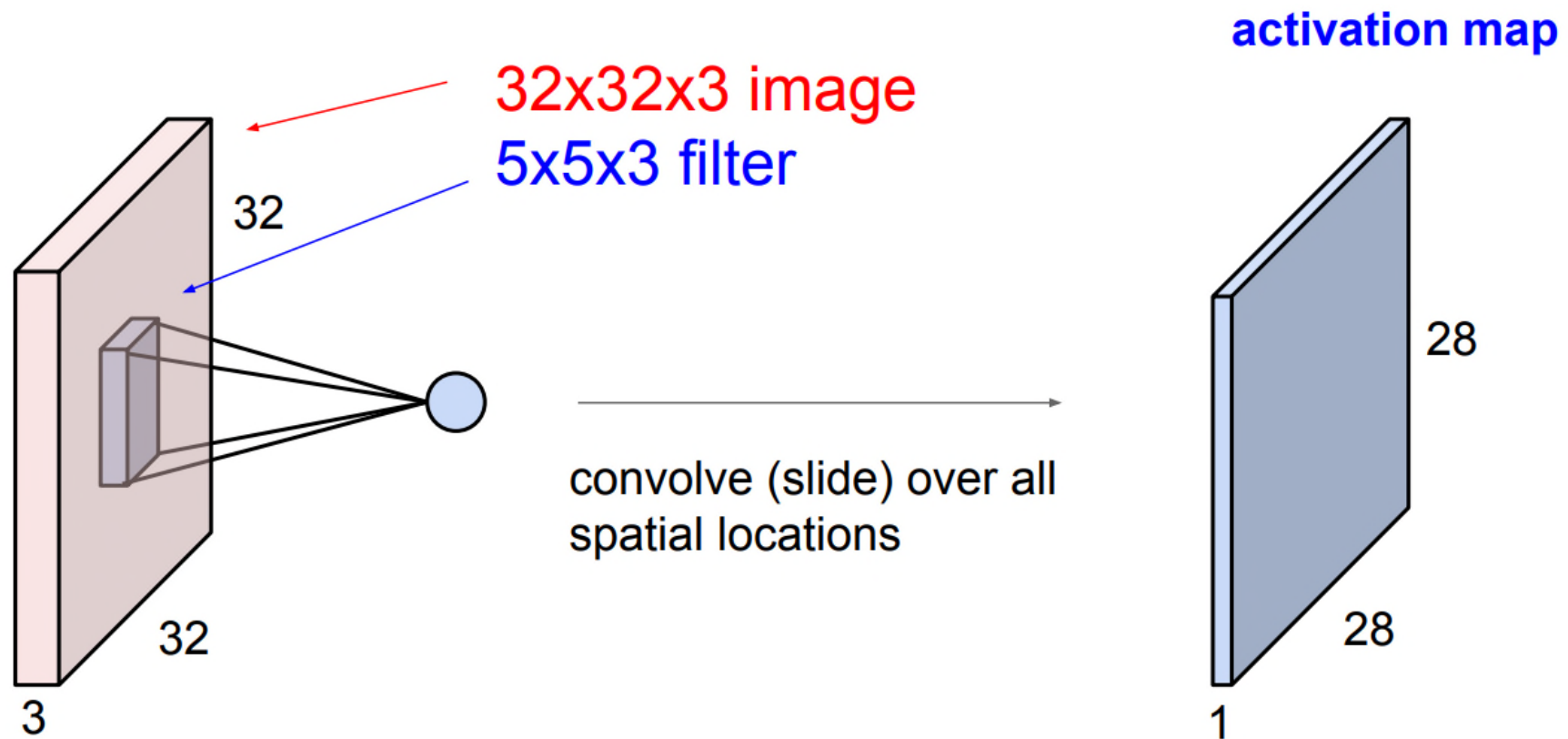
- Extract features in a hierarchical manner.
- Early layers extract low-level features whereas later layers extract high-level features.
- Reduce parameters to be trained by sharing weights through convolution kernel.

Convolutional Layer Feature Visualization



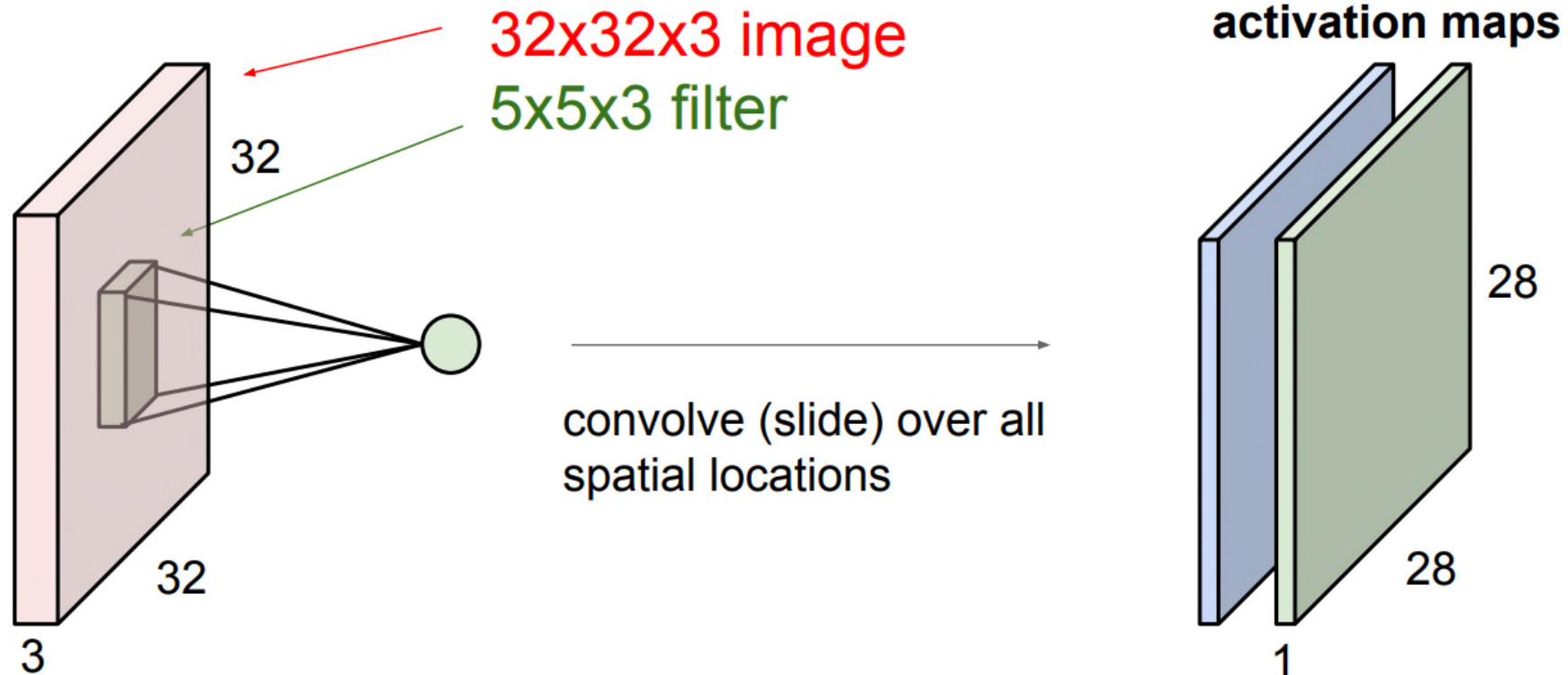
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Convolutional Layer



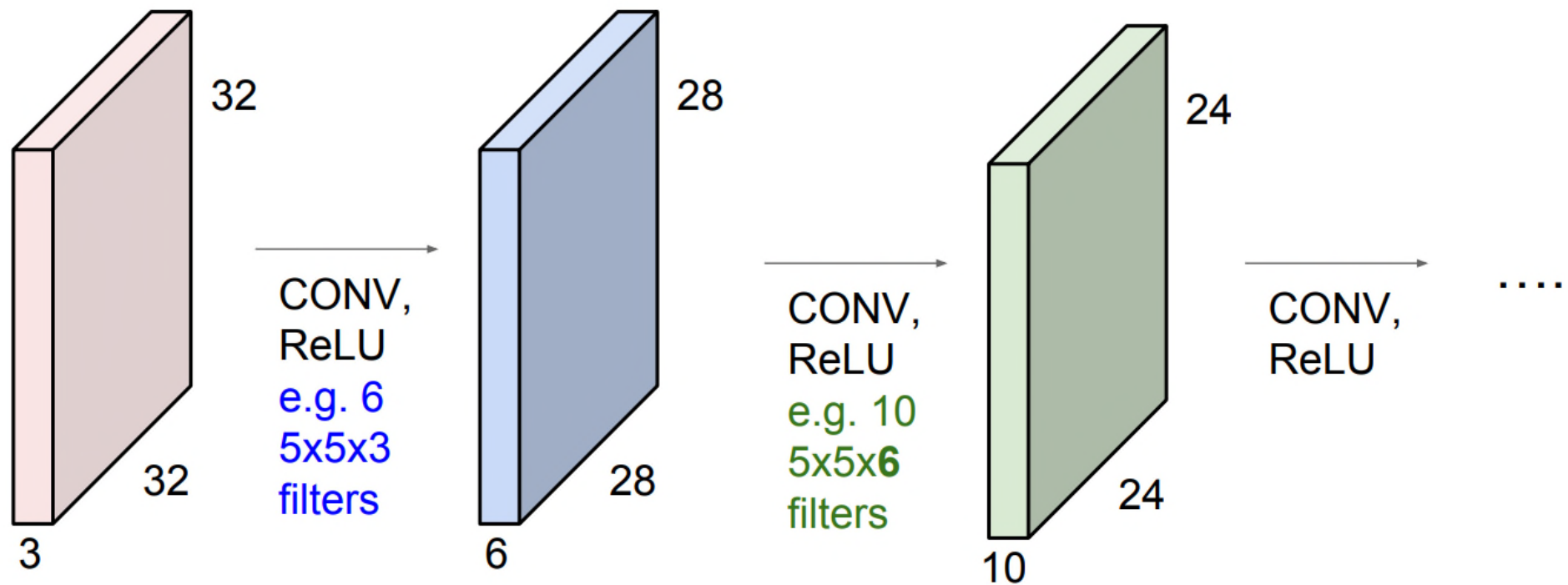
Convolutional Layer

consider a second, **green** filter

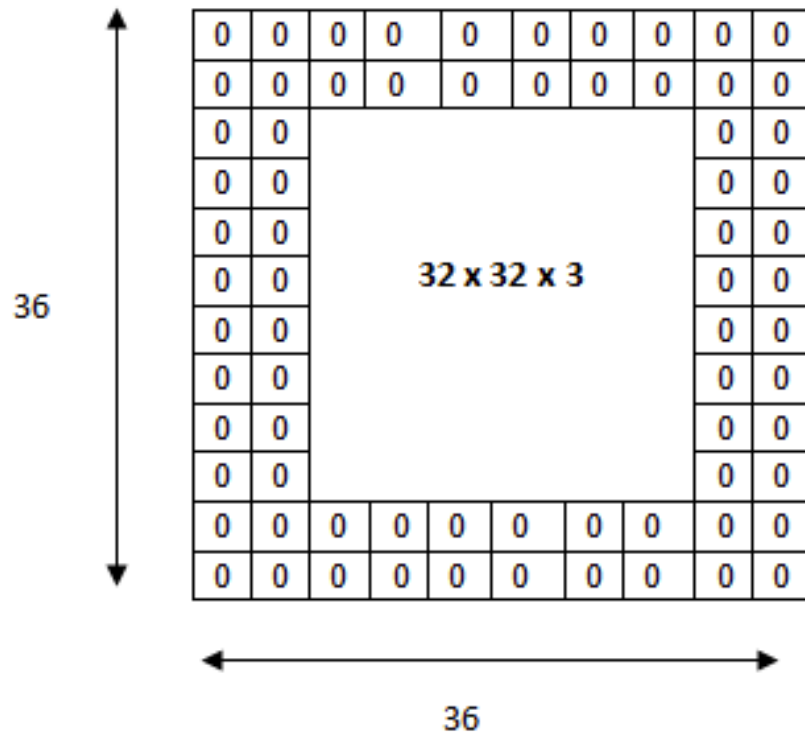


Convolutional Layers

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

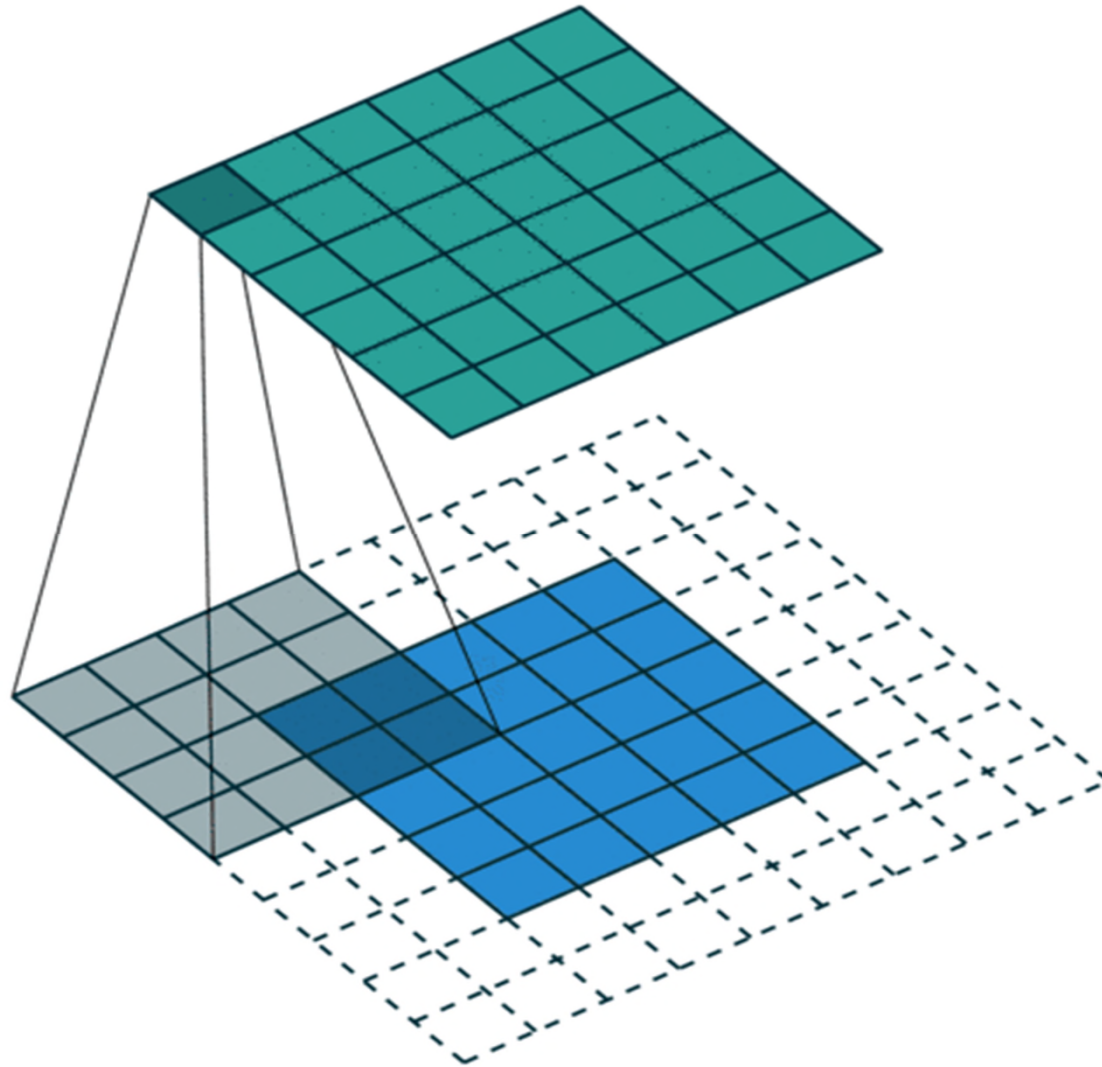


Padding



The input volume is $32 \times 32 \times 3$. If we pad two borders of zeros around the volume, this gives us a $36 \times 36 \times 3$ volume. Then, when we apply our conv layer with three $5 \times 5 \times 3$ filters and a stride of 1, we will get a $32 \times 32 \times 3$ output volume.

Padding



Padding + Convolution

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+

+

+ 1 = -25

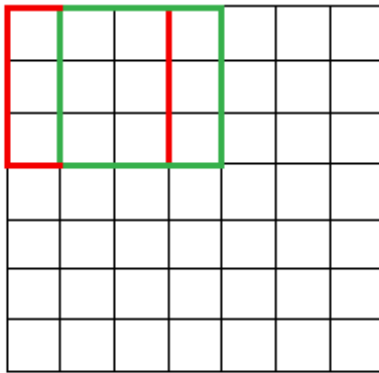
↑
Bias = 1

Output

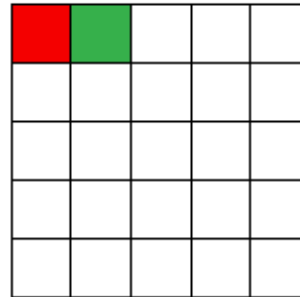
-25				...
				...
				...
				...
...	...	...	...	...

Stride

7 x 7 Input Volume

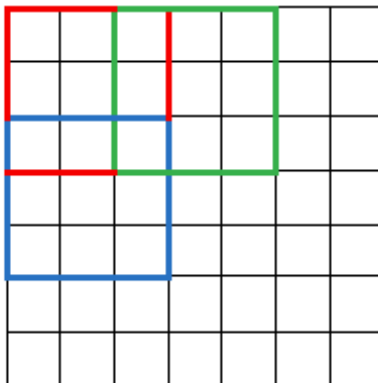


5 x 5 Output Volume

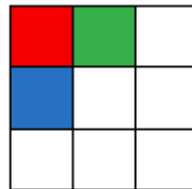


3×3 filter with
stride 1

7 x 7 Input Volume



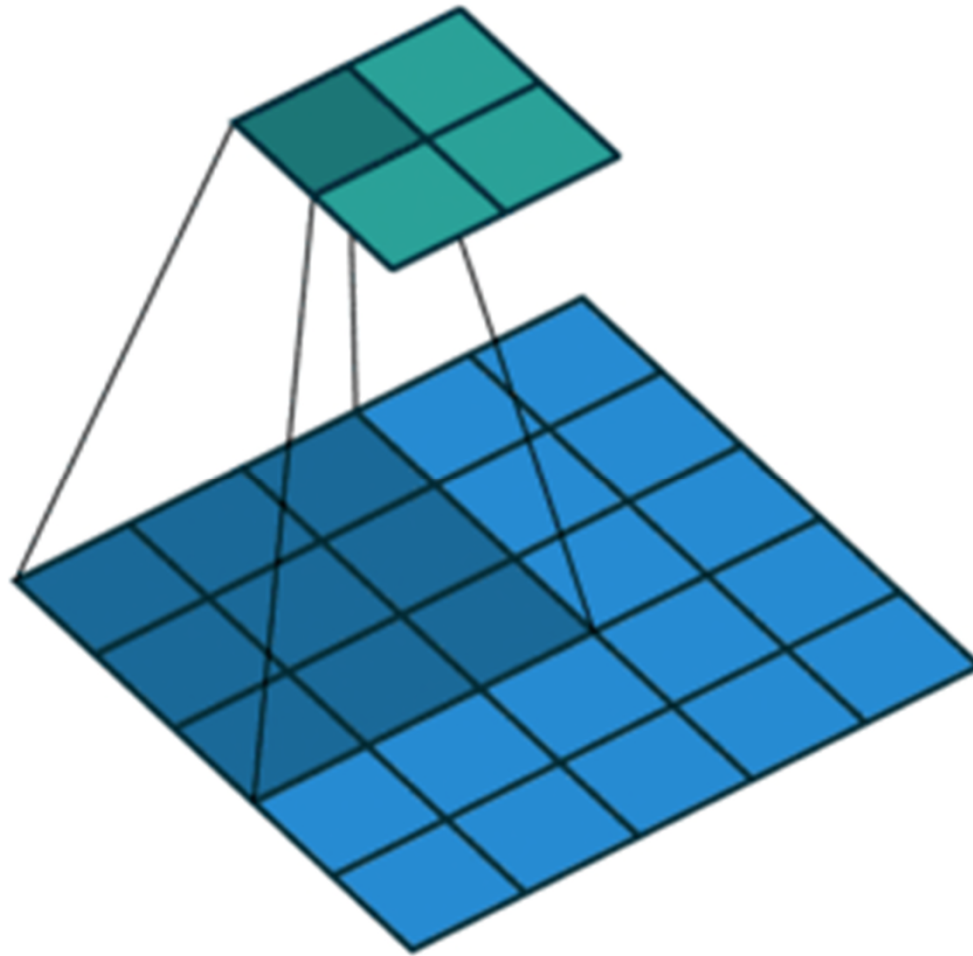
3 x 3 Output Volume



3×3 filter with
stride 2

Stride

- Stride = 2:



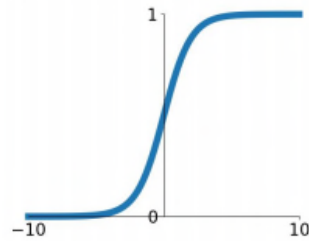
Activation Layer

- Perform element-by-element nonlinear activation function mapping.
- RELU is one of most popular activation functions.
- Often combined with convolution layer.

Activation Layer

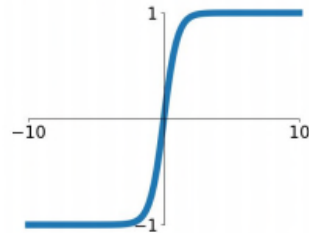
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



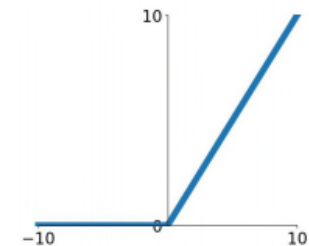
tanh

$$\tanh(x)$$



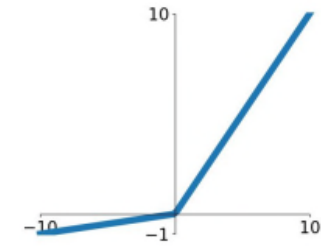
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

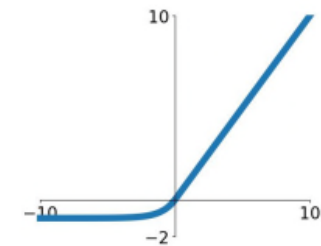


Maxout

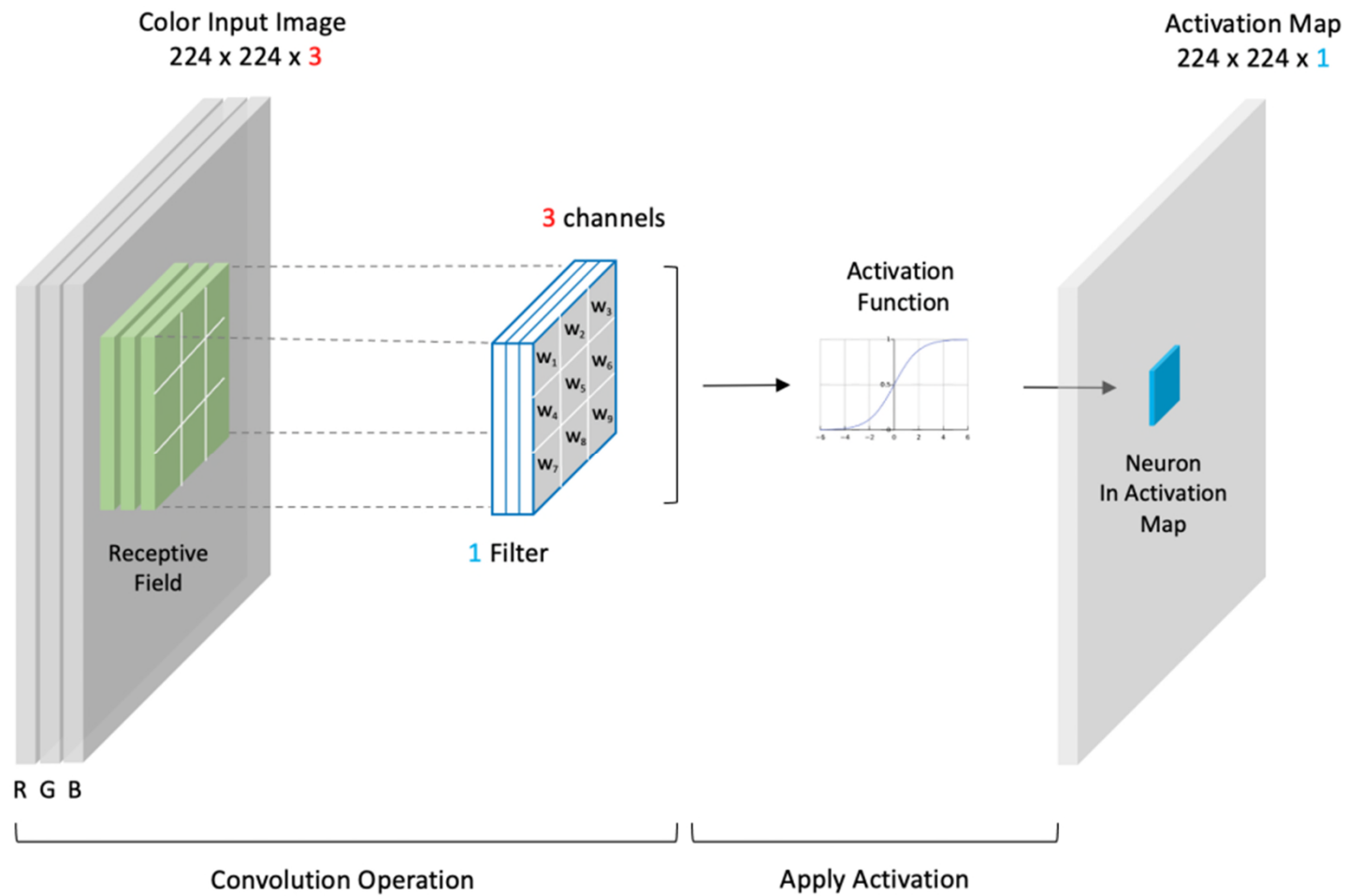
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Conv + RELU

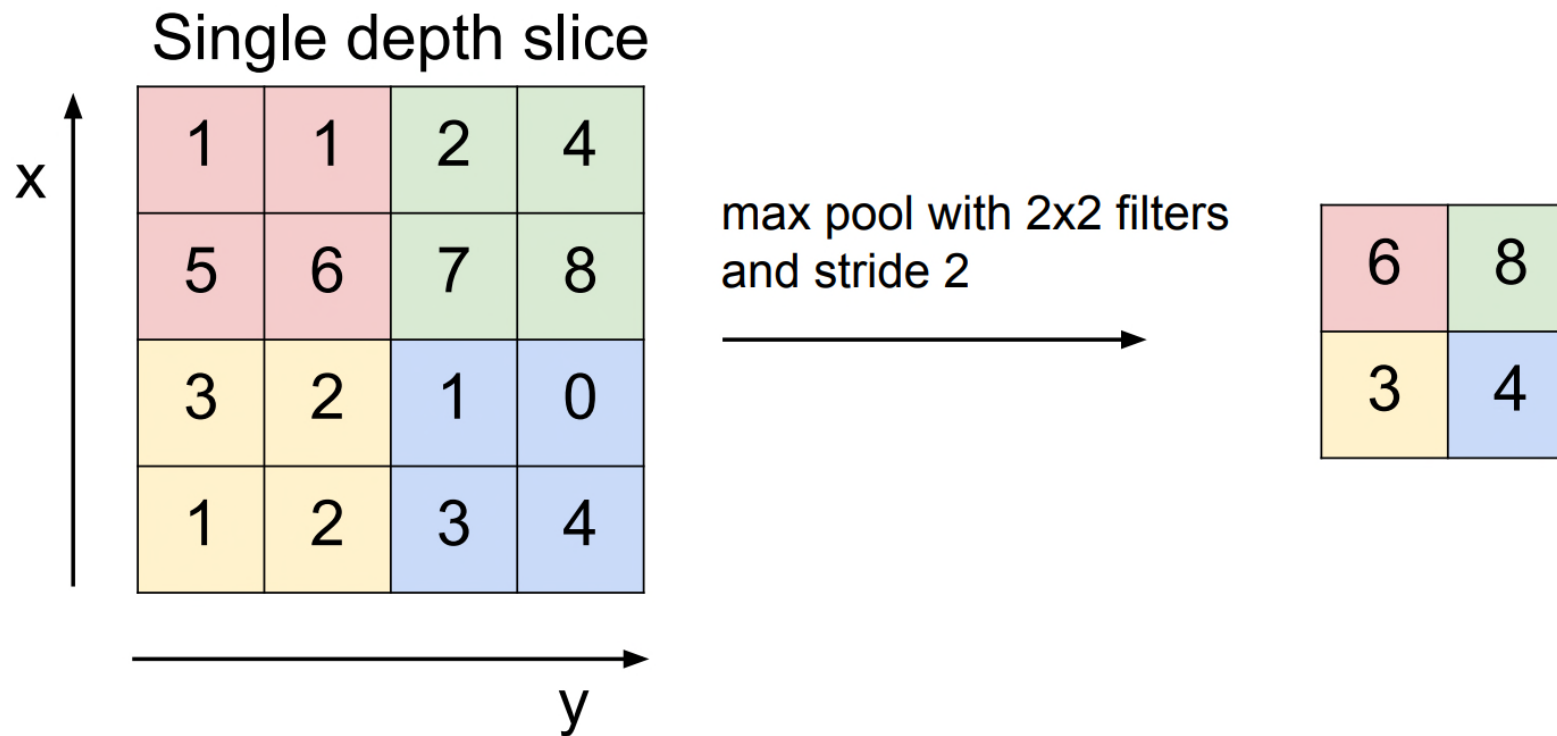


Pooling Layer

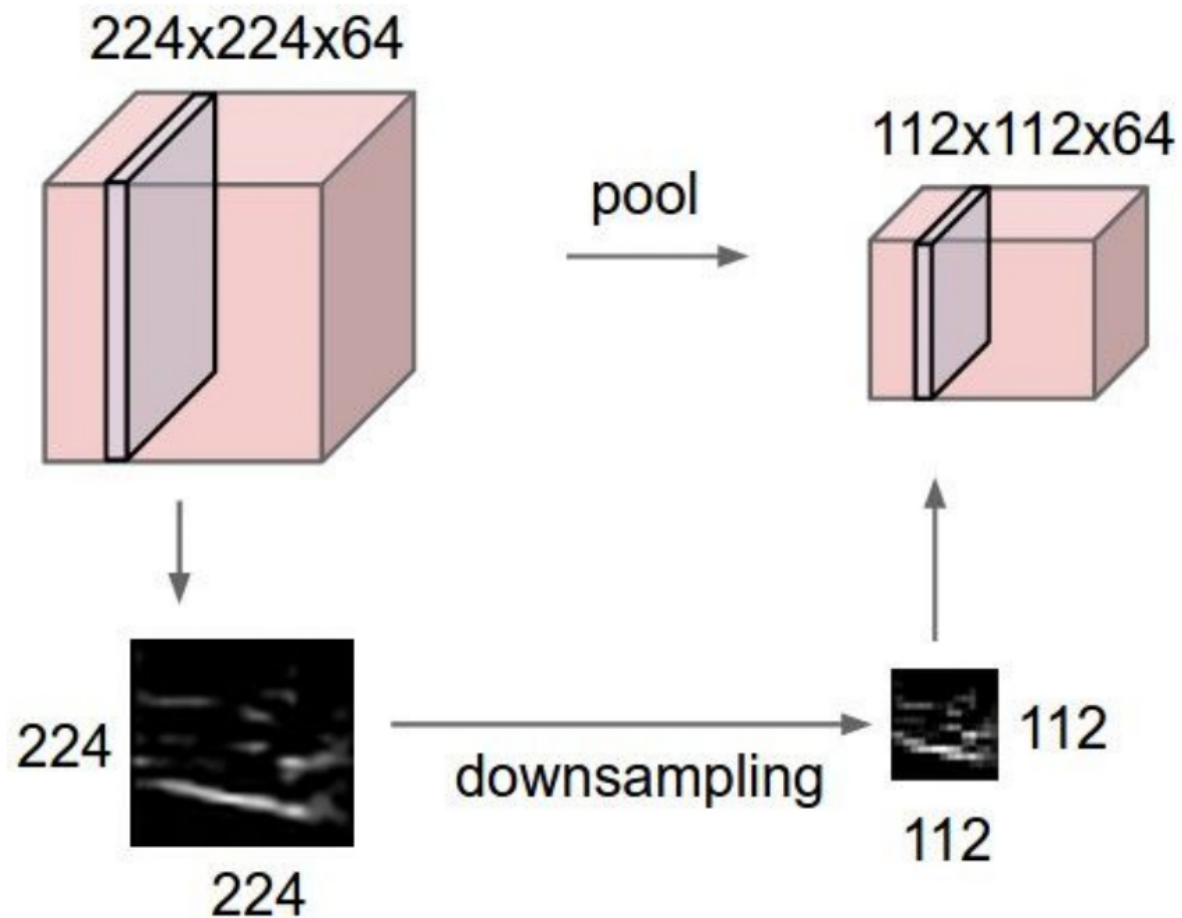
- Reduce activation map dimension, hence reducing computation and storage requirement.
- Common pooling operation: max pooling, average pooling.
- Each channel of activation/feature maps operate independently.

Pooling Layer

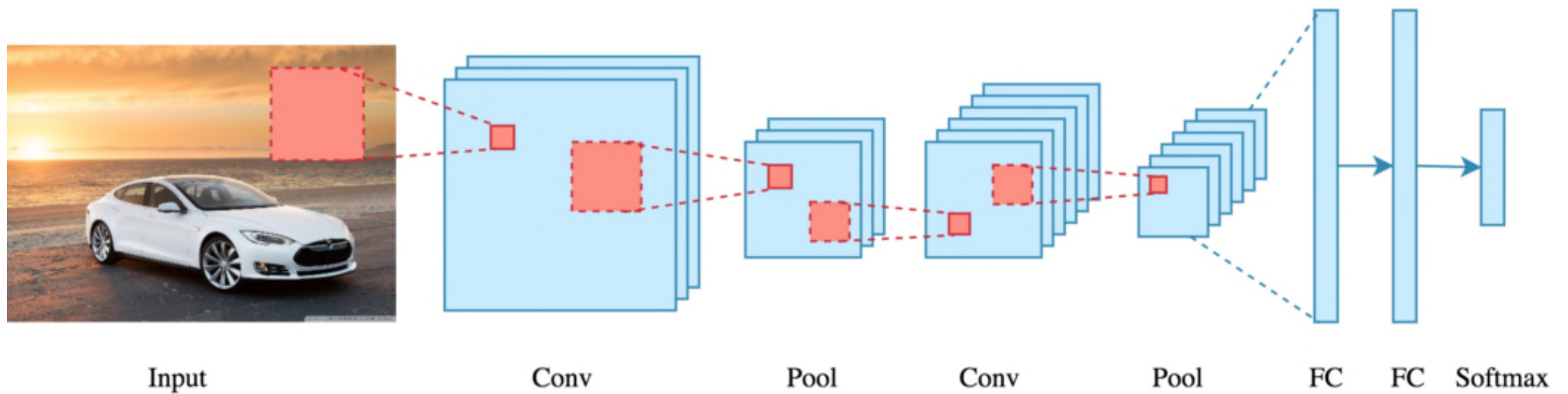
MAX POOLING



Pooling Layer



FC Layer



Fully Connected (FC) Layer

- All nodes in one layer are connected to all nodes in the next layer.
- FC layer feature vector can be used as feature (also known as embedding) to represent input image.
- FC layer behaves like a linear layer for classification.

FC Layer

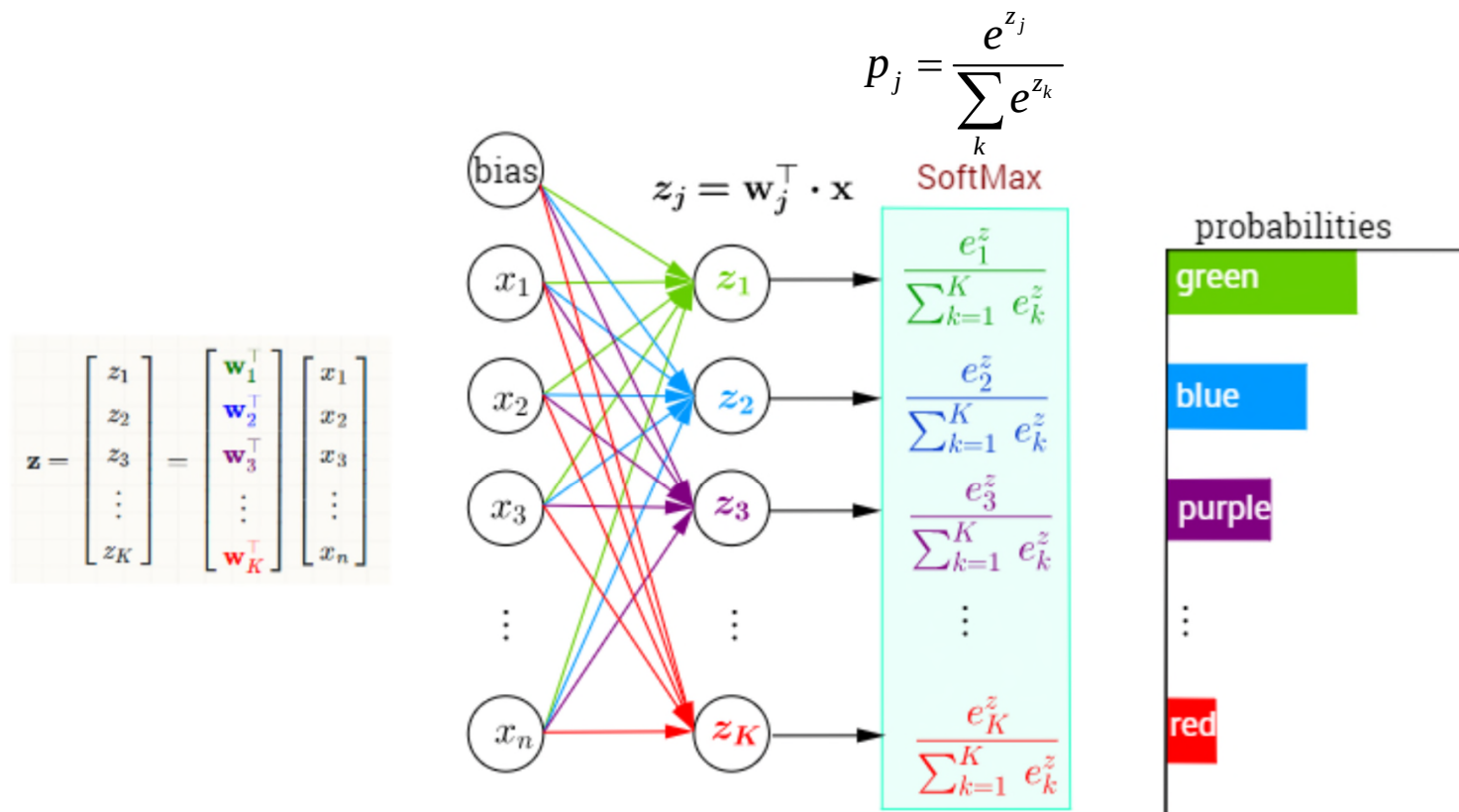
32x32x3 image -> stretch to 3072 x 1

Each neuron
looks at the full
input volume



Softmax Layer

- Map the output scores (logits) from last FC layer into probabilities for classification problem.
- Softmax loss is computed based on the probabilities.



Sample Pytorch CNN Implementation

```
class Net(nn.Module):
```

```
    def __init__(self):
        super(Net, self).__init__()
        # 1 input image channel, 6 output channels, 3x3 square convolution
        # kernel
        self.conv1 = nn.Conv2d(1, 6, 3)
        self.conv2 = nn.Conv2d(6, 16, 3)
        # an affine operation:  $y = Wx + b$ 
        self.fc1 = nn.Linear(16 * 6 * 6, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

Conv Layer

Fully-connected Layer

Layer
Definition

```
    def forward(self, x):
        # Max pooling over a (2, 2) window
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
        # If the size is a square you can only specify a single number
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.view(-1, self.num_flat_features(x))
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Activation Layer

Feed Forward
Logic

Exercise: CNN

- (a) In a simple Convolutional Neural Network (CNN), an input image $\mathbf{A}$ passes through a convolutional layer, followed by an activation layer and a max pooling layer. The output from the max pooling layer is then used for further processing. The grayscale image $\mathbf{A}$ is given by:

$$\mathbf{A} = \begin{bmatrix} 4 & 0 & 1 \\ 4 & 0 & 2 \\ 0 & 2 & 2 \end{bmatrix}.$$

The convolutional layer has the following settings: the current filter is given by $\mathbf{F}$ below, the amount of zero padding at each side of the image is 1, and the stride both horizontally and vertically is 2.

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

The activation function used in the activation layer is given by a sigmoid function:

$$\sigma(x) = \frac{1}{1+e^{-x}}.$$

The max pooling layer uses 2×2 max pooling with a stride of 2.

Exercise: CNN

- (i) Find the output after the convolution layer.
- (ii) Briefly discuss the effect of filter **F** when applied to the input image.
- (iii) Find the output after the activation layer.
- (iv) Find the output after the max pooling layer.
- (v) A student would like to make the following changes to the input image and the convolutional layer:
 - Change the grayscale image **A** to an RGB image **B** with a spatial dimension of 100×100 .
 - Change the channel number of the output feature maps to 6 for the new convolutional layer. Assume the spatial dimension of the filter remains the same.

Find the number of trainable parameters of the new convolutional layer after the changes. Assume no bias is used in the calculation.

(18 Marks)

Solution

Exercise: CNN FC and Softmax Layers

An input feature vector $\mathbf{x}$ passes through a Fully Connected (FC) layer of a CNN with the following settings:

$$\text{Input: } \mathbf{x} = \begin{bmatrix} 0.3 \\ 0 \\ 0.2 \\ 0.1 \end{bmatrix}, \quad \text{Weights: } \mathbf{W} = \begin{bmatrix} 0 & 3 & 7 & 8 \\ 1 & 8 & 0 & 0 \\ 0 & 8 & 1 & 0 \end{bmatrix}, \quad \text{Bias: } \mathbf{b} = \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$$

Softmax function is then applied to the output of this FC layer.

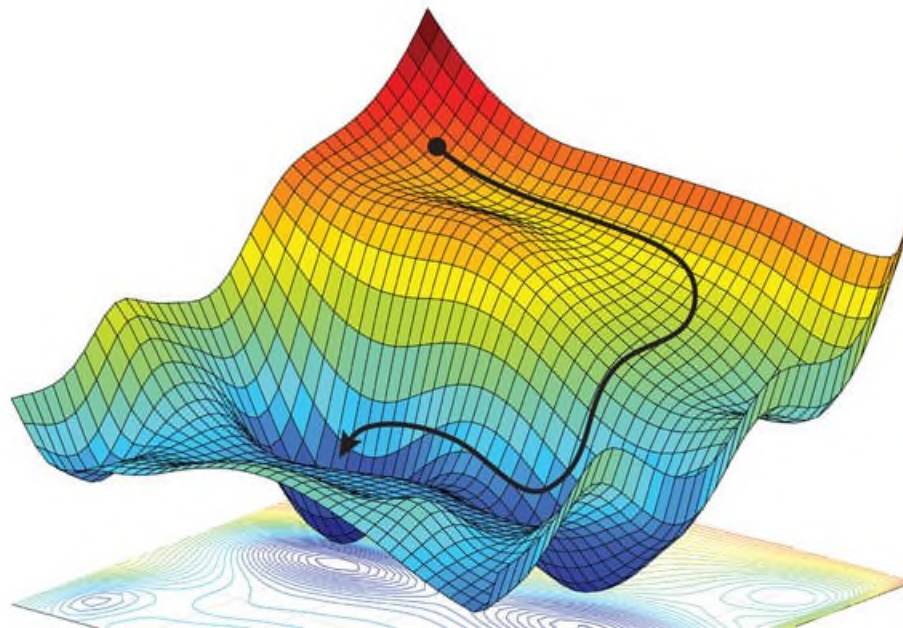
- (a) Find the output after the FC layer.
- (b) Find the output after the Softmax layer.

Solution

CNN Training & Optimization

CNN Training

- The objective is to minimize/optimize the loss function.
- Common strategy is centered to stochastic gradient descent (SGD) and its variants.
- Use computational graph to compute the gradient descent and parameter update.

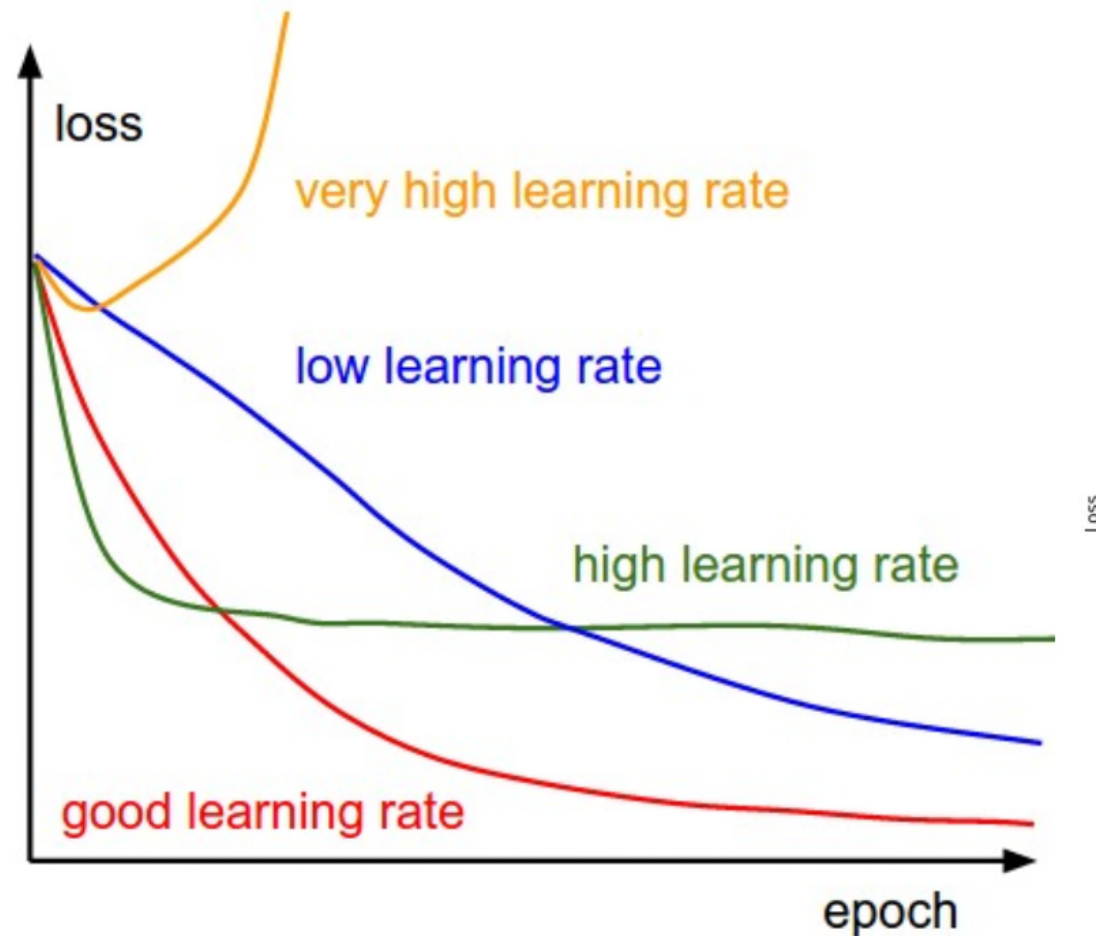


CNN Training

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W)$$

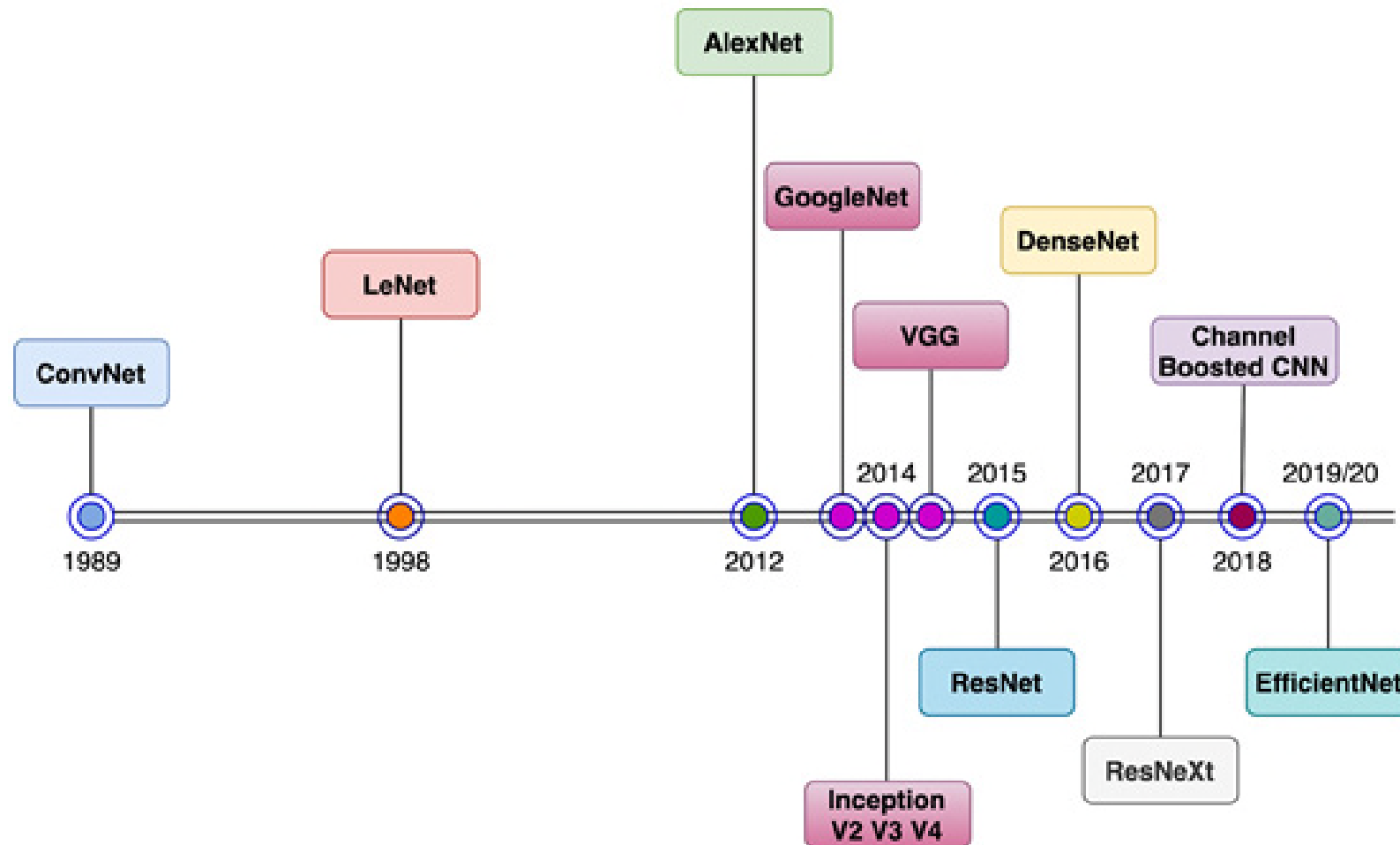
$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$

$$w_{t+1} = w_t - \alpha \nabla L(w_t)$$



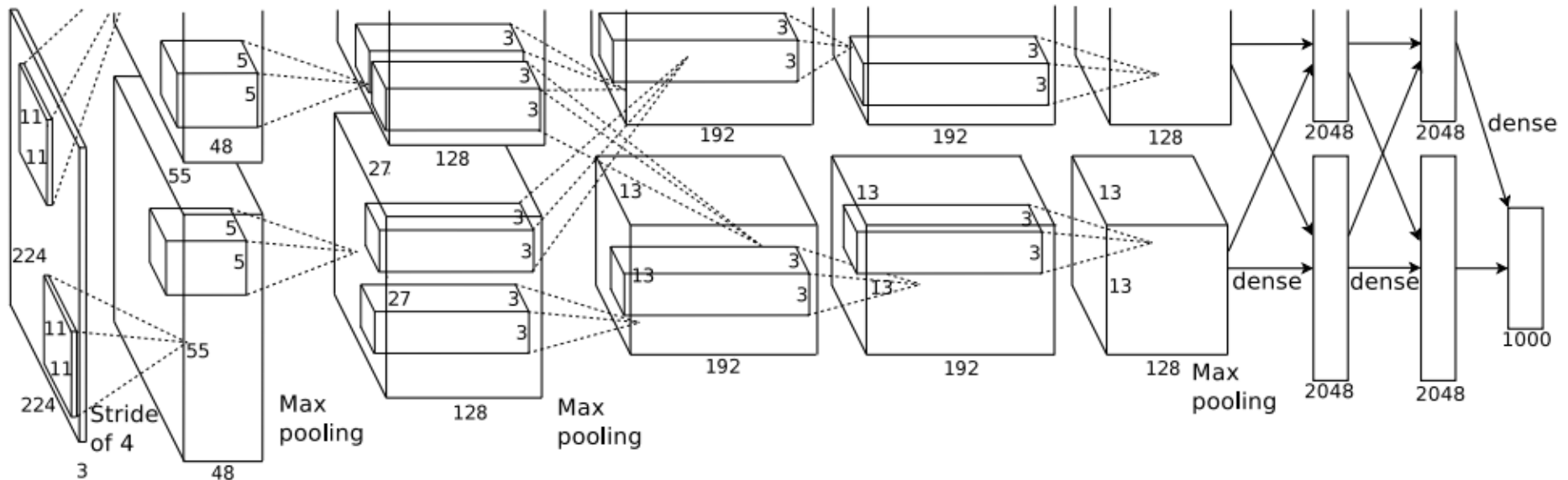
Well-Known CNN Architectures

CNN Timeline / Milestone



AlexNet

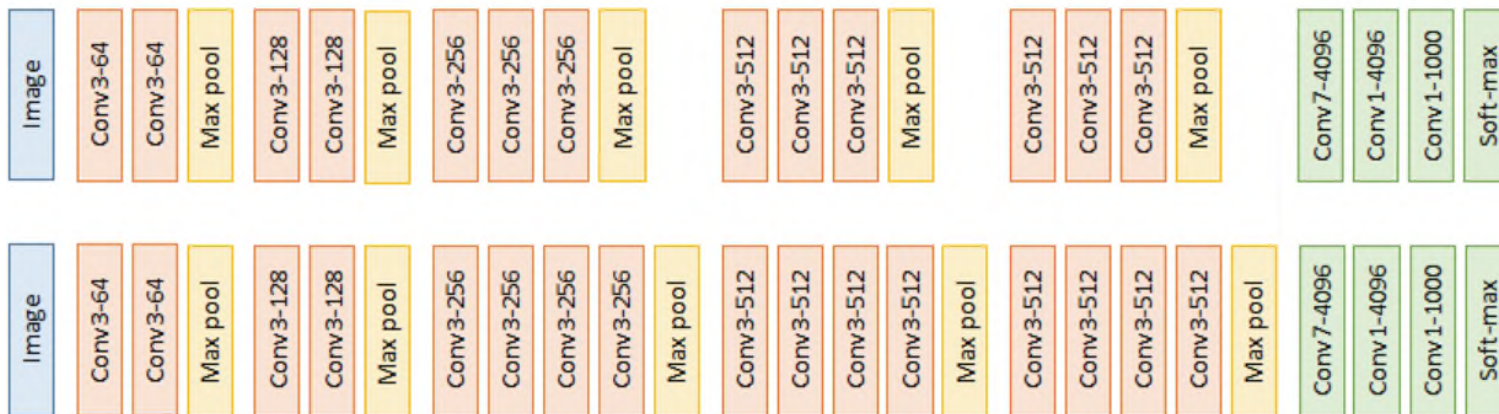
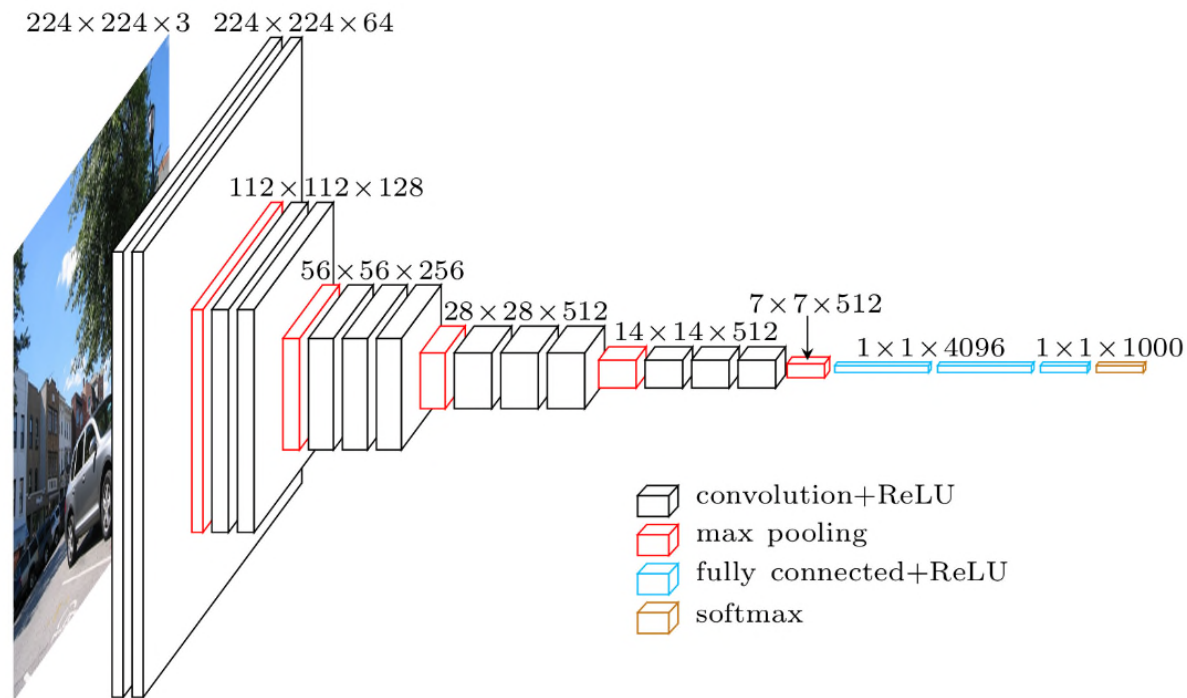
- First neural network applied on large scale image data.
- Classical structure for image classification consisting of convolution layers and fully connected layers.
- ILSVRC champion in 2012.



VGG Network

- Runner up of ILSVRC 2014.
- Elegant network architecture.
- Use deep network with small 3×3 kernels.
- Require large number of parameters.
- Two common variants: VGG-16 and VGG-19

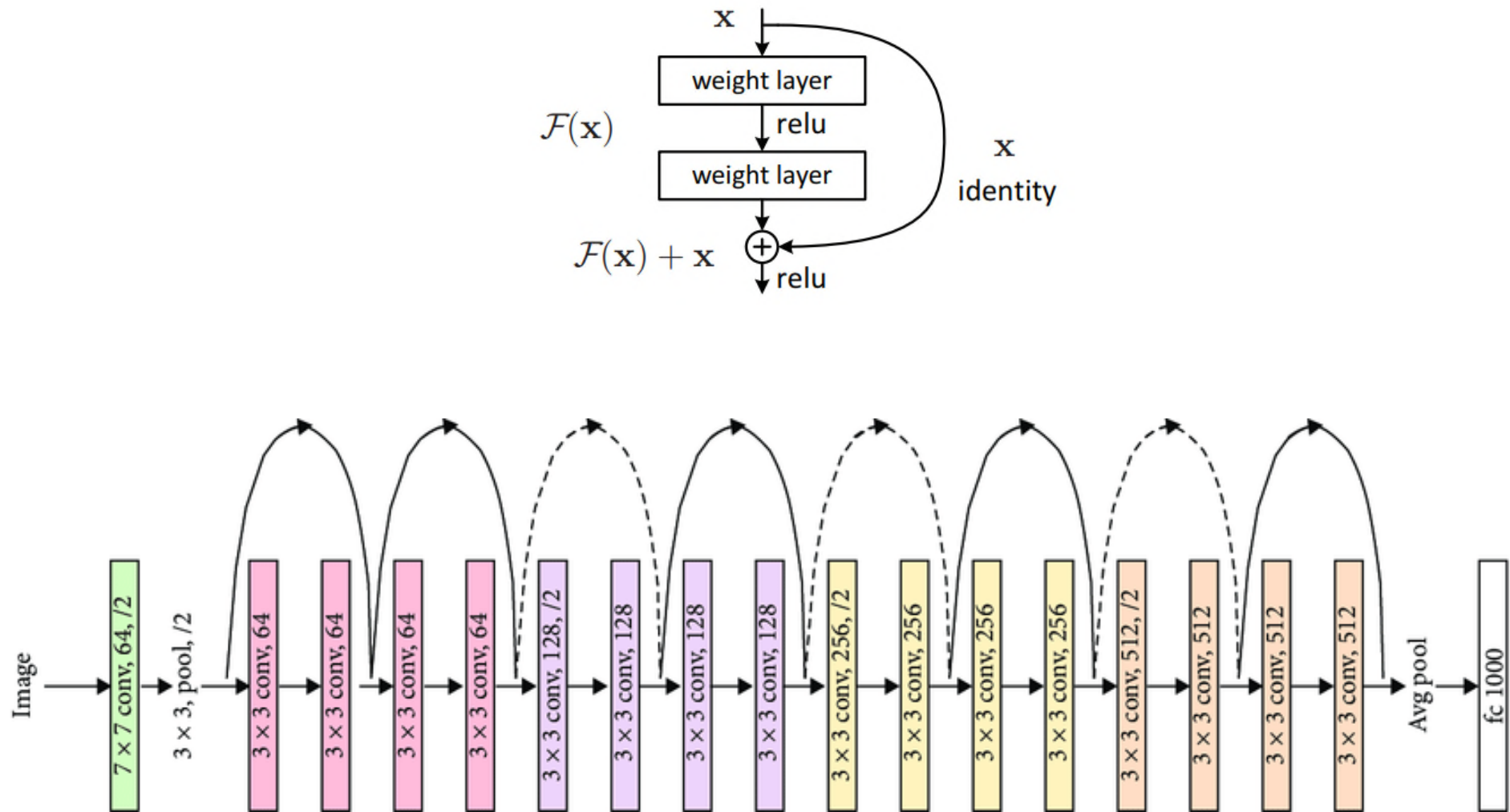
VGG Network



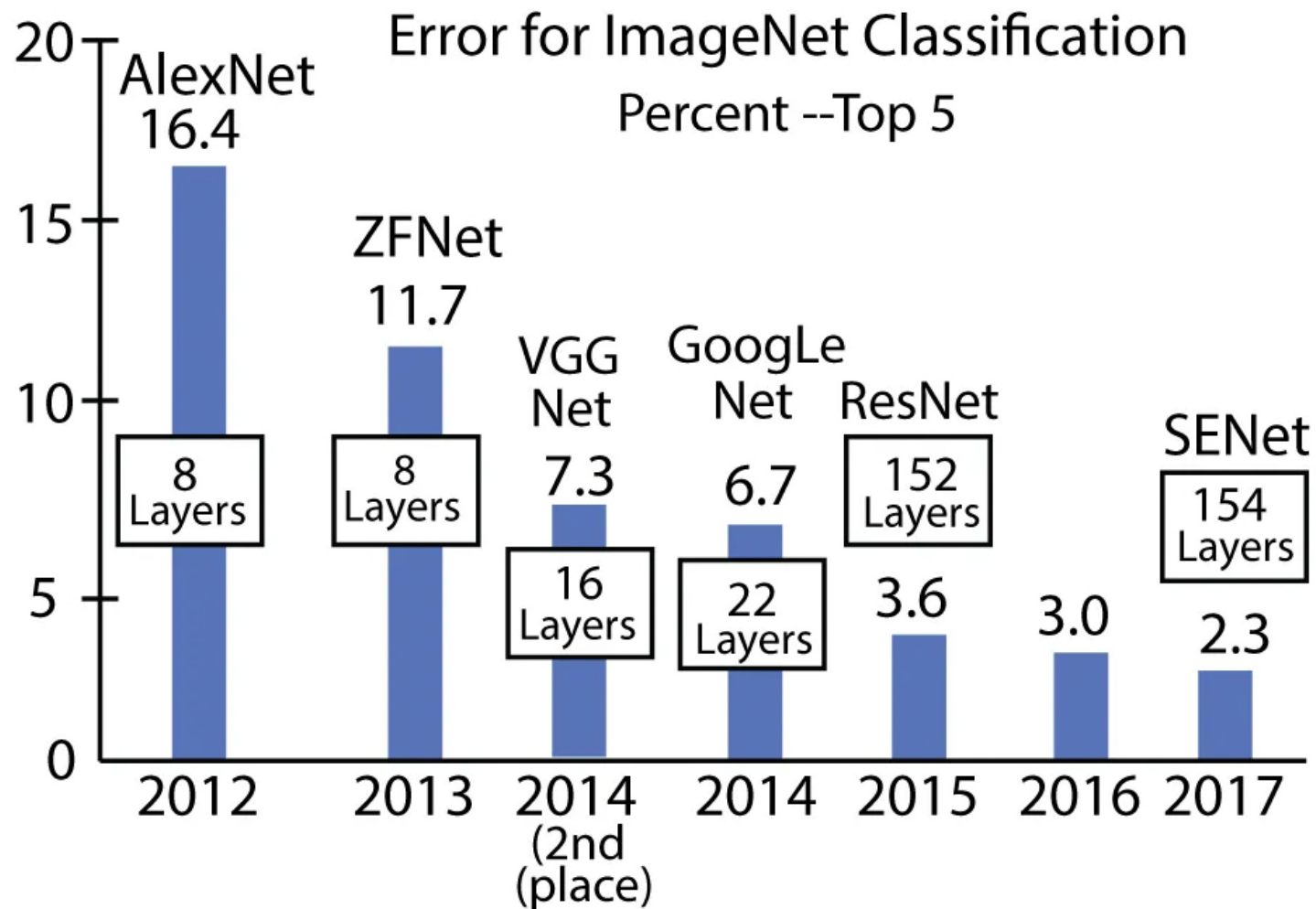
ResNet

- Winner of ILSVRC 2015
- Very deep structure
- Use residual block and highway/skip connection for better gradient backpropagation.

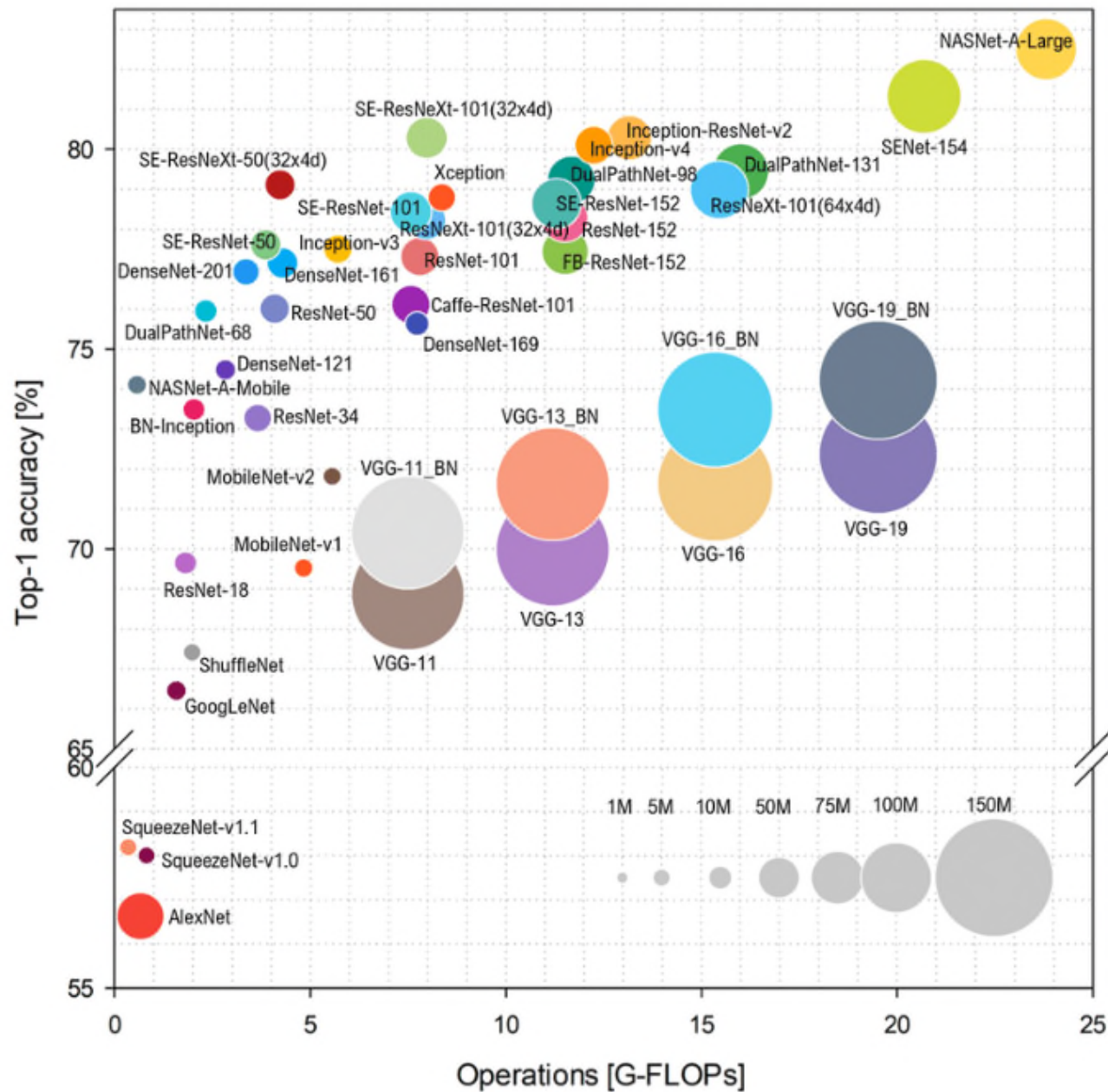
ResNet



Performance Comparison



Performance Comparison on ImageNet



Key Performance Metrics

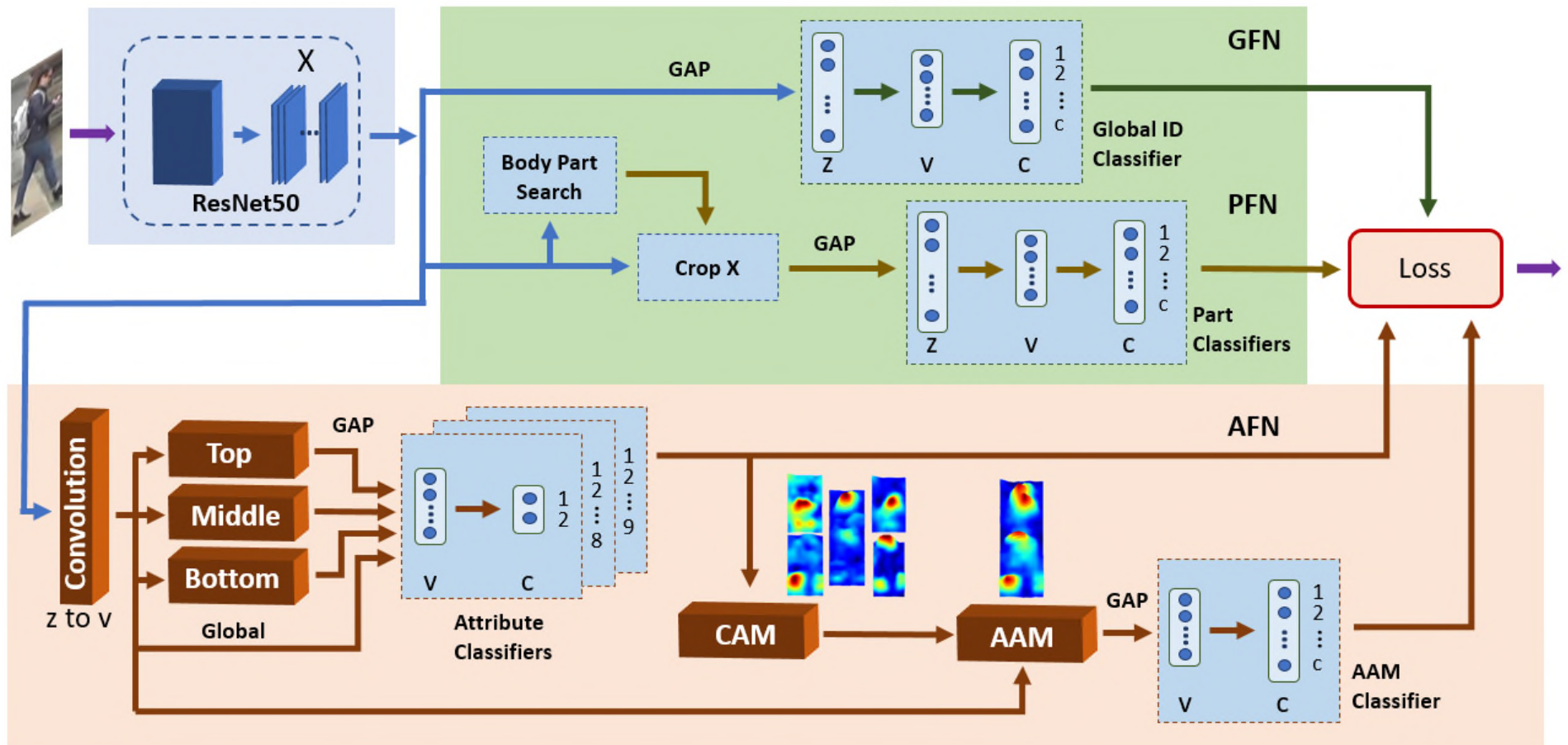
- Accuracy
- Memory footprint (parameters/weights + activation maps)
- Speed/computational complexity (FLOPS)
- Often, what is the important metric will depend on the applications/problems.

Applications

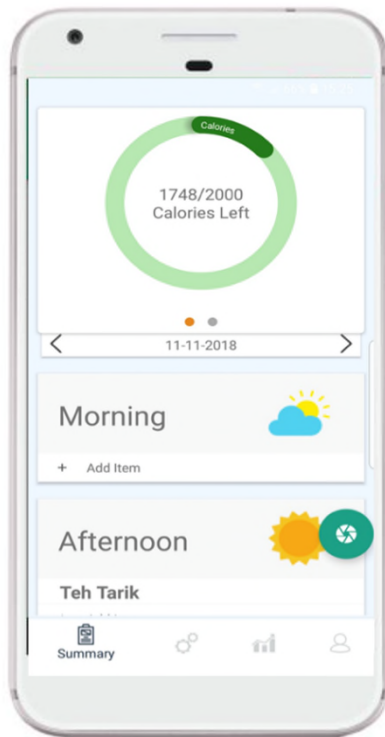
Person Re-Identification



Person Re-Identification



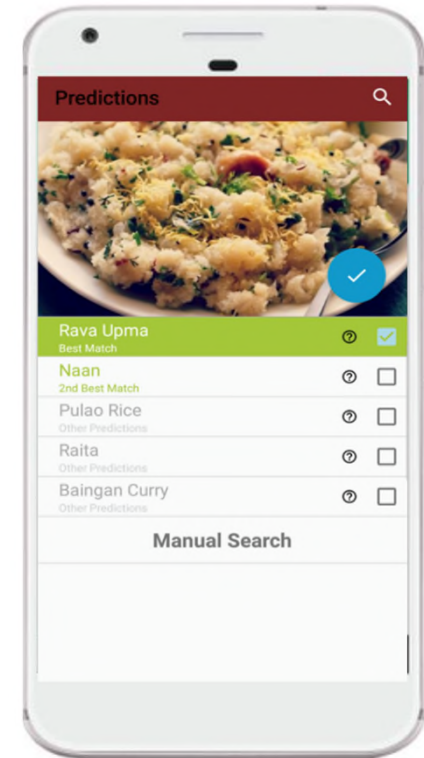
Visual Food Recognition



Home page



Taking picture



Food recognition

Visual Food Recognition

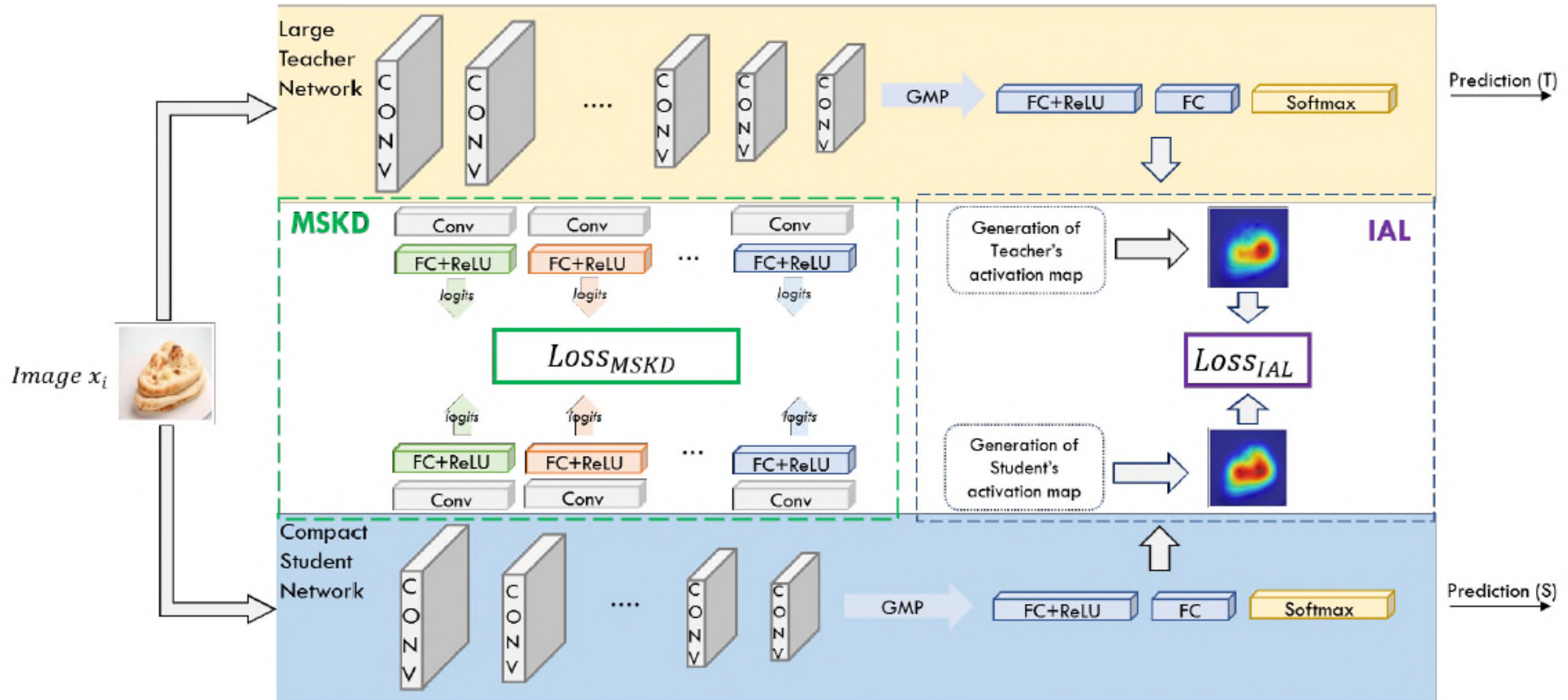


Figure 2: Overview of the proposed JLDNet. A full teacher network and a compact student network are jointly trained using the proposed Multi-Stage Knowledge Distillation (MSKD) and Instance Activation Learning (IAL) methods to achieve high recognition accuracy at low memory footprint.

Section I Summary

- The section covers the following topics:
 - Introduction
 - Linear Classifier
 - Convolutional Neural Networks (CNNs)
 - CNN Training & Optimization
 - Well-Known CNN Architectures
 - Applications

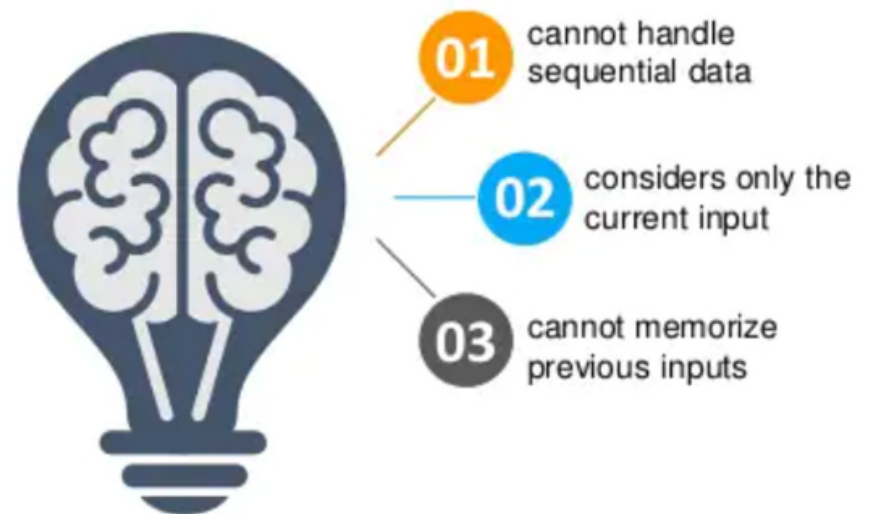
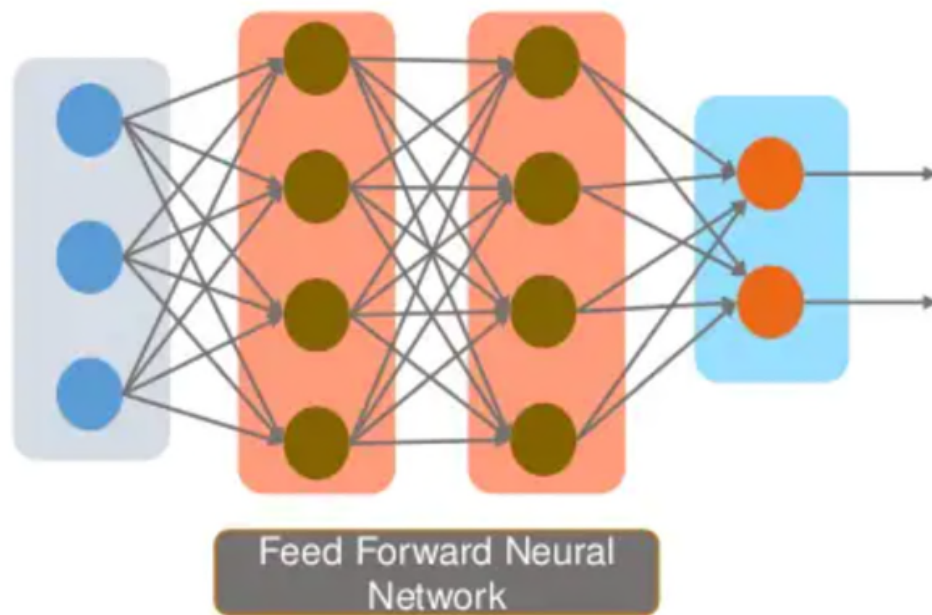
Section II

Recurrent Neural Network (RNN) & Long Short-Term Memory (LSTM)

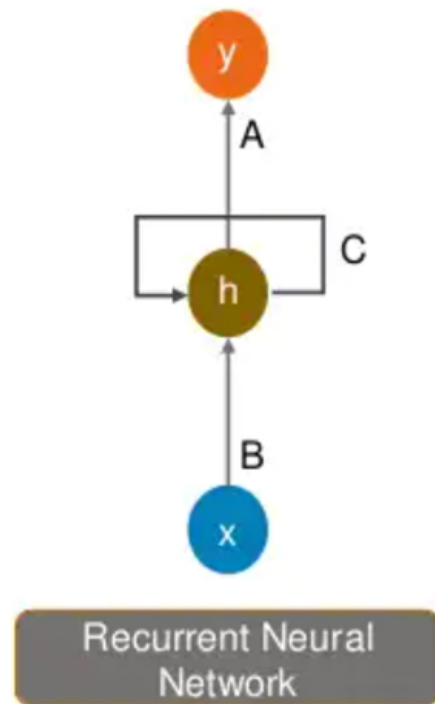
Section II Overview

- The section covers the following topics:
 - Introduction
 - Recurrent Neural Network (RNN)
 - RNN Training & Optimization
 - Long Short-Term Memory (LSTM)
 - Applications

Limitations of Feedforward Network



Why Recurrent Neural Network?



- 01 can handle sequential data
- 02 considers the current input and also the previously received inputs
- 03 can memorize previous inputs due to its internal memory

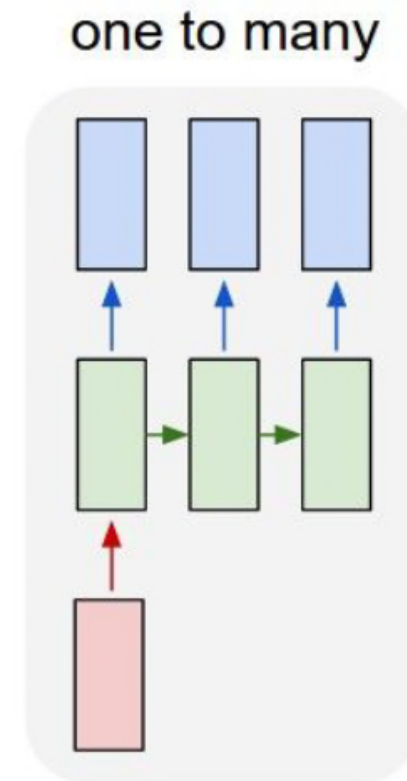
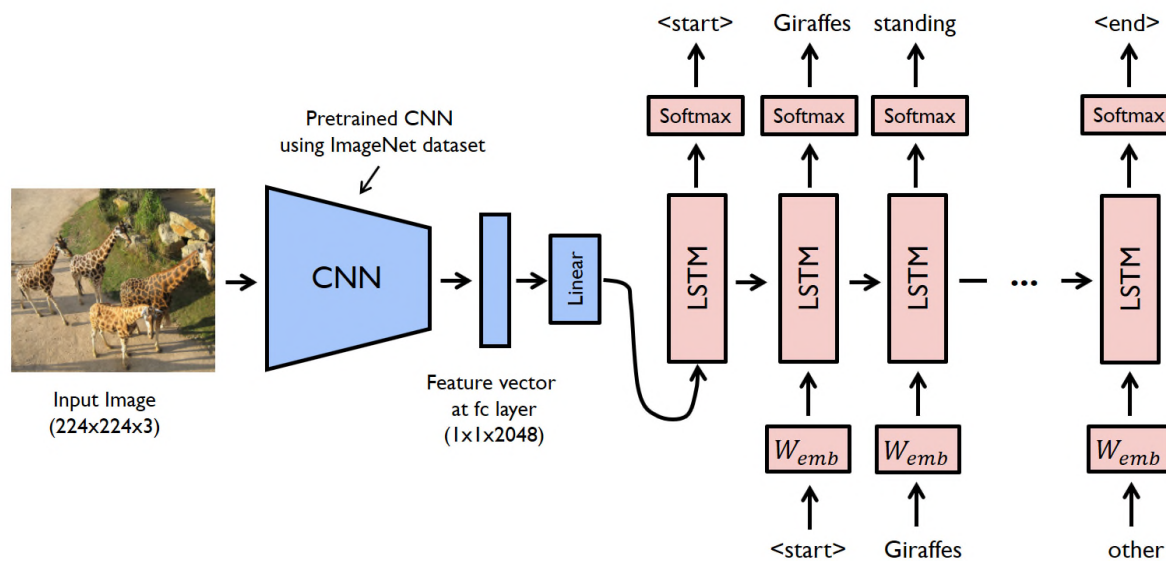
Sample Applications

- Time series prediction / forecasting
 - Prediction of stock prices, product sales, etc.
- Speech recognition
- Natural Language Processing (NLP)
 - Language translation
 - Text sentiment classification
- Image captioning

Sequence Modelling

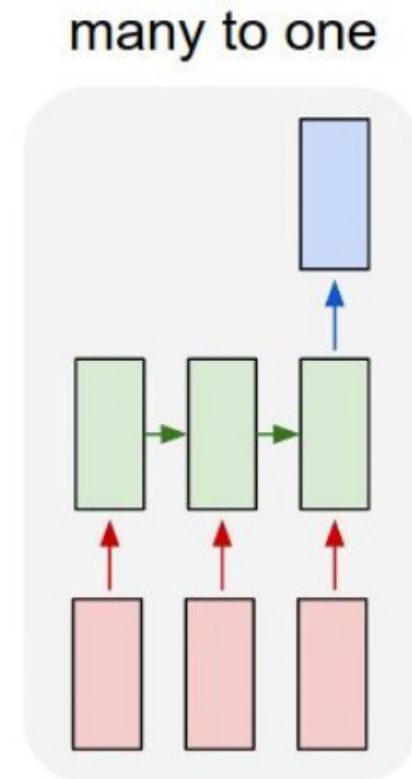
Sequence Modelling (One-to-Many Mapping)

- One input, many outputs (sequence of outputs)
- E.g., image captioning
 - Image $\rightarrow$ sequence of words



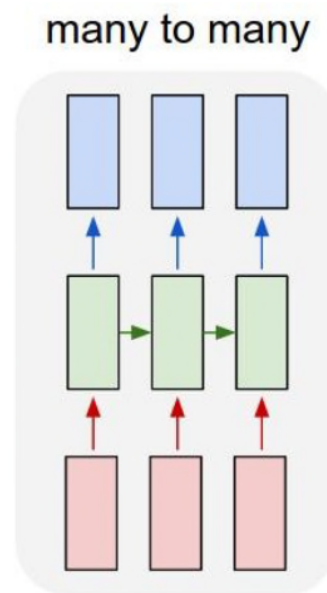
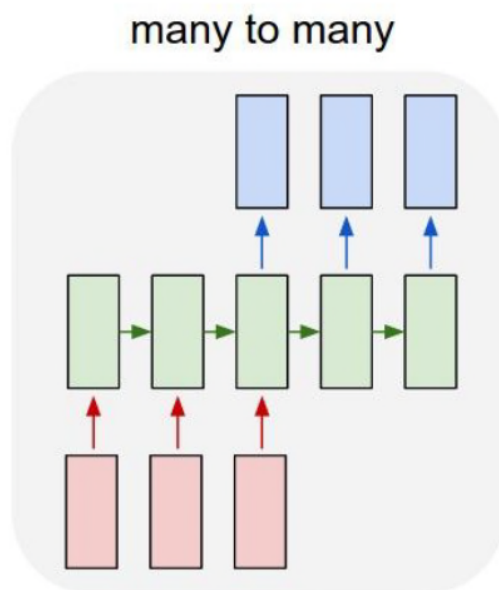
Sequence Modelling (Many-to-One Mapping)

- Many inputs (sequence of inputs), one output
- E.g., Video classification
 - Sequence of images → Label
- Sentiment classification
 - Sequence of words → Label

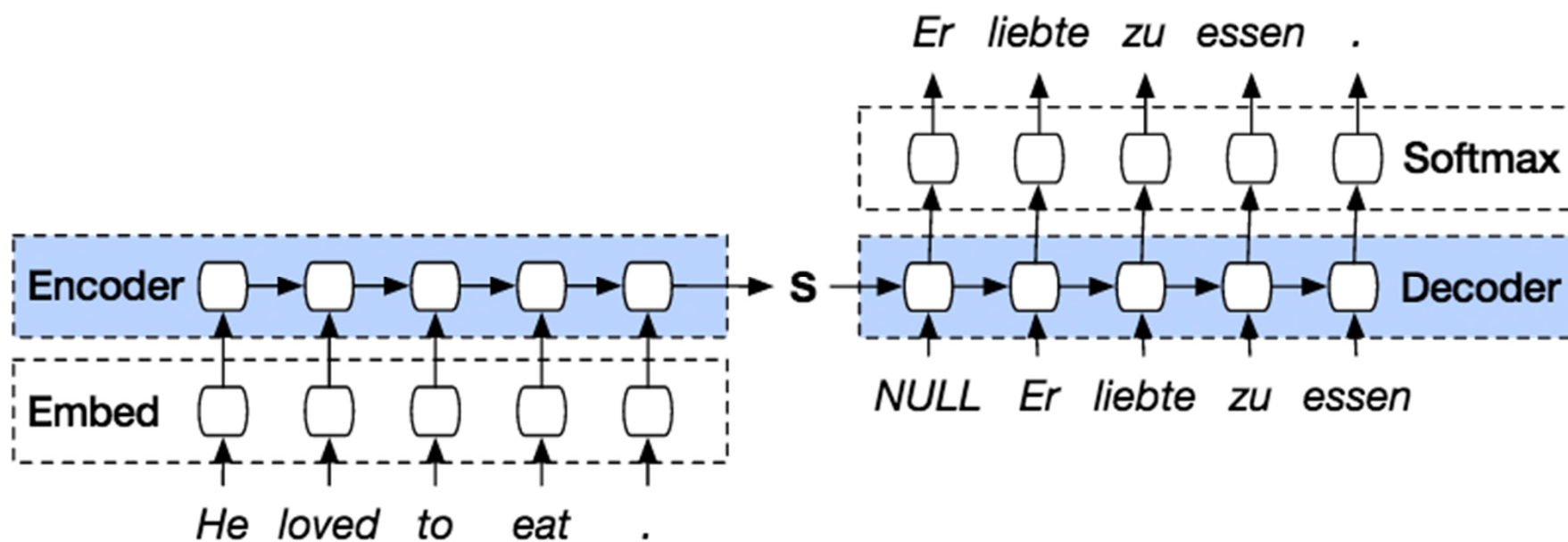


Sequence Modelling (Many-to-Many Mapping)

- Many inputs (sequence of inputs), many outputs (sequence of outputs)
- E.g., machine translation
 - Sequence of words → sequence of words
- Per-frame video classification
 - Sequence of images → sequence of labels



Sequence to Sequence: Encoder + Decoder Modelling

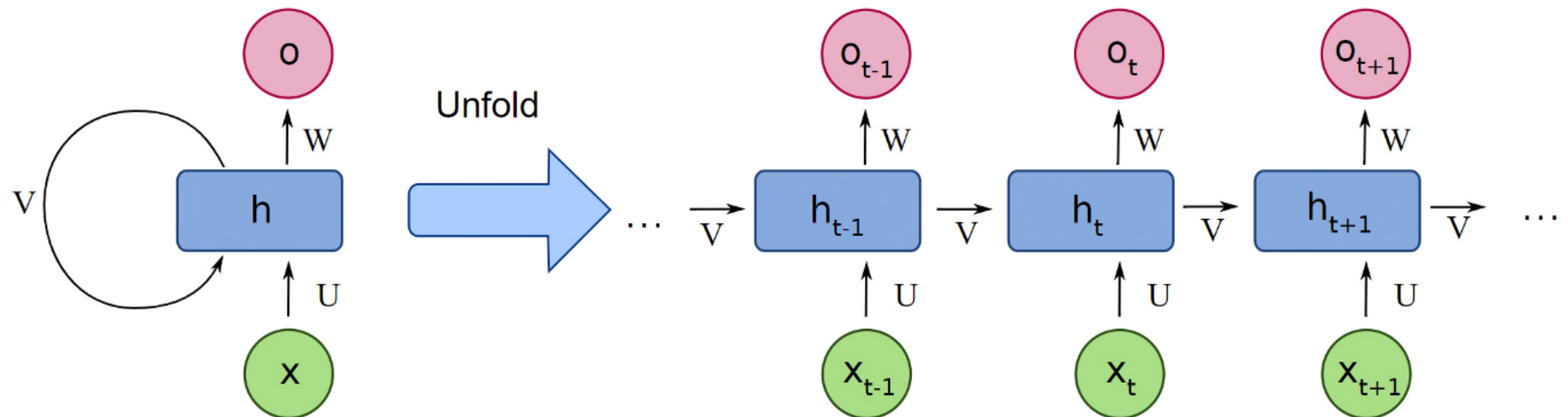


Recurrent Neural Networks (RNNs)

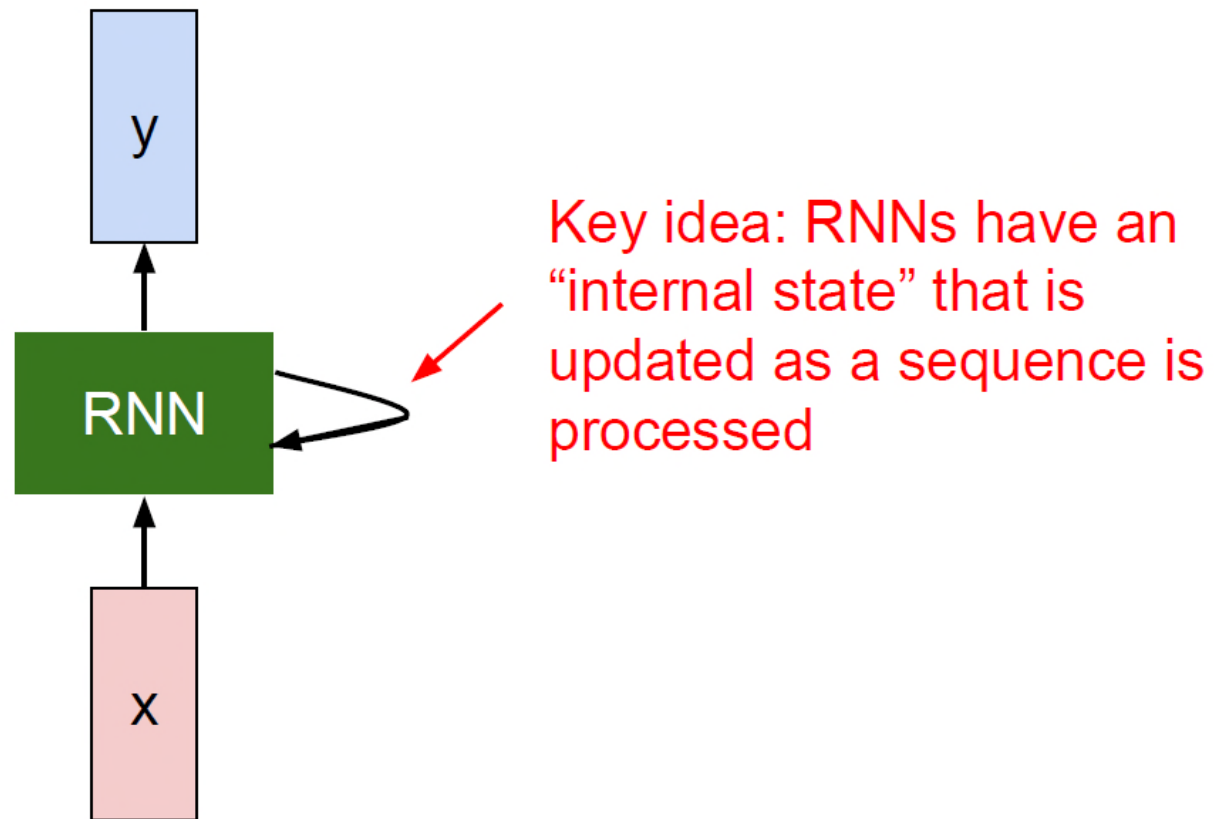
What are RNNs?

- RNNs are a class of neural networks which have inputs in the forms of sequential data, e.g., time series data.
- Commonly used in analysis of temporal / sequential data.
- Widely deployed in applications including Natural Language Processing (NLP), machine translation, image captioning, etc.

RNN Architecture



RNN Model



Hidden State Update

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

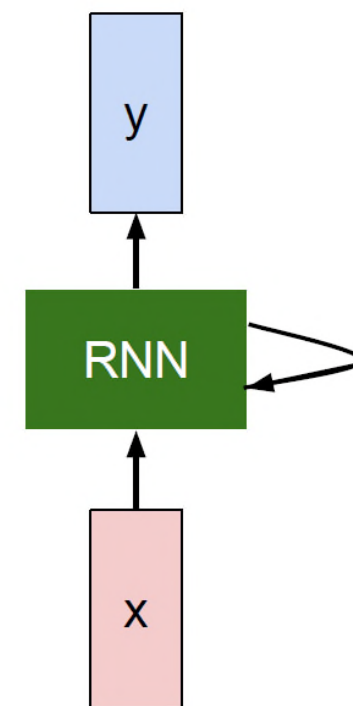
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



Output Generation

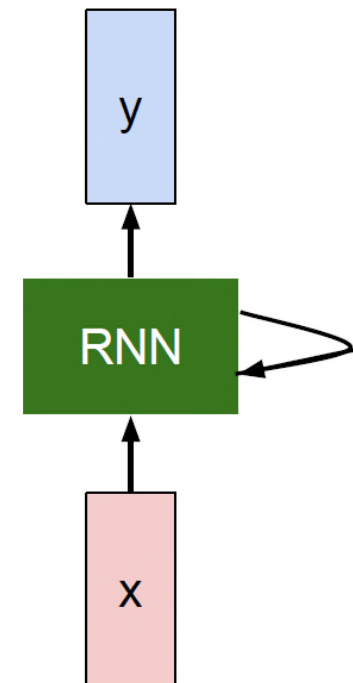
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

output

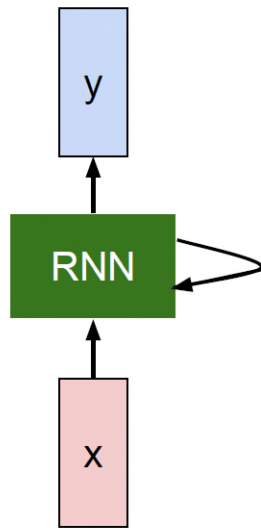
another function with parameters W_o

new state



Vanilla RNN (Elman RNN)

The state consists of a single “hidden” vector $\mathbf{h}$:



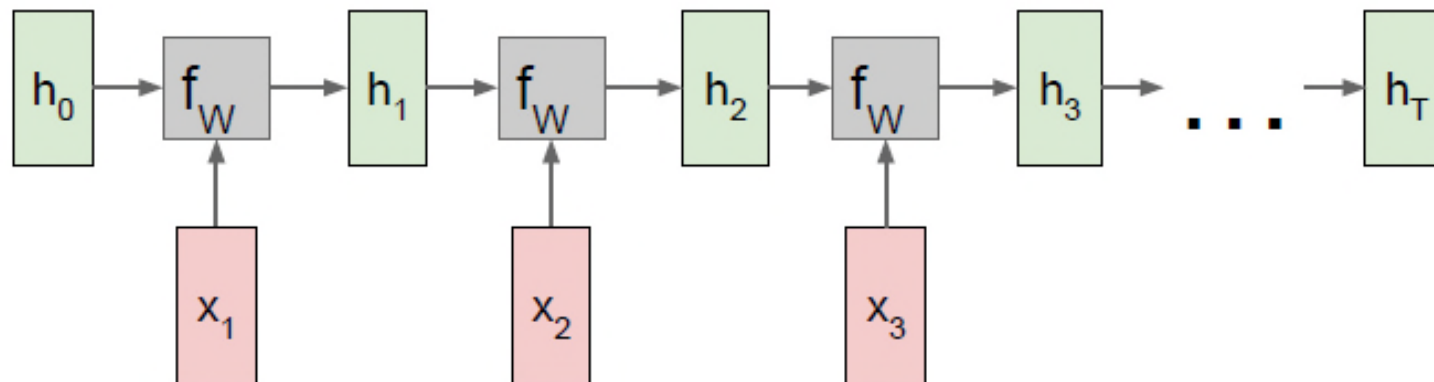
$$h_t = f_W(h_{t-1}, x_t)$$



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman



Exercise: RNN

- (a) A Vanilla Recurrent Neural Network (RNN) has the following settings.

Initial hidden state, $\mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$,

Hidden state weight matrix, $\mathbf{W}_{hh} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$,

Input weight matrix, $\mathbf{W}_{xh} = \begin{bmatrix} 0.5 & 0.2 \\ 0.2 & 0.1 \end{bmatrix}$,

Output weight matrix, $\mathbf{W}_{hy} = [0.1 \quad 0.4]$.

Assume no bias is used in the computation of the RNN.

A 2-timestep input is given by $\mathbf{x} = [\mathbf{x}_1 \quad \mathbf{x}_2]$ where $\mathbf{x}_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $\mathbf{x}_2 = \begin{bmatrix} 1 \\ 6 \end{bmatrix}$.

- (i) Find the hidden state $\mathbf{h}_1$ at timestep $t = 1$.
- (ii) Find the output y_1 at timestep $t = 1$.
- (iii) Find the output y_2 at timestep $t = 2$.

Solution

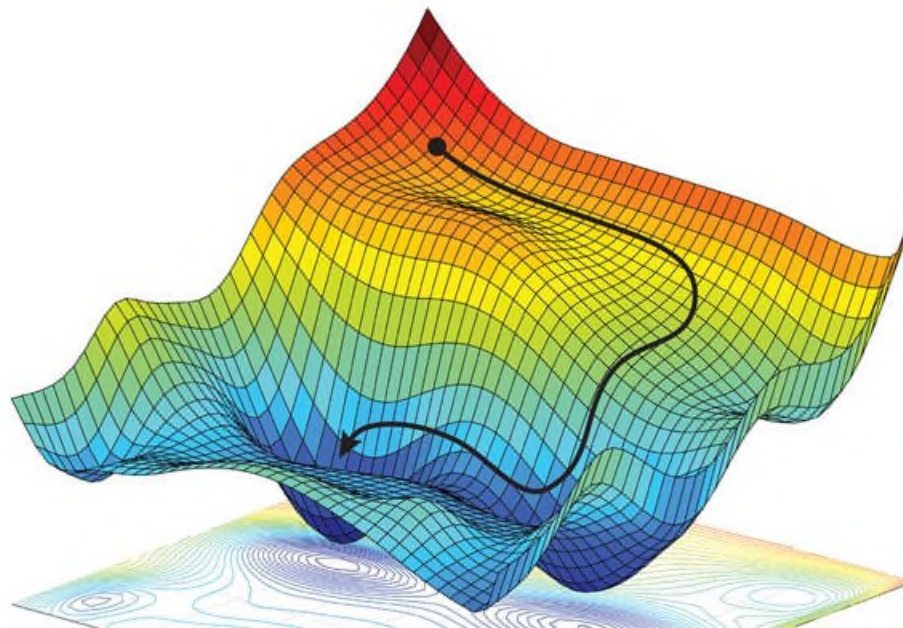
RNN Pros and Cons

- Pros:
 - Can process any length of input
 - Computation for step t can in theory use information from many steps back
 - Model size does not increase for longer input
 - Same weights applied for every timestep, so there is consistency in how inputs are processed.
- Cons:
 - Recurrent computation is slow
 - In practice, difficult to leverage information from many steps back

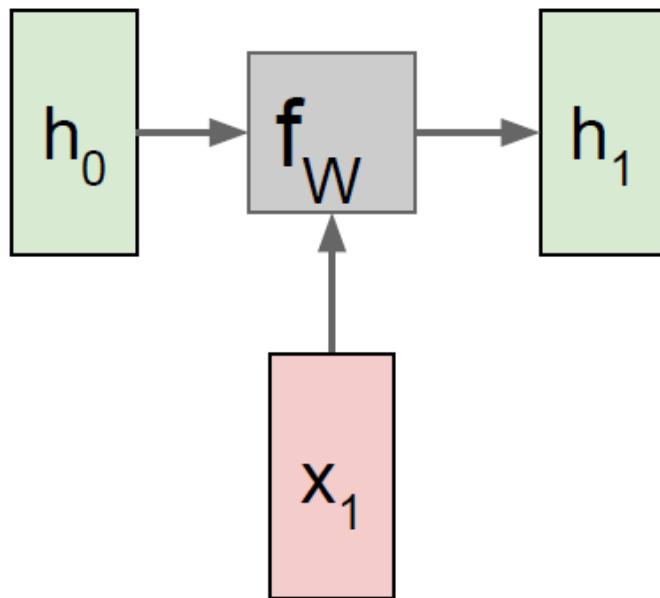
RNN Training & Optimization

Recall: CNN Training

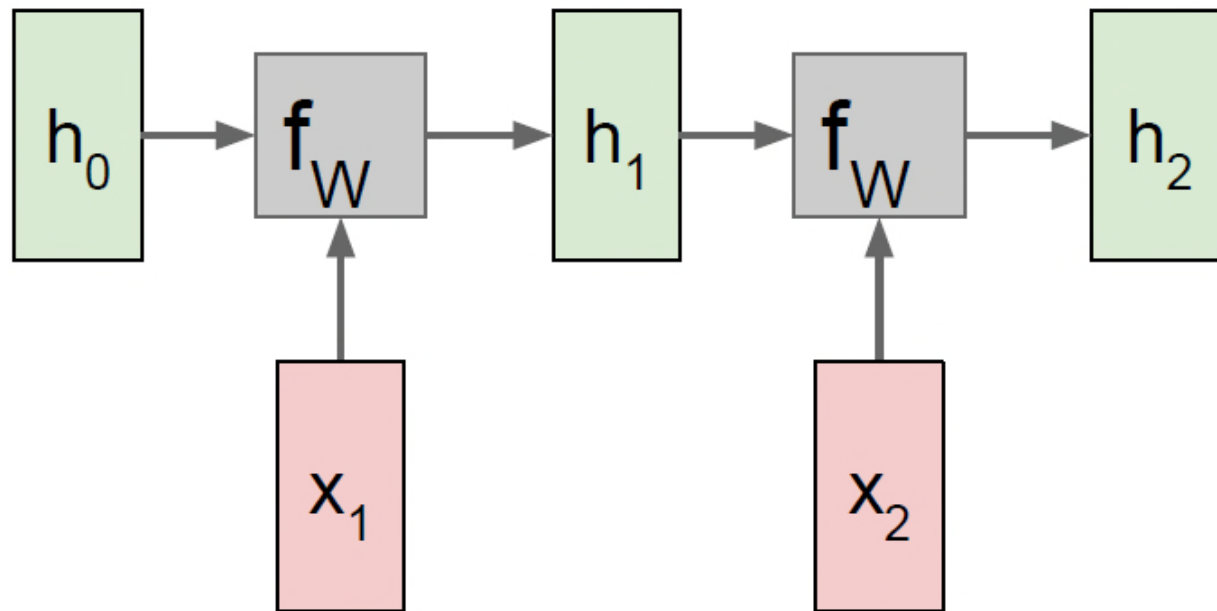
- The objective is to minimize/optimize the loss function.
- Common strategy is centered on stochastic gradient descent (SGD) and its extension.
- Use computational graph to compute the gradient descent and parameter update.



Computational Graph (1)

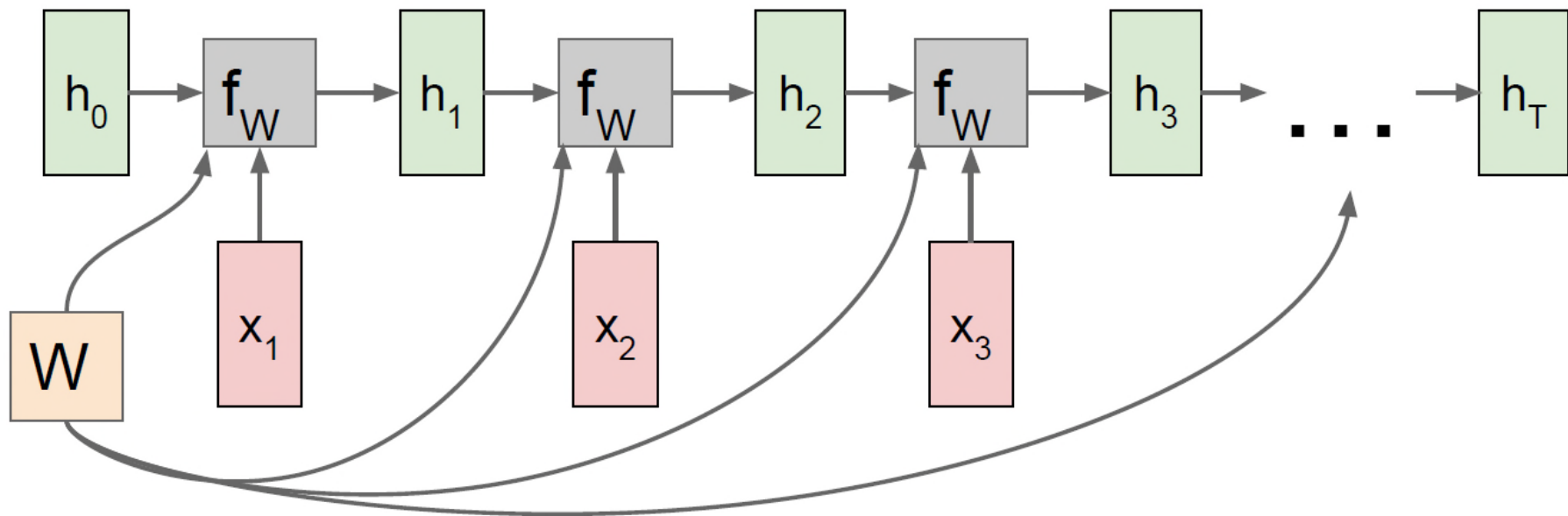


Computational Graph (2)

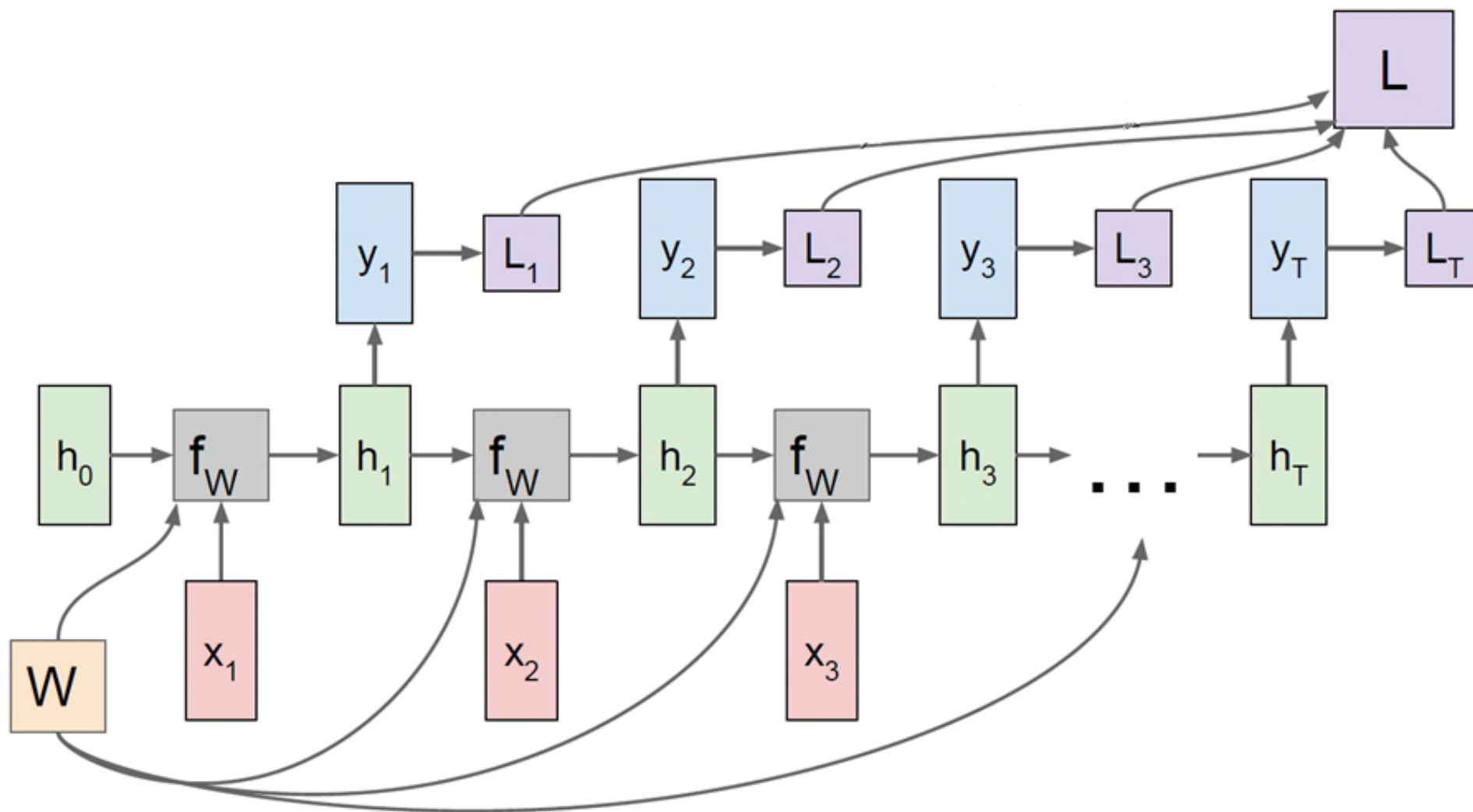


Computational Graph (3)

Re-use the same weight matrix at every time-step

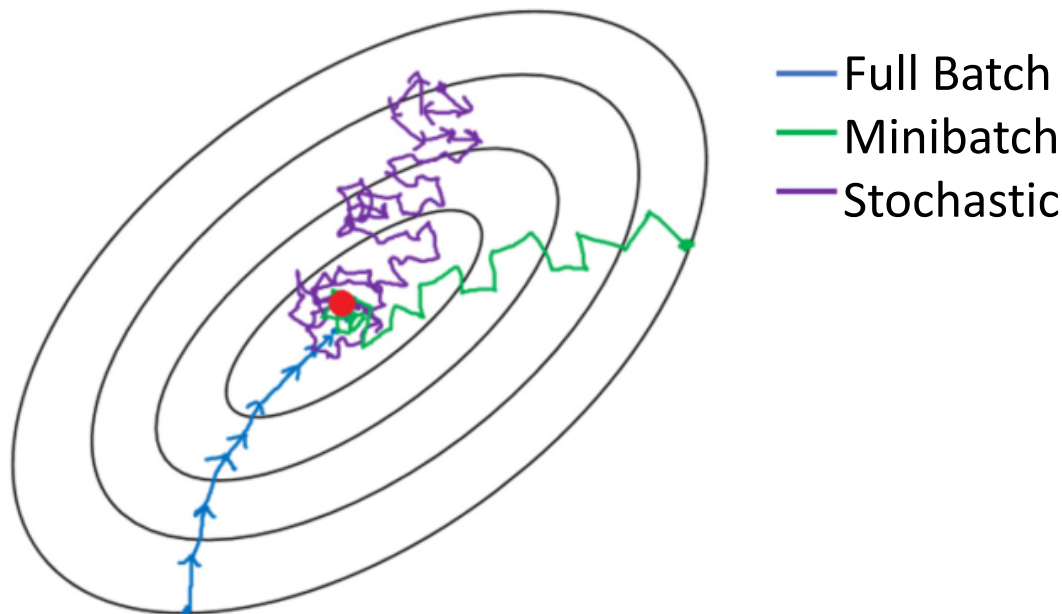


Computational Graph (4)



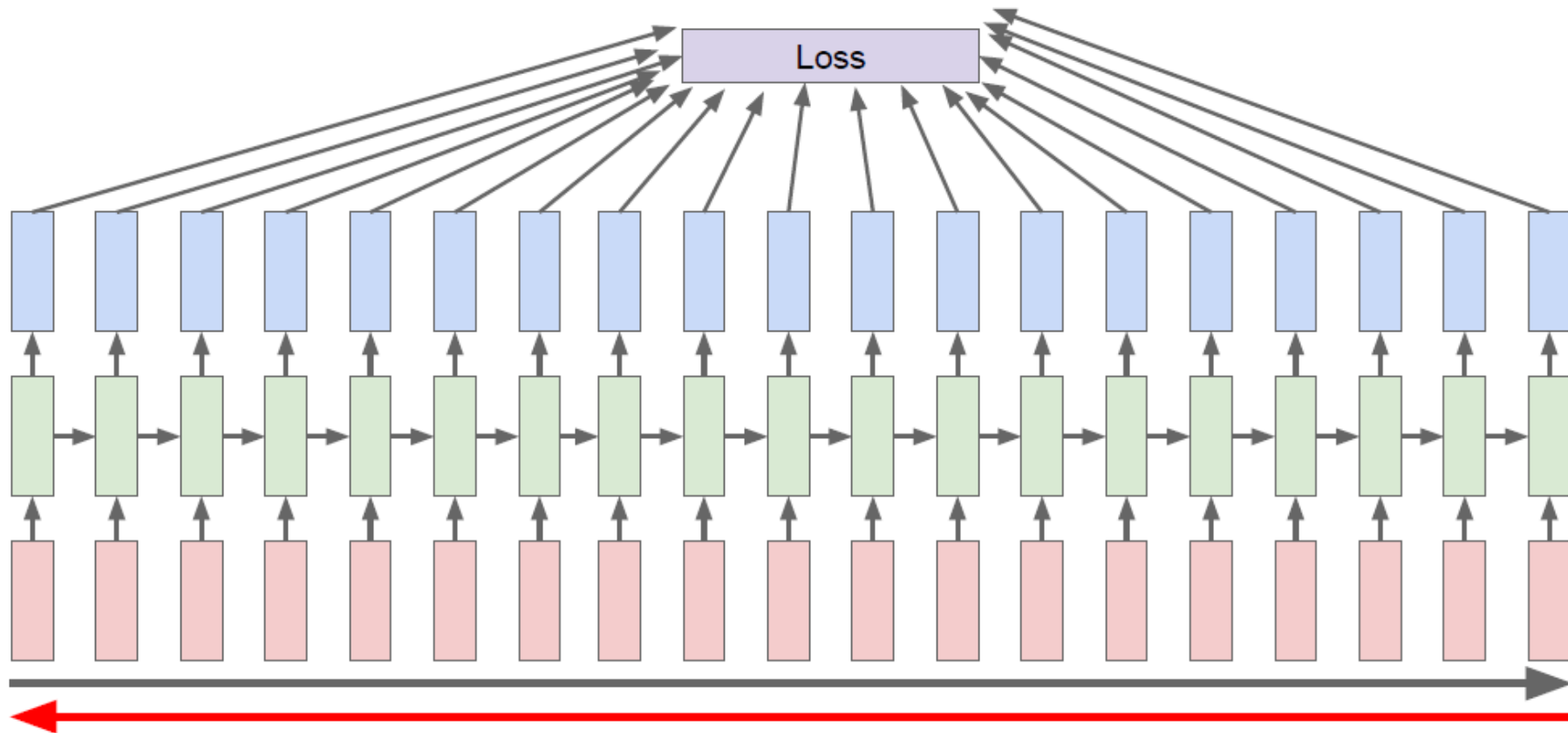
Batch Training

- Full Batch: use entire set of training sequences at each iteration
- Stochastic: use a single sequence at each iteration
- Minibatch (Stochastic): use a few training sequences at each iteration

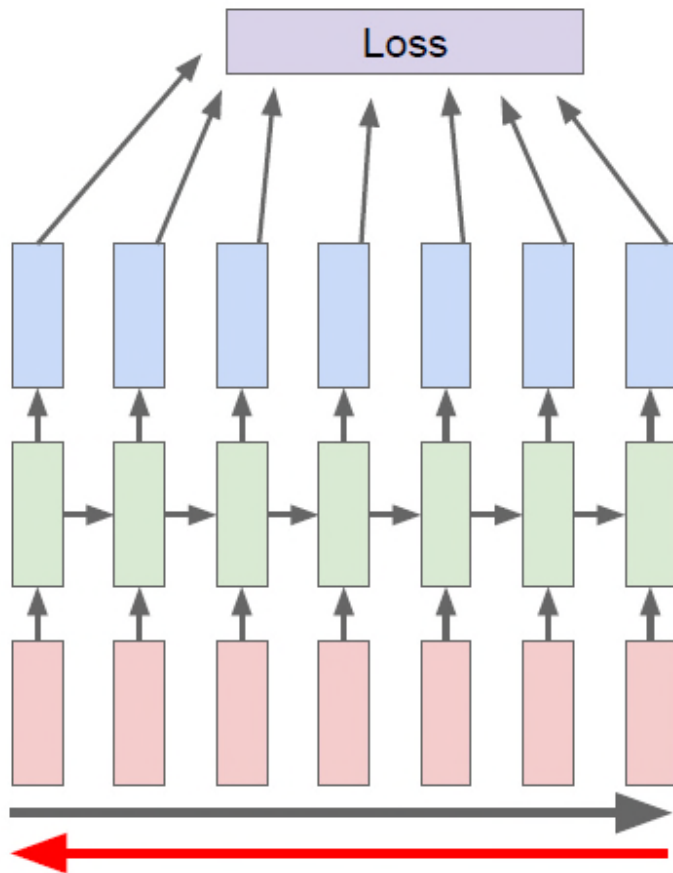


Backpropagation Through Time (BPTT)

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

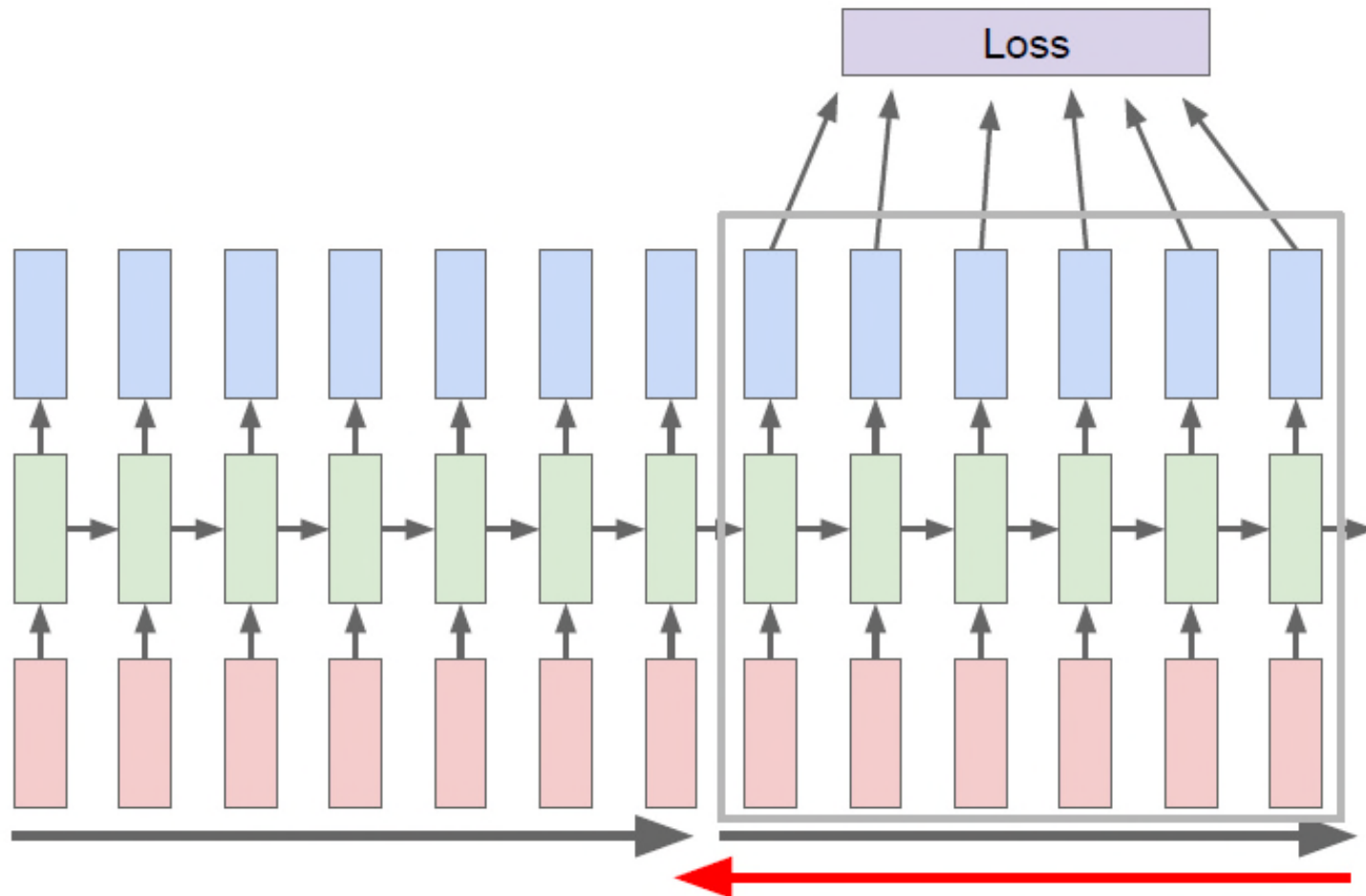


Truncated Backpropagation Through Time (1)



Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation Through Time (2)

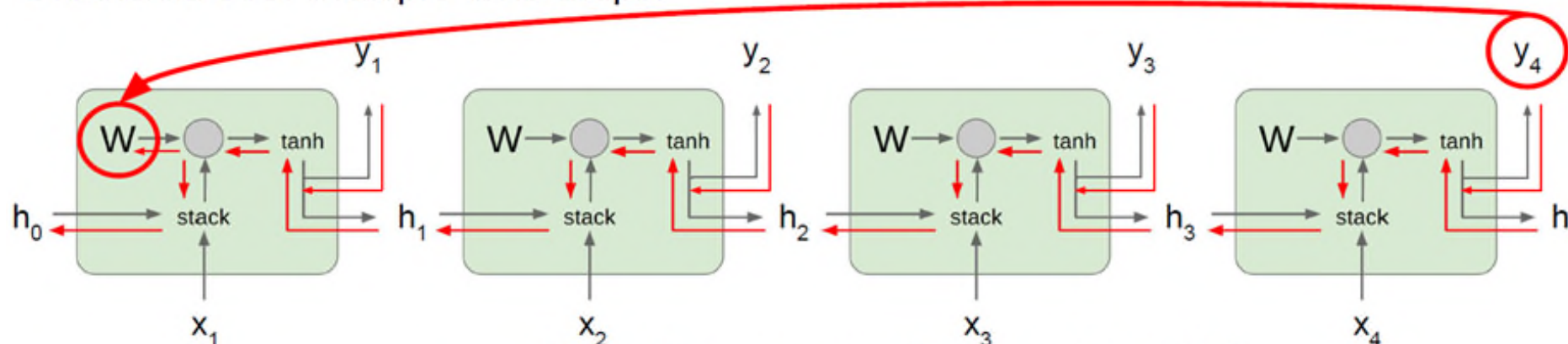


Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Exploding / Vanishing Gradients (1)

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:
Exploding gradients

Largest singular value < 1:
Vanishing gradients

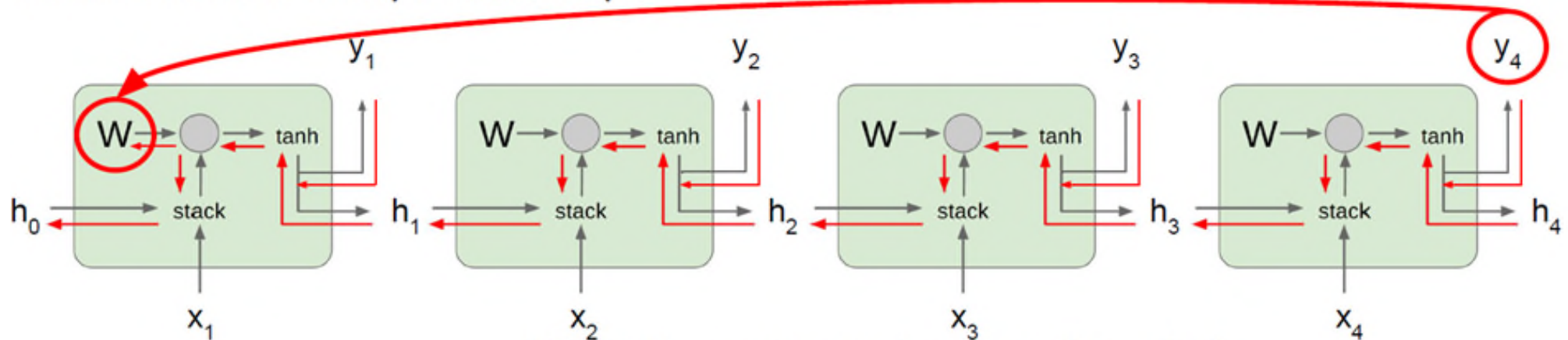
Gradient clipping:
Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

Exploding / Vanishing Gradients (2)

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1:
Exploding gradients

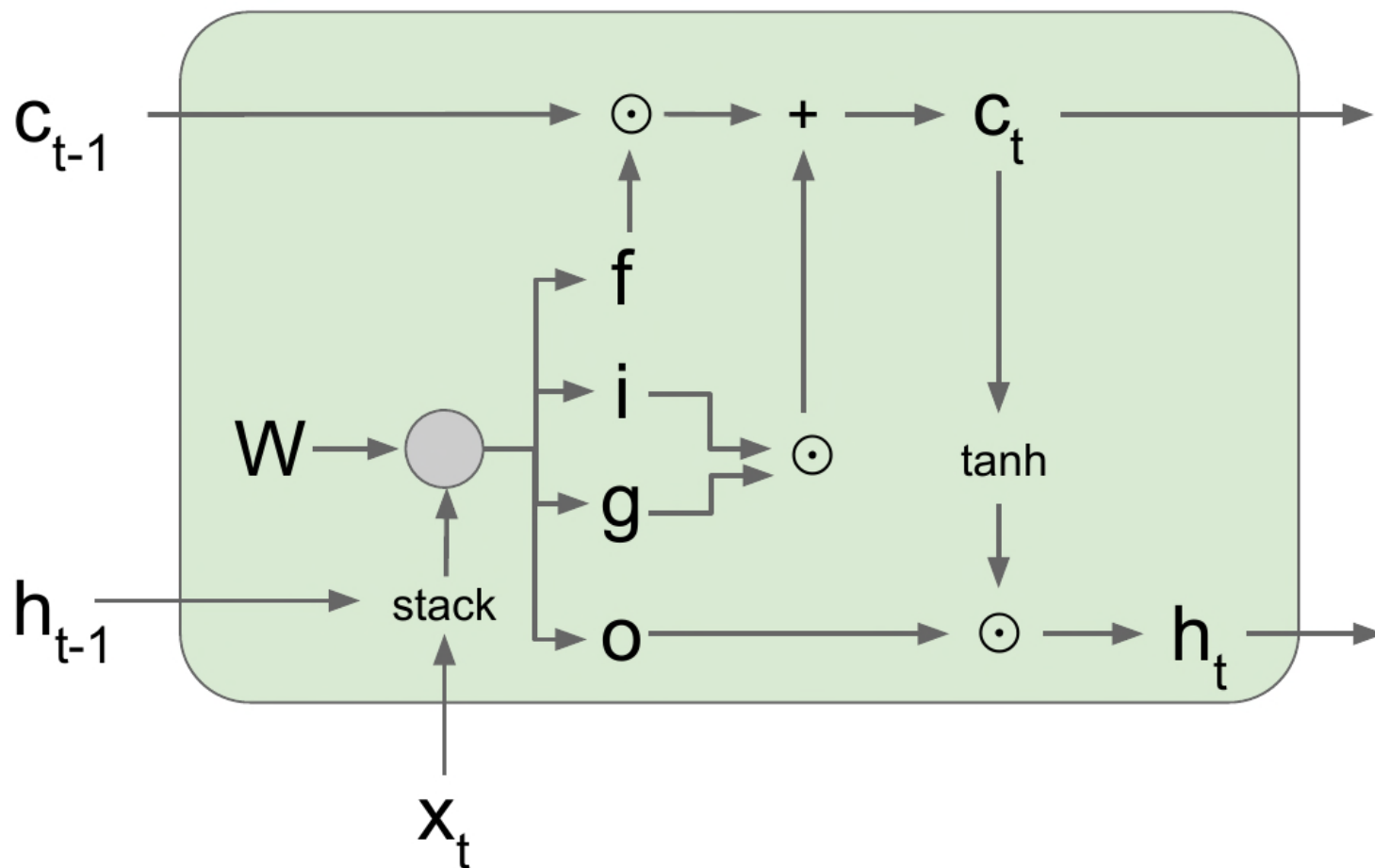
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:
Vanishing gradients

→ Change RNN architecture

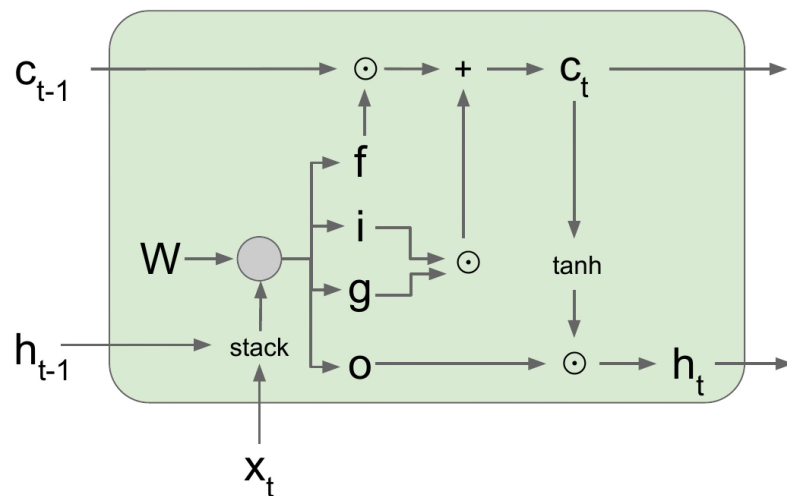
Long Short-Term Memory (LSTM)

LSTM Architecture



Key Ideas

- A memory cell which can maintain its state over time.
- Consist of cell state (c_t), hidden state (h_t) and 4 gates (i, f, o, g).
- c_t : long-term memory, h_t : short-term memory.
- Cell state (c_t) undergoes changes via forgetting old memory (through forget (f) gate) and adding new memory (through input (i) gate & gate (g) gate).
- Hidden state (h_t) is updated by passing cell state (c_t) through output gate.
- Gates control the flow of information to the memory.
- Gate are obtained through a sigmoid/tanh layer, and they update c_t and h_t cell states using pointwise multiplication operator.



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

RNNs vs LSTMs

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

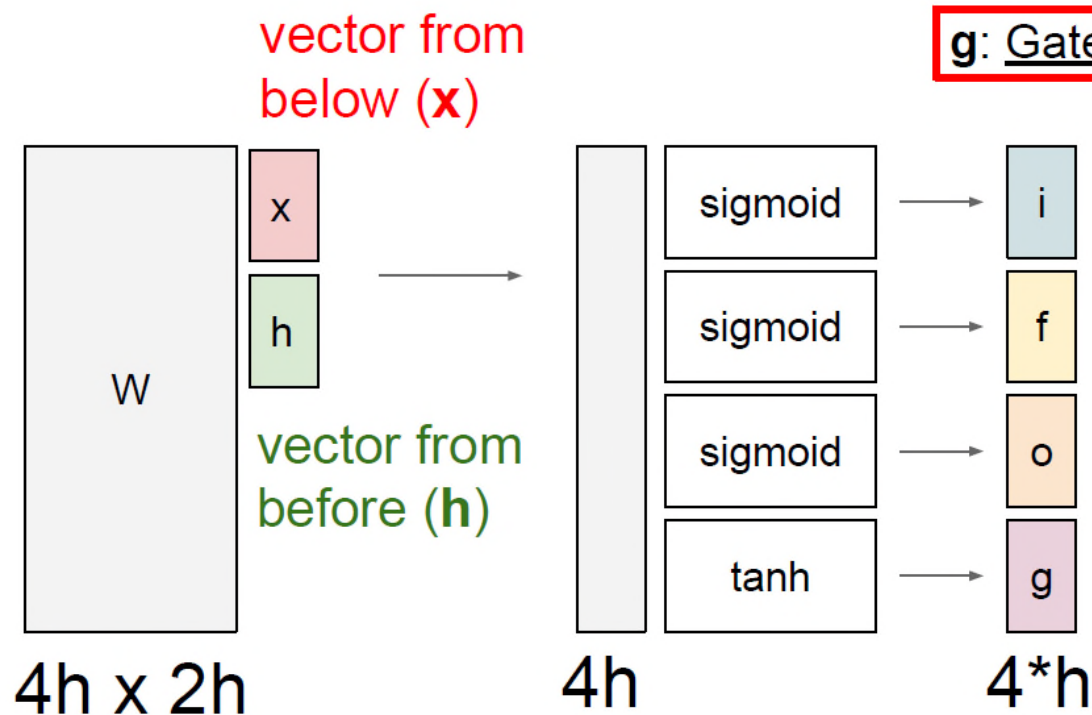
LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM Computation (1)



g: Gate gate (?), How much to write to cell

g: also known as candidate gate

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

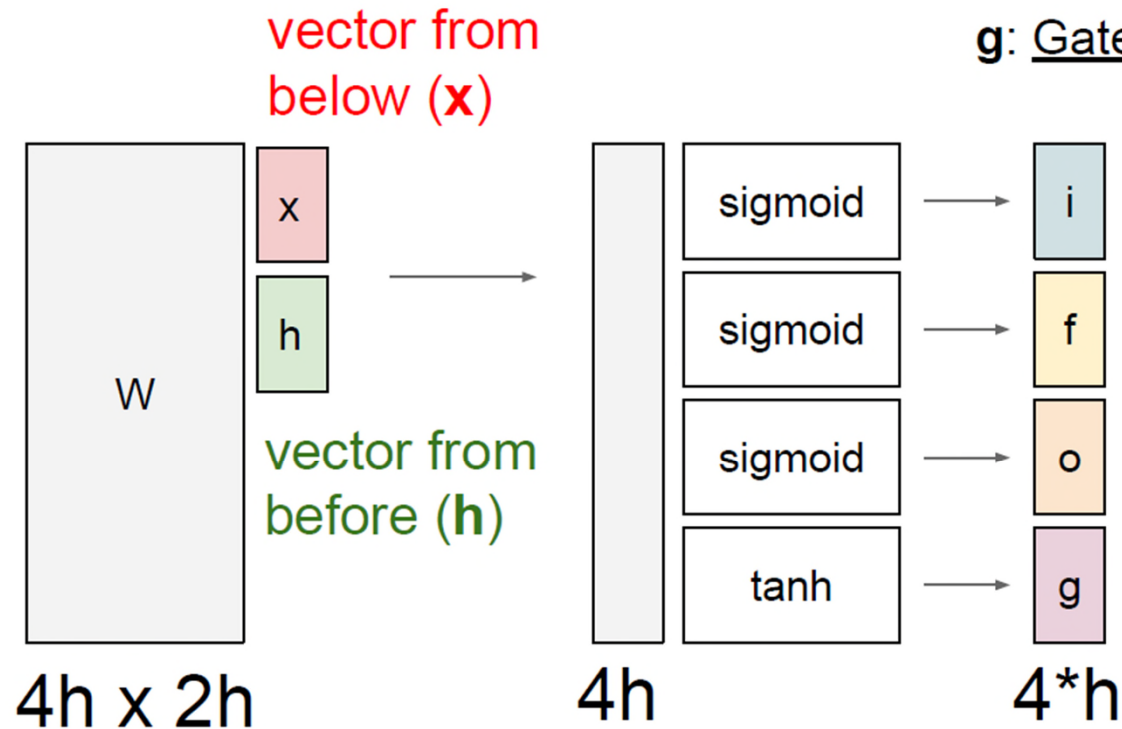
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM Computation (2)

i: Input gate, whether to write to cell

g: Gate gate (?), How much to write to cell

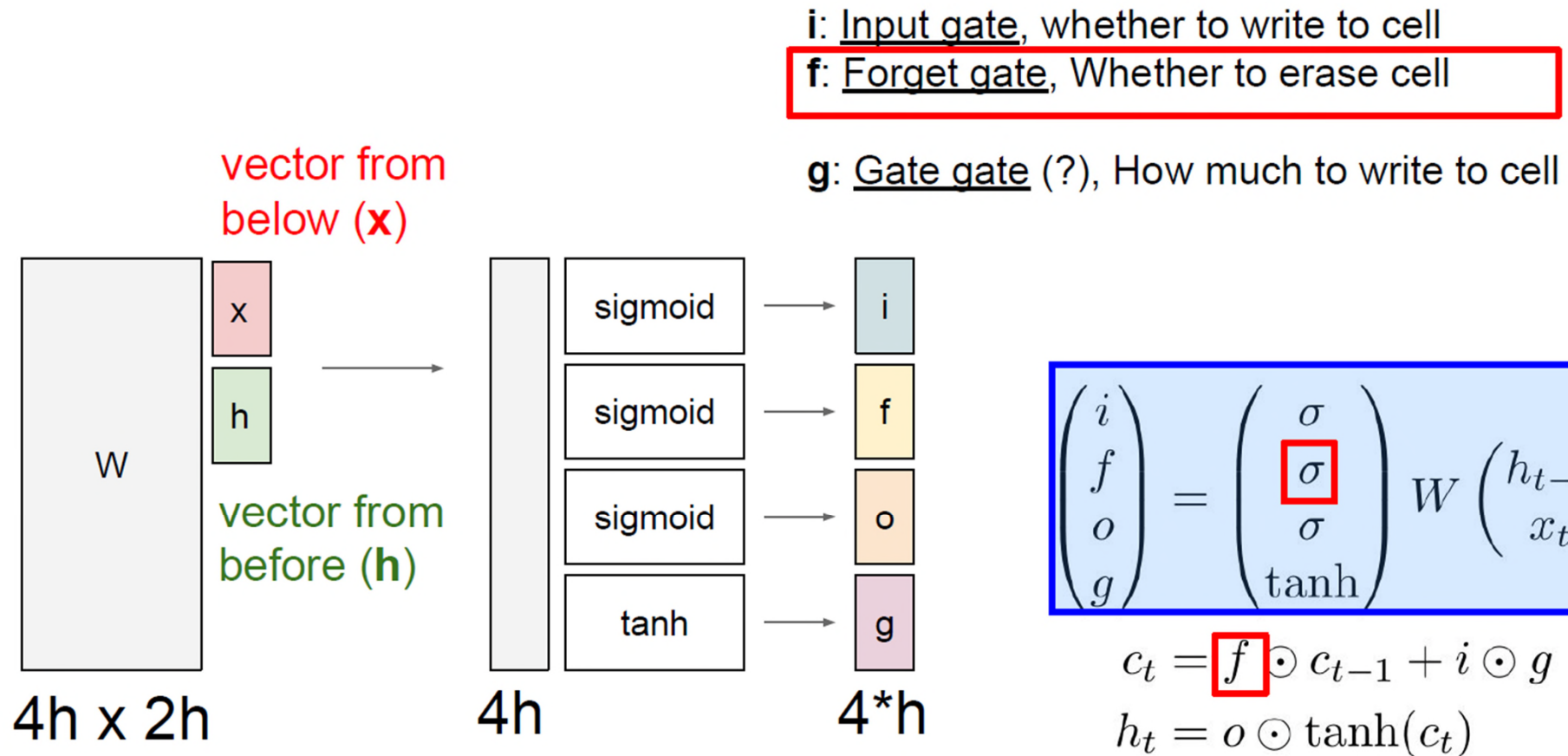


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

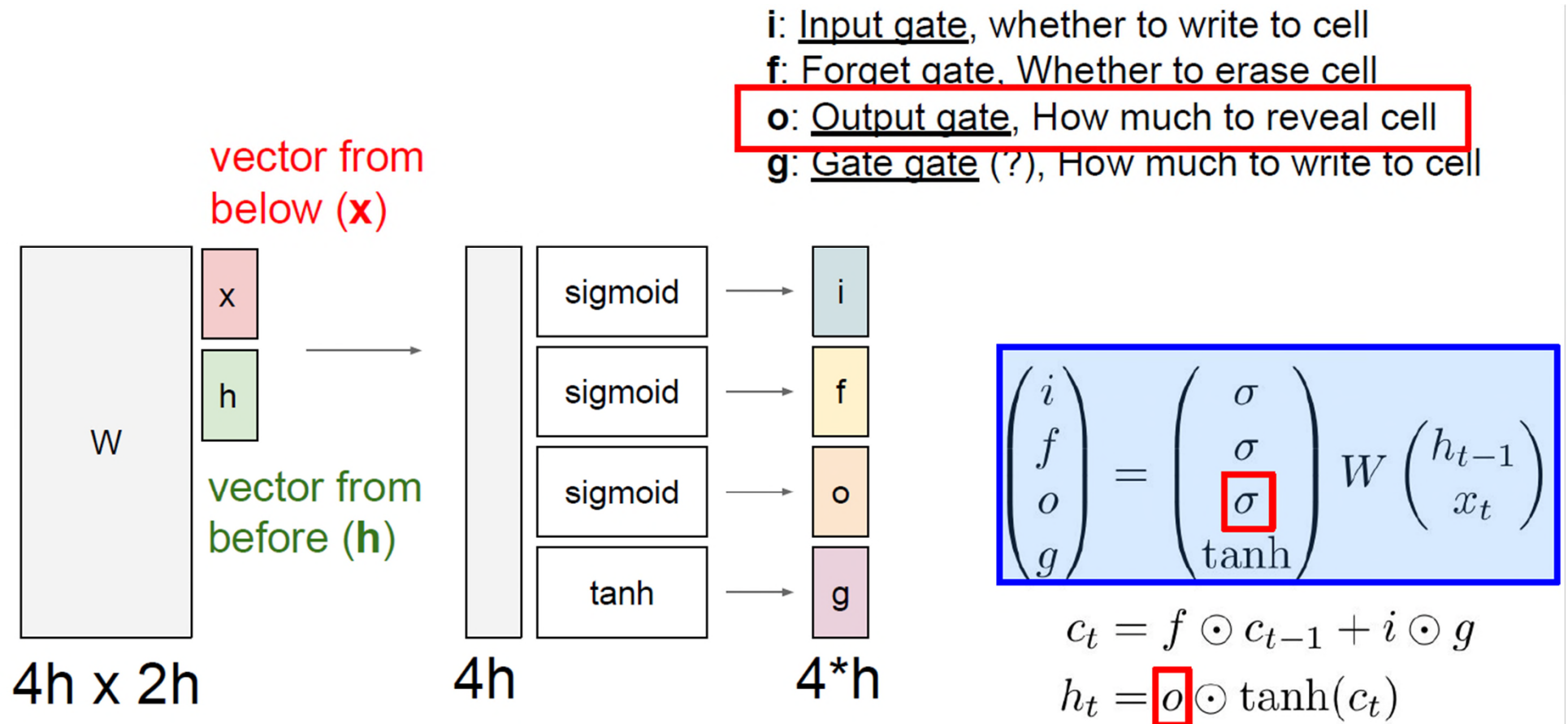
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM Computation (3)



LSTM Computation (4)



Exercise: LSTM

- (a) A Long Short-Term Memory (LSTM) network has the following settings.

Initial hidden state, $\mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, Initial cell state, $\mathbf{c}_0 = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$,

Forget gate weight matrix, $\mathbf{W}_f = \begin{bmatrix} \mathbf{W}_{hf} & \mathbf{W}_{xf} \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 & 0.5 & 0.6 \\ 0.3 & 0.4 & 0.7 & 0.8 \end{bmatrix}$,

Input gate at timestep $t=1$, $\mathbf{i}_1 = \begin{bmatrix} 0.3 \\ 0.4 \end{bmatrix}$,

Gate gate at timestep $t=1$, $\mathbf{g}_1 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$,

Output gate at timestep $t=1$, $\mathbf{o}_1 = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$,

Input at timestep $t=1$, $\mathbf{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

Assume no bias is used in the computation of the LSTM. The sigmoid and tanh functions are given as follows.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- (i) Find the forget gate $\mathbf{f}_1$ at timestep $t=1$. Comment on your obtained result.
- (ii) Find the cell state $\mathbf{c}_1$ at timestep $t=1$.

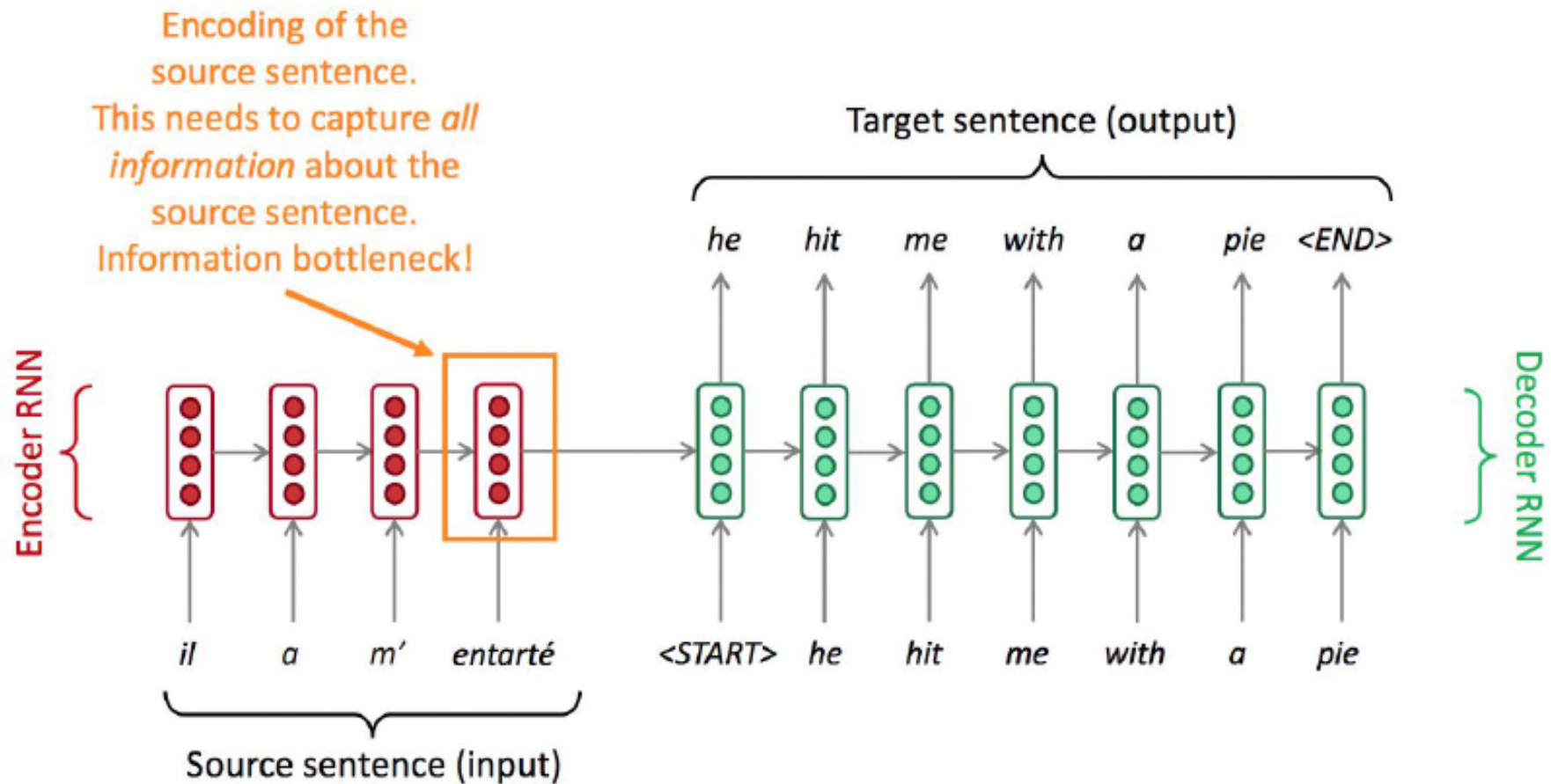
Solution

Applications

Time Series Prediction



Machine Translation



Section II Summary

- The section covers the following topics:
 - Introduction
 - Recurrent Neural Network (RNN)
 - RNN Training & Optimization
 - Long Short-Term Memory (LSTM)
 - Applications

Section III

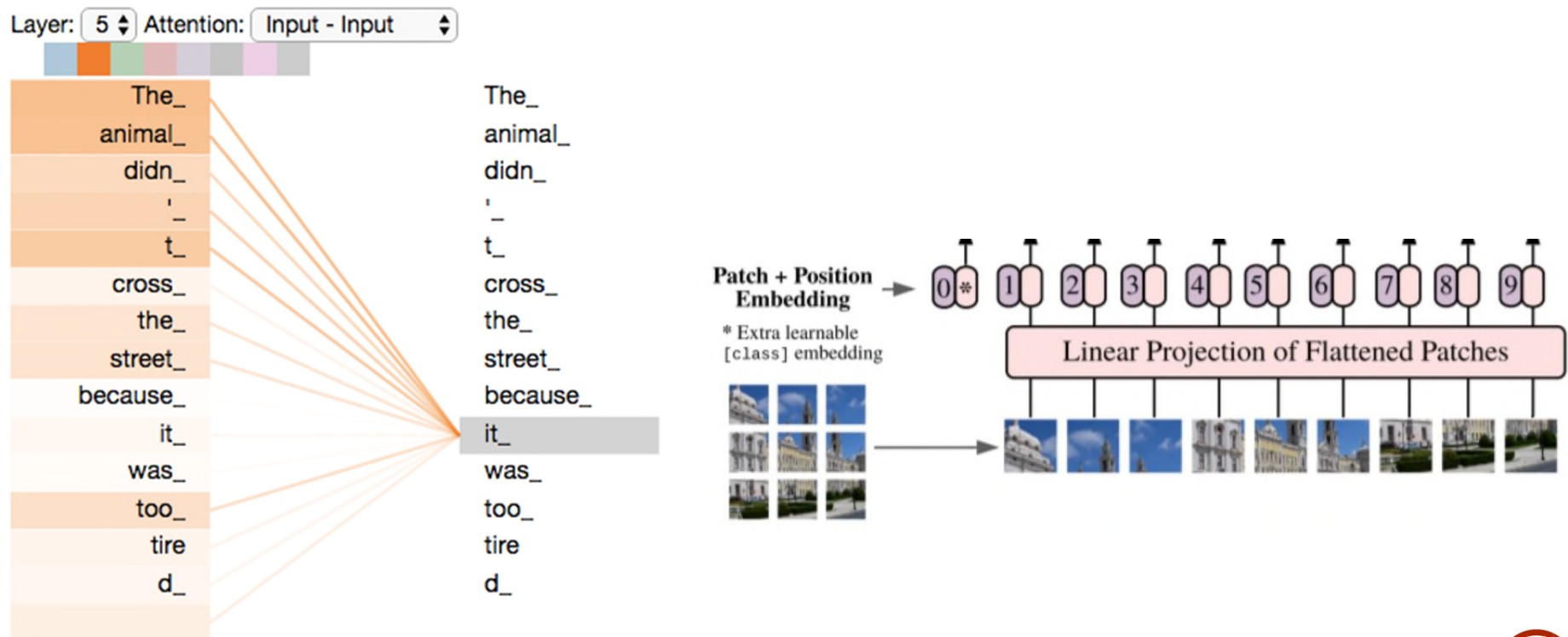
Transformer

Section III Overview

- The section covers the following topics:
 - Attention Concept
 - Transformer Architecture
 - Vision Transformer

What is Attention?

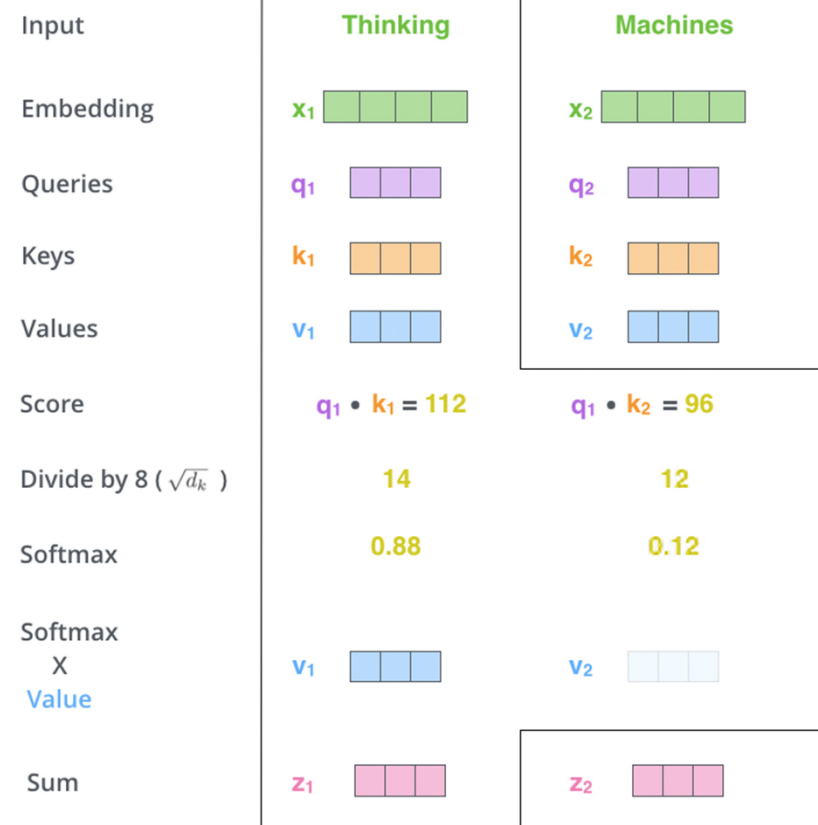
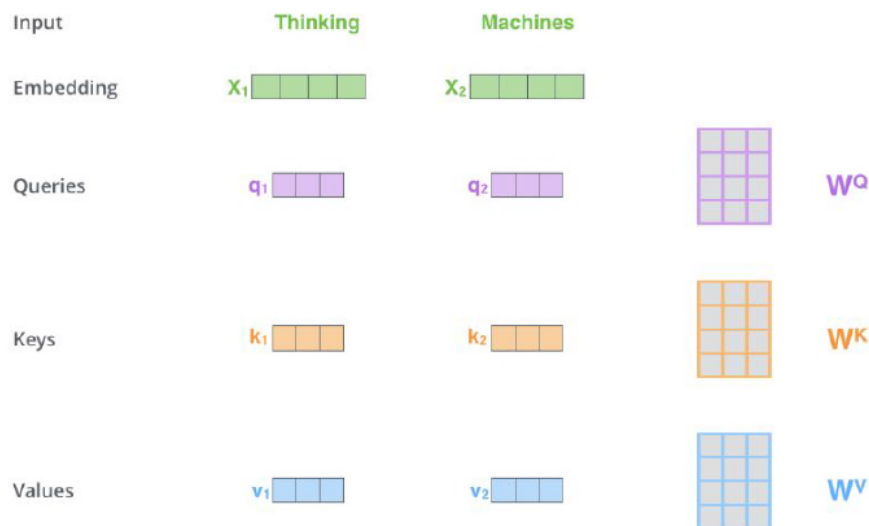
- Attention is used to determine which input tokens (e.g., words in NLP, image patches in CV) are relevant to the current input / token.
- Attention is computed through correlation (dot product) between 2 vectors.
- Correlation -> similarity / relatedness / importance -> attention



How is Attention Computed?

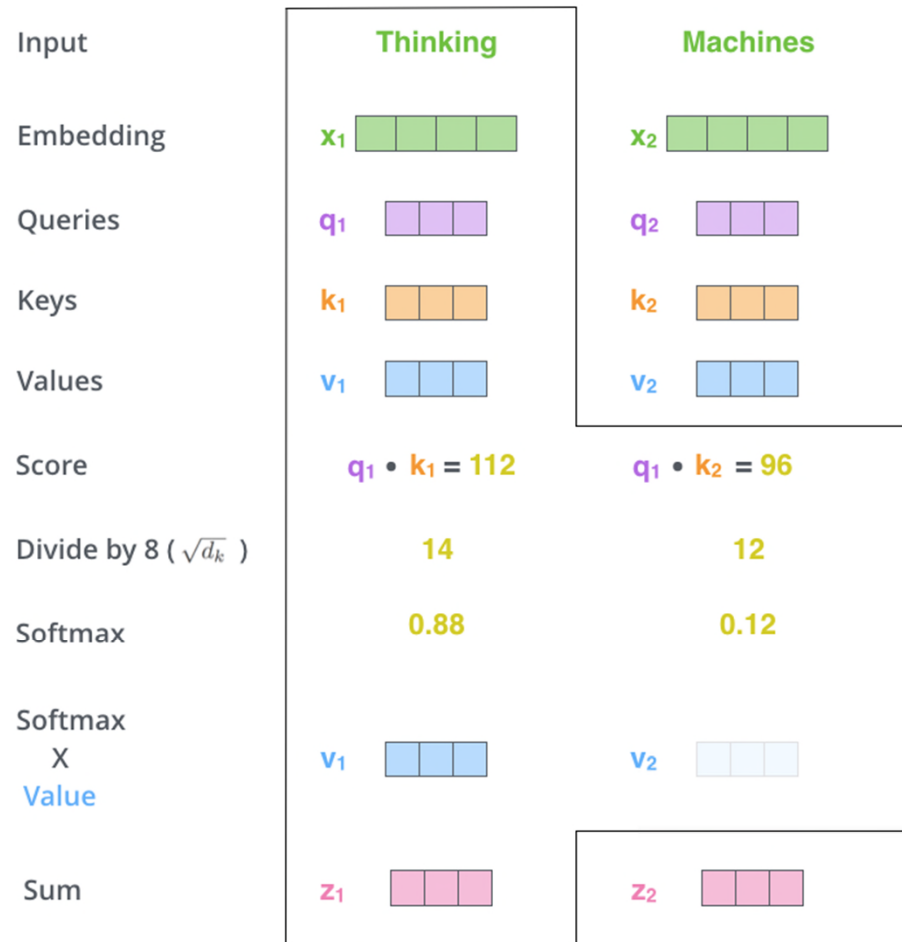
- Each input token generates 3 vectors: query (q), key (k) and value (v). These vectors provide more flexible representation through linear mapping (W_Q , W_K , W_V) to learn the underlying relationship / attention between input tokens.
- Attention is computed using:
 - Step 1: compute the correlation (dot product) between the query (q) and key (k) vectors.
 - Step 2: correlation values from Step 1 are scaled and normalized using Softmax function.
 - Step 3: multiplied output from Step 2 by corresponding value (v) vectors and sum them up.

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Scaled Dot-Product Attention

- Step 1: compute the correlation (dot product) between the query (q) and key (k) vectors.
- Step 2: correlation values from Step 1 are scaled and normalized using Softmax function.
- Step 3: multiplied output from Step 2 by corresponding value (v) vectors and sum them up.



$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$= \text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \text{2x2 purple grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{2x2 orange grid} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \text{2x2 blue grid} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \text{2x2 pink grid} \end{matrix}$$

Exercise: Self-Attention

In an application, a student would like to perform self-attention for an input data consists of 2 time-steps given by:

$$\mathbf{x}_1 = \begin{bmatrix} 1 & 4 \end{bmatrix} \text{ and } \mathbf{x}_2 = \begin{bmatrix} 2 & 1 \end{bmatrix}.$$

The query, key and value matrices are given by:

$$\mathbf{W}_Q = \begin{bmatrix} 0.4 & 0.3 \\ 0.2 & 0.1 \end{bmatrix}, \mathbf{W}_K = \begin{bmatrix} 0.1 & 0.2 \\ 0.1 & 0.3 \end{bmatrix}, \mathbf{W}_V = \begin{bmatrix} 0.6 & 0.7 \\ 0.8 & 0.9 \end{bmatrix}.$$

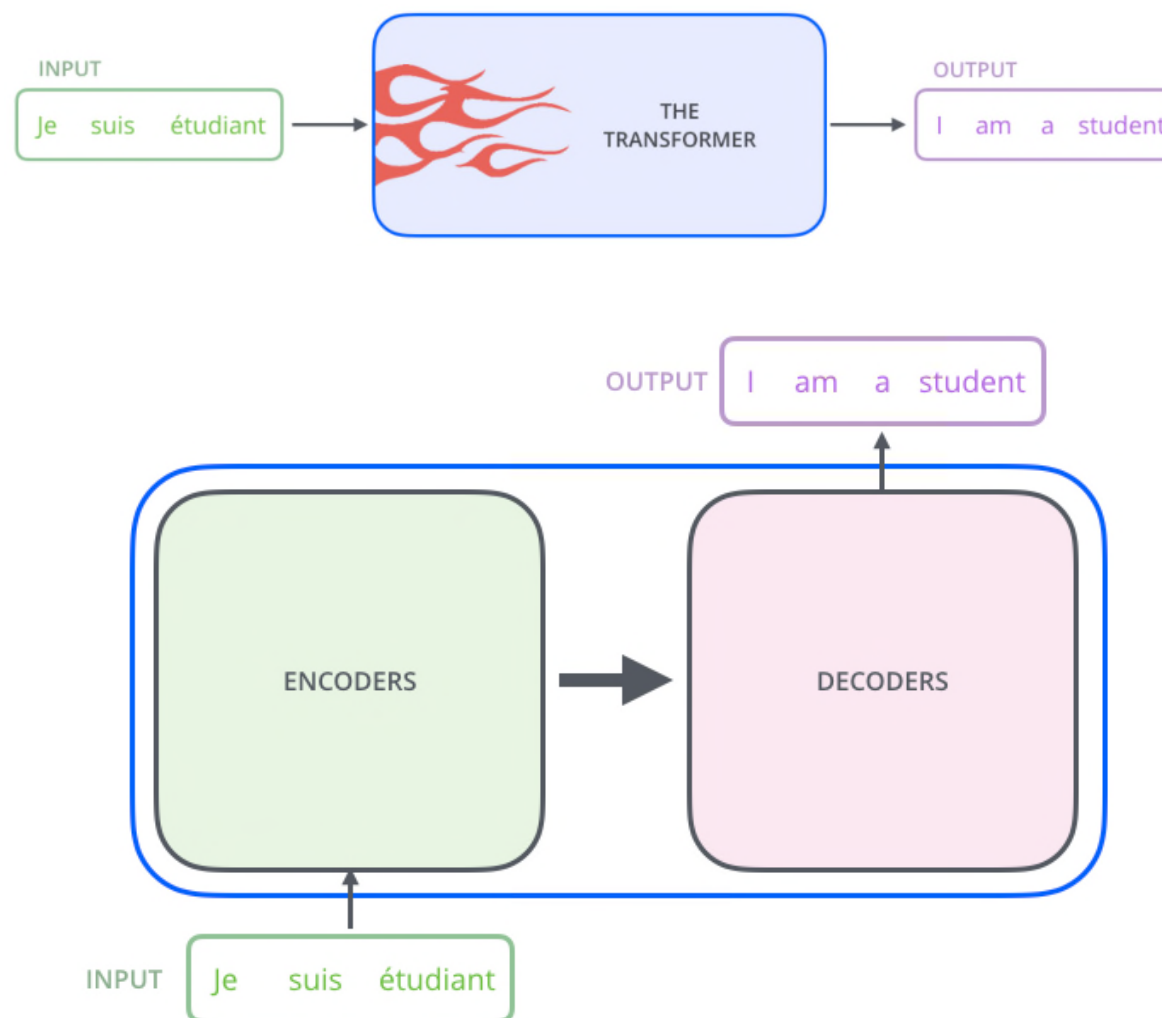
- (a) Find the query ($\mathbf{q}_1$) and key ($\mathbf{k}_1$) vectors for $\mathbf{x}_1$.
- (b) Find the query ($\mathbf{q}_2$) and key ($\mathbf{k}_2$) vectors for $\mathbf{x}_2$.
- (c) Find the dot product between $\mathbf{q}_1$ and $\mathbf{k}_1$.
- (d) Find the dot product between $\mathbf{q}_1$ and $\mathbf{k}_2$.
- (e) Compare and discuss the attention scores for input $\mathbf{x}_1$.

Solution

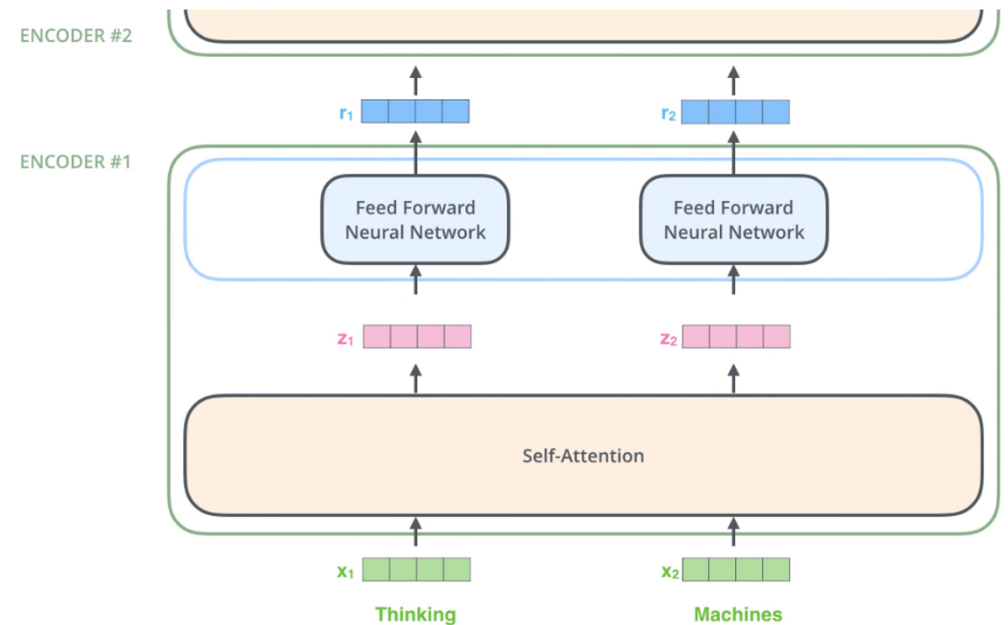
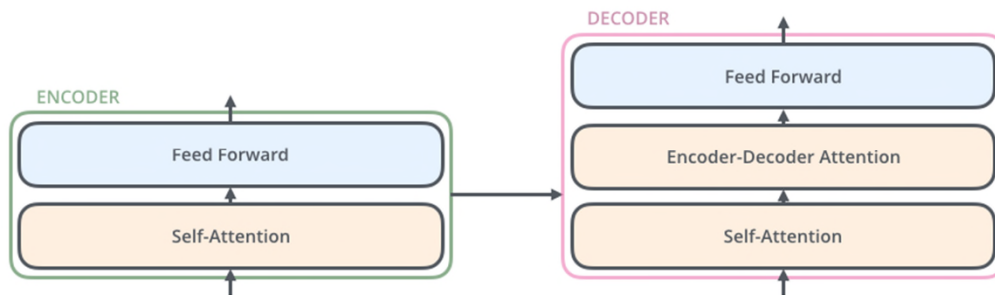
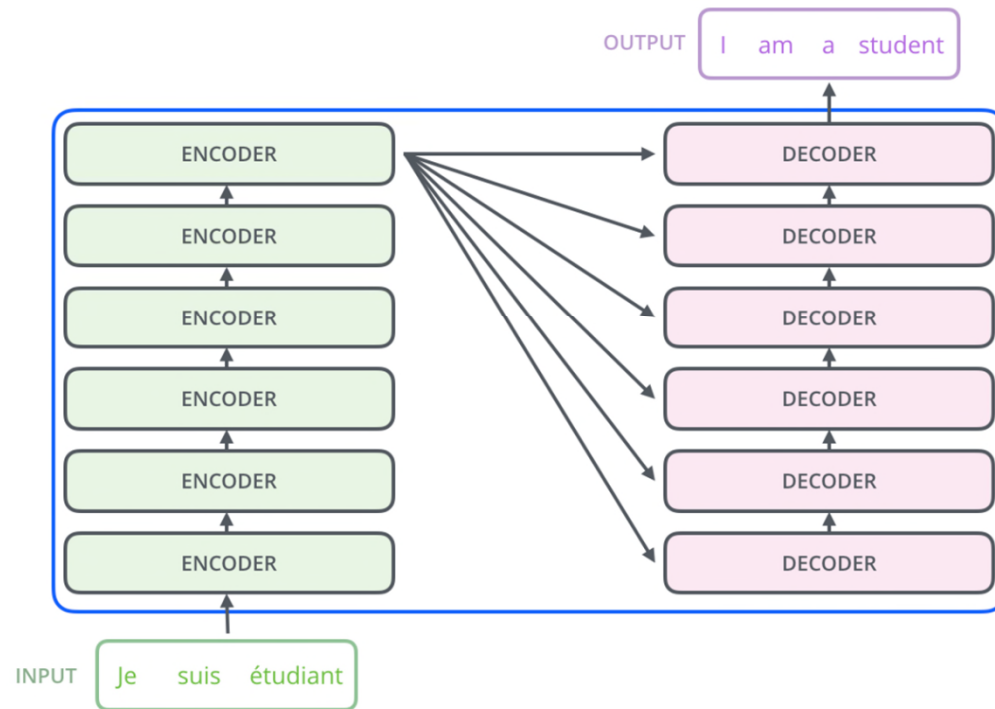
What is a Transformer?

- Transformer uses attention mechanism to process input sequence in parallel.
- Use attention mechanism and dense/feedforward/MLP layers.
- Highly parallelizable.
- Can offer global attention.
- Good at modelling long-range dependency.
- Achieve state-of-the-art performance in many vision and NLP applications.
- Lead to other SOTA methods such as BERT: Pre-training of Deep Bidirectional Transformers for Language.

Transformer in Machine Translation (1)



Transformer in Machine Translation (2)



Attention is All You Need

- Initially designed for neural machine translation.
- Later extended to visual tasks such as recognition, detection, etc, with great success.
- Leverage on attention mechanism to analyze the importance of a token (word/image patch) with respect to other tokens/image patches.
- Consist of transformer encoder and transformer decoder.

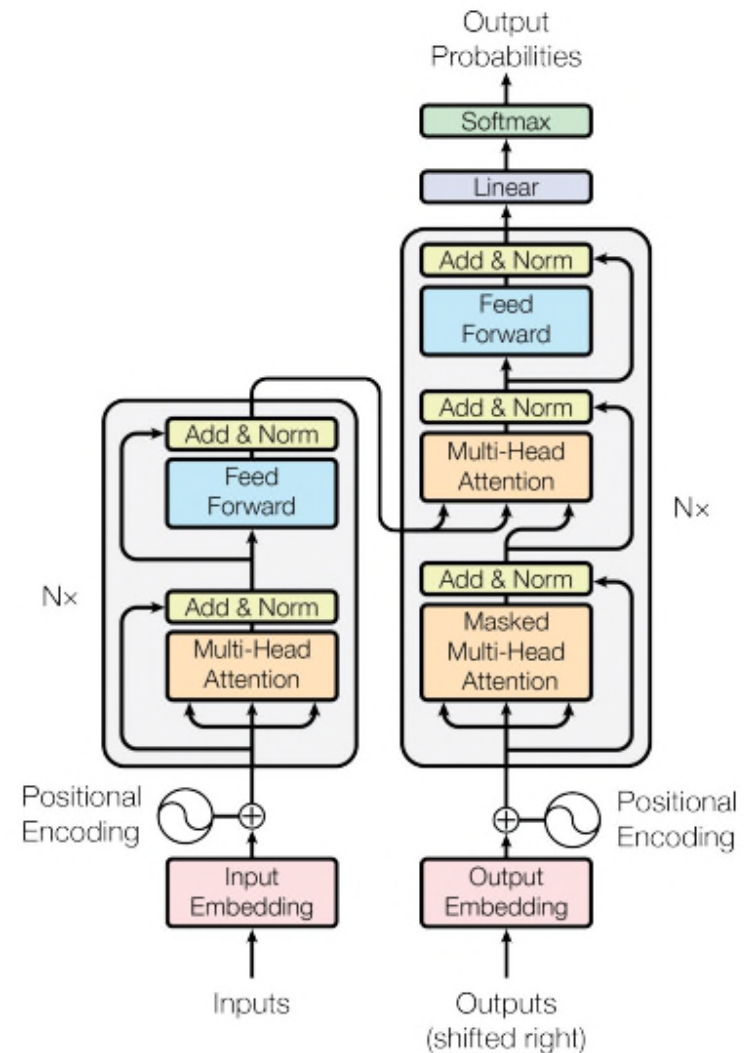
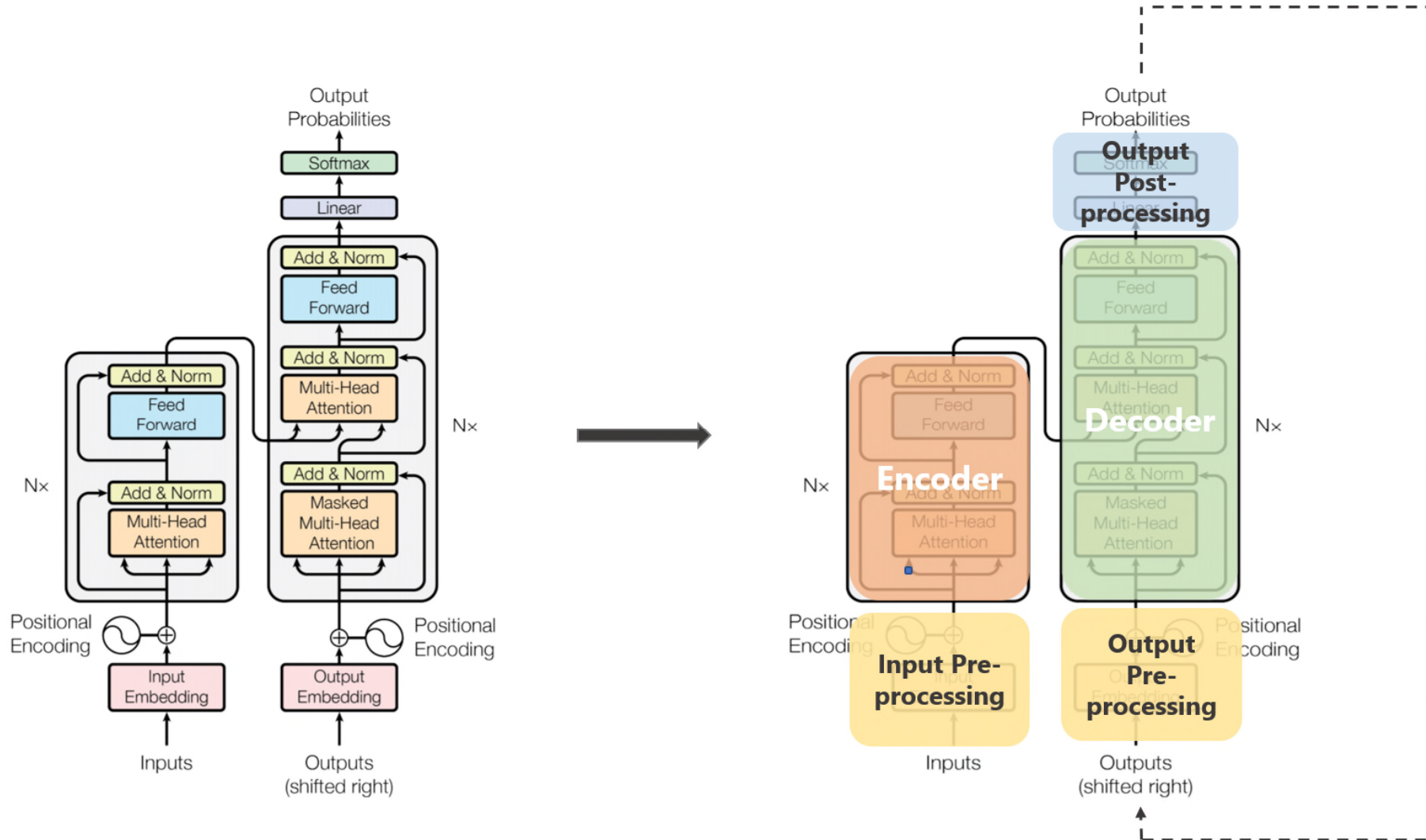
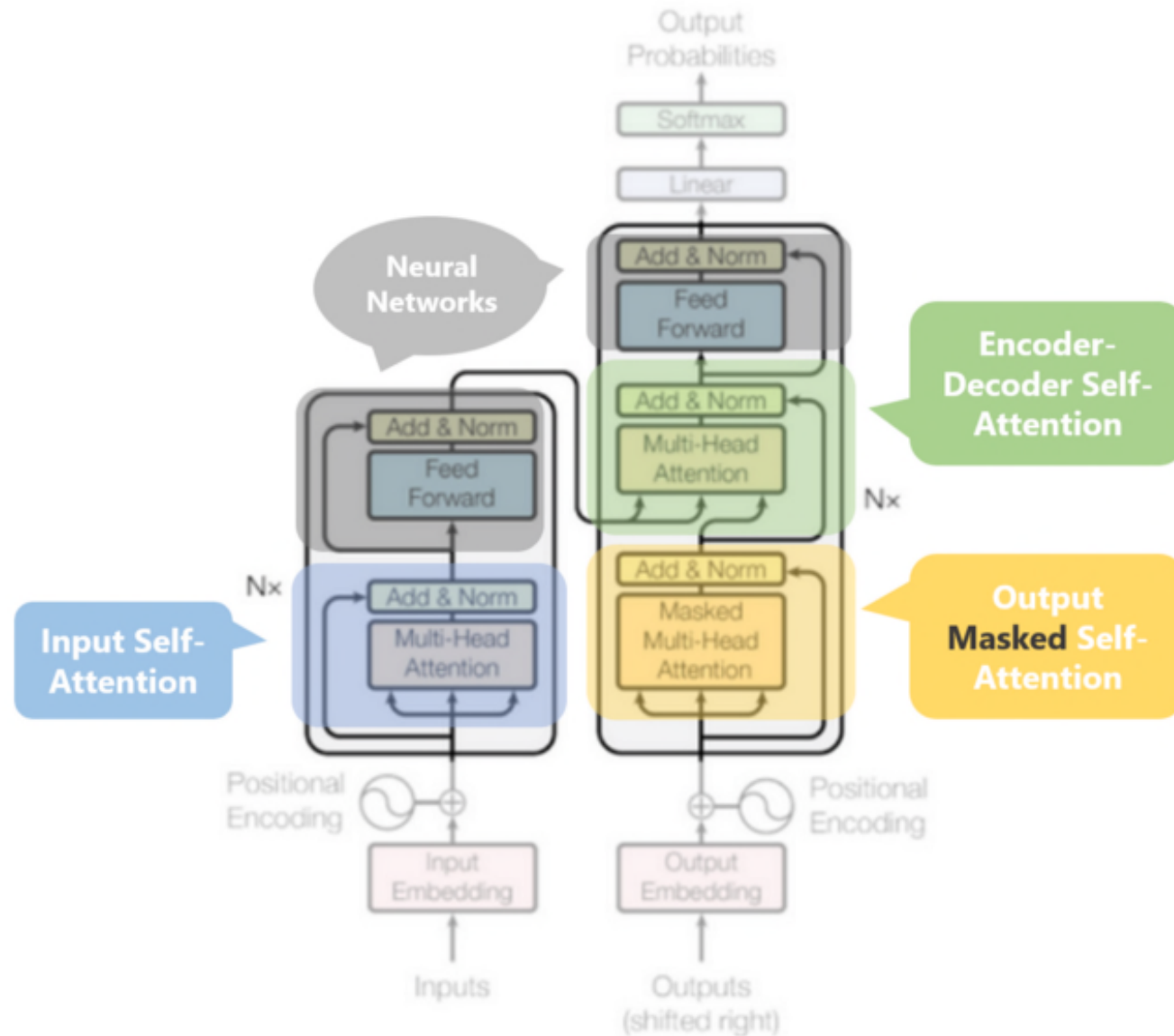


Figure 1: The Transformer - model architecture.

Transformer Architecture (1)

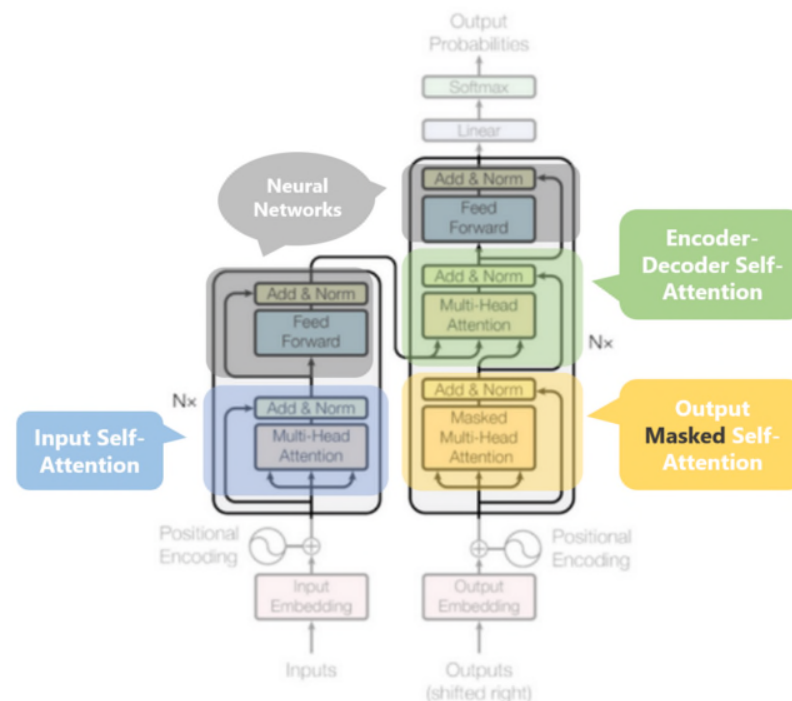
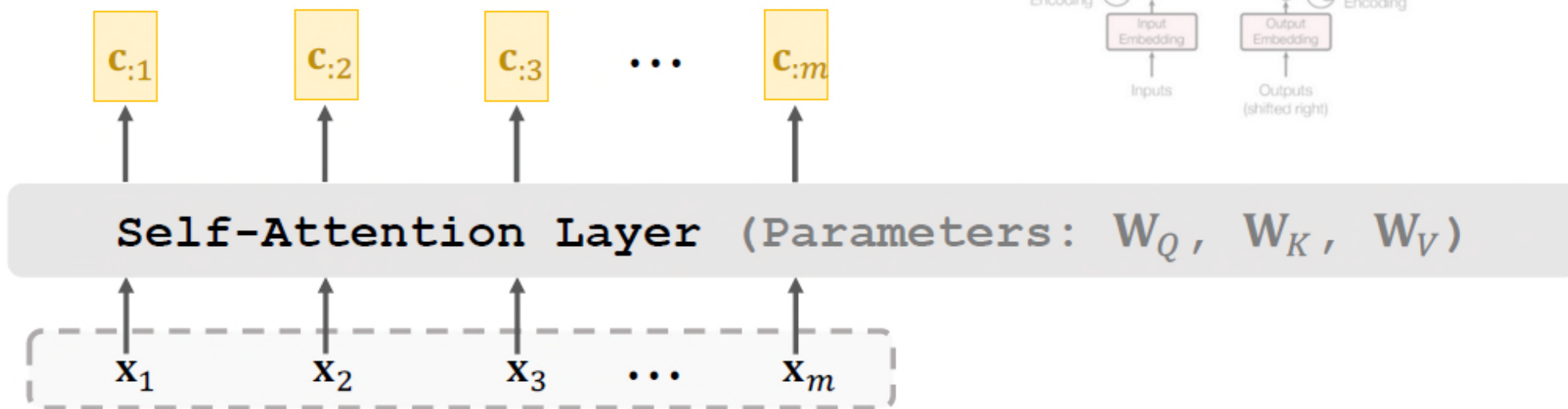


Transformer Architecture (2)



Self-Attention Layer Overview

- Self-attention layer: $\mathbf{C} = \text{Attn}(\mathbf{X}, \mathbf{X})$.
 - Inputs: $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m]$.
 - Parameters: $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$.

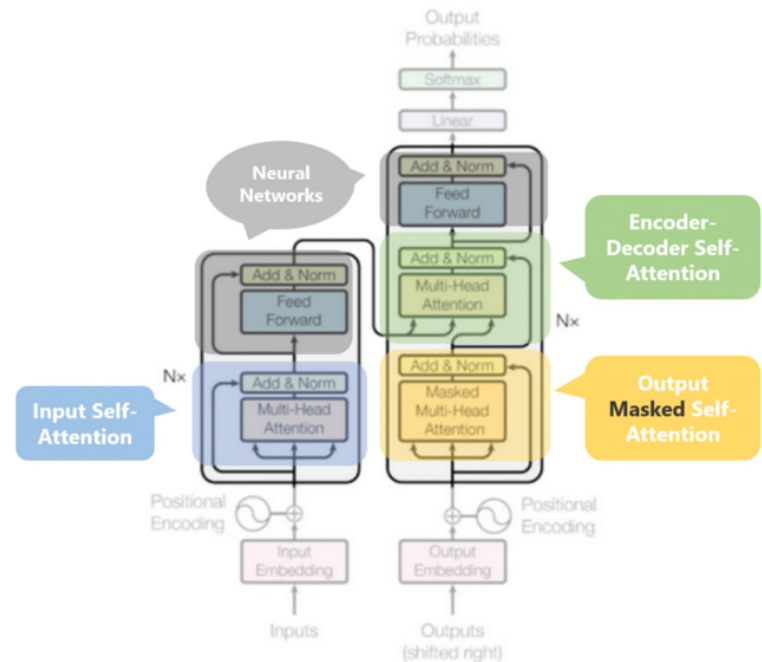
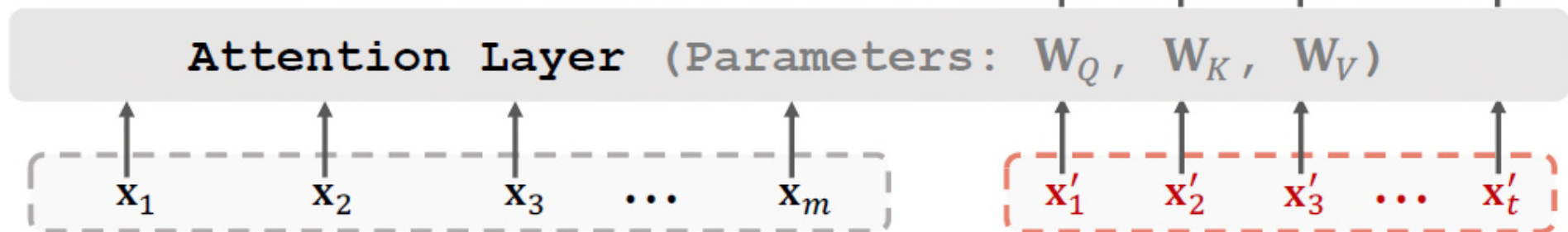


Cross-Attention / Encoder-Decoder Self-Attention

- Attention layer: $\mathbf{C} = \text{Attn}(\mathbf{X}, \mathbf{X}')$.

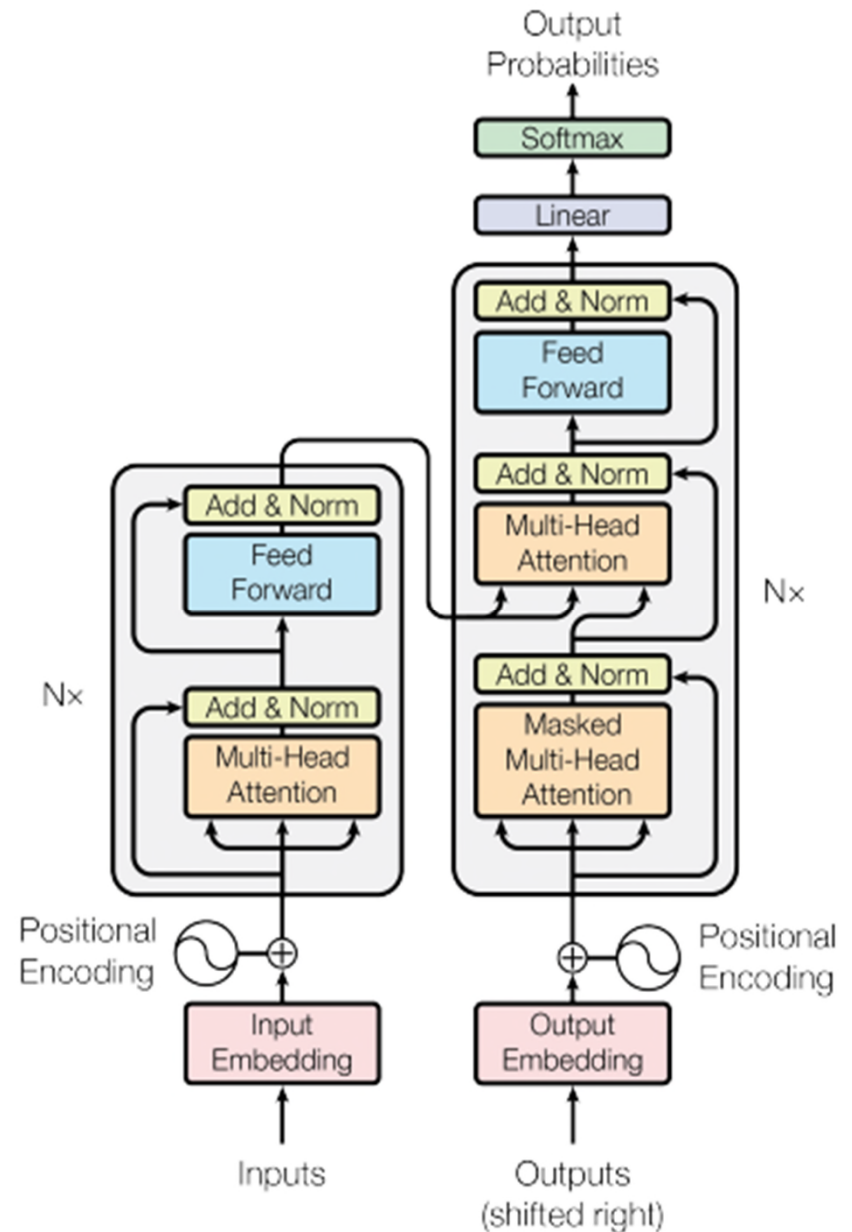
- Query: $\mathbf{q}_{:j} = \mathbf{W}_Q \mathbf{x}'_j$,
- Key: $\mathbf{k}_{:i} = \mathbf{W}_K \mathbf{x}_i$,
- Value: $\mathbf{v}_{:i} = \mathbf{W}_V \mathbf{x}_i$.
- Output: $\mathbf{c}_{:j} = \mathbf{V} \cdot \text{Softmax}(\mathbf{K}^T \mathbf{q}_{:j})$.

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

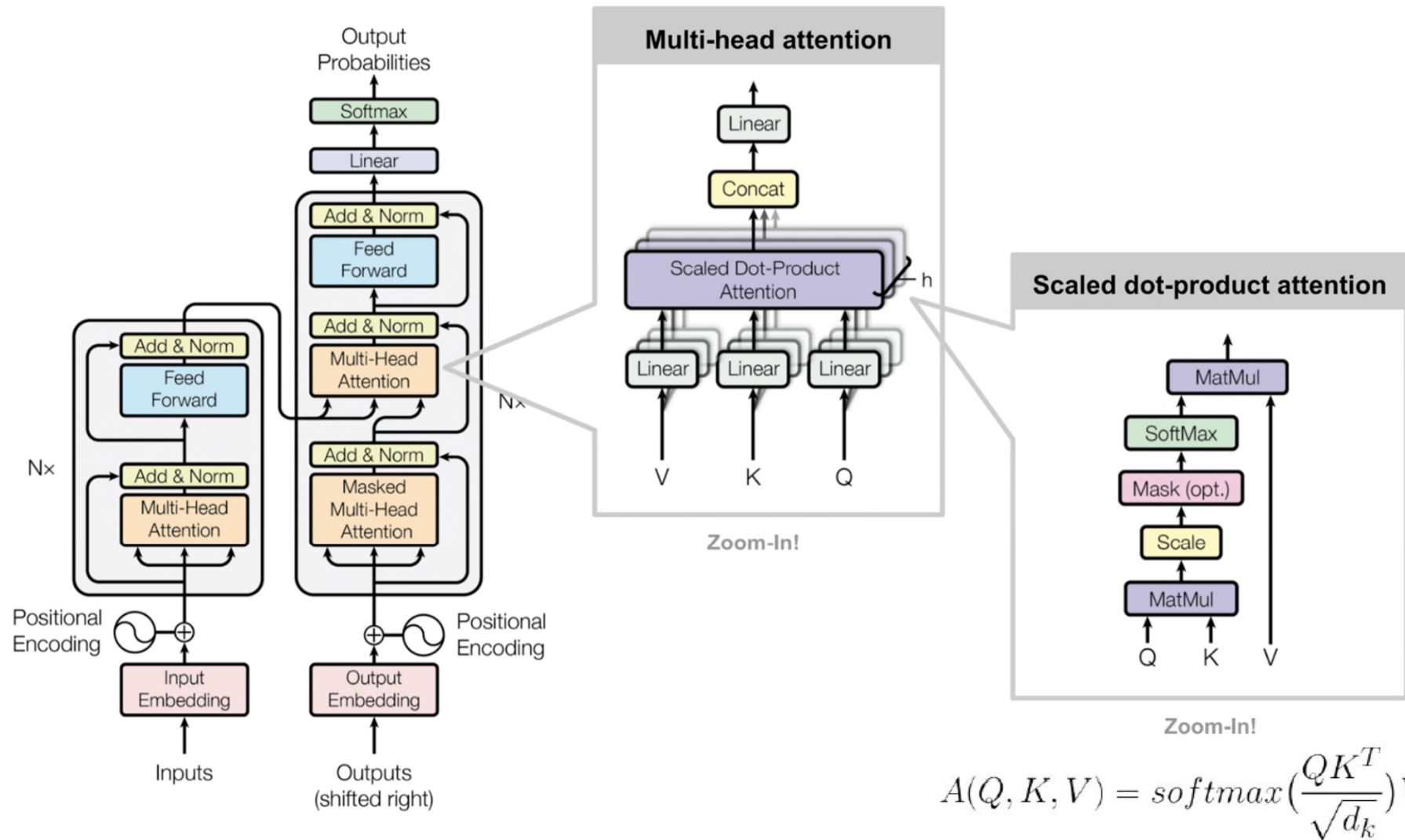


More Details

- Multi-Head Attention
- Feed Forward Network
- Positional Encoding
- Residual Connection
- Layer Normalization (Norm)
- Input/Output Embedding
- Masked Attention

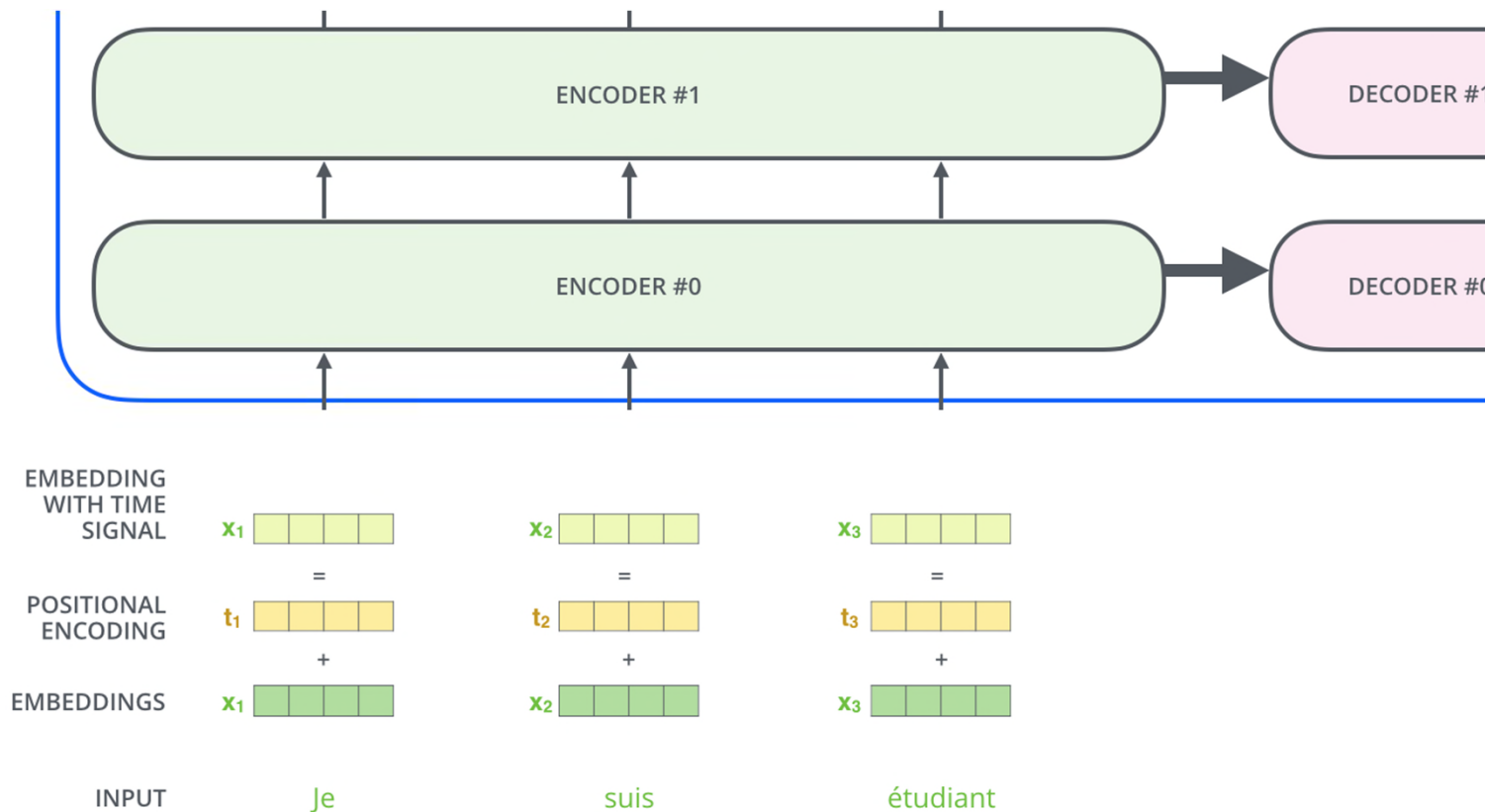


Multi-Head Attention



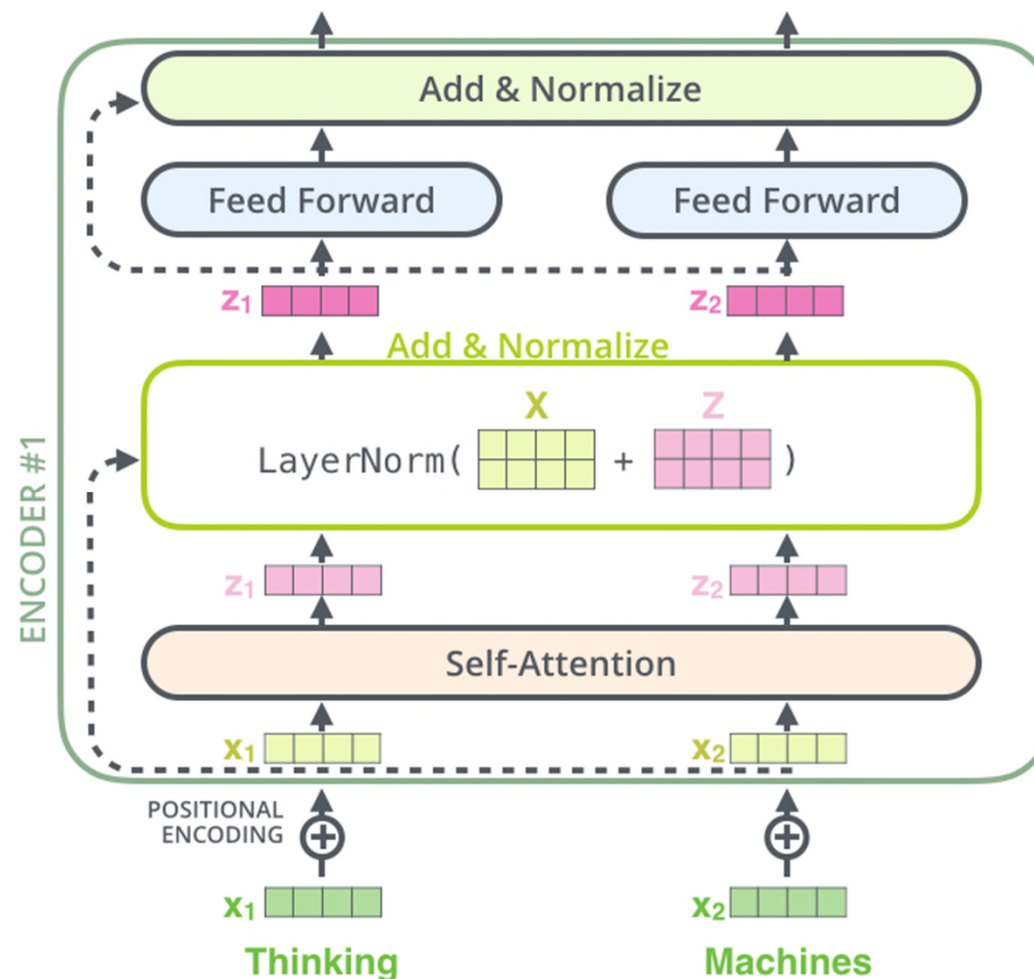
Position Encoding

- Represent the position information of individual input tokens.
- Position encoding using sin / cos functions are often used.

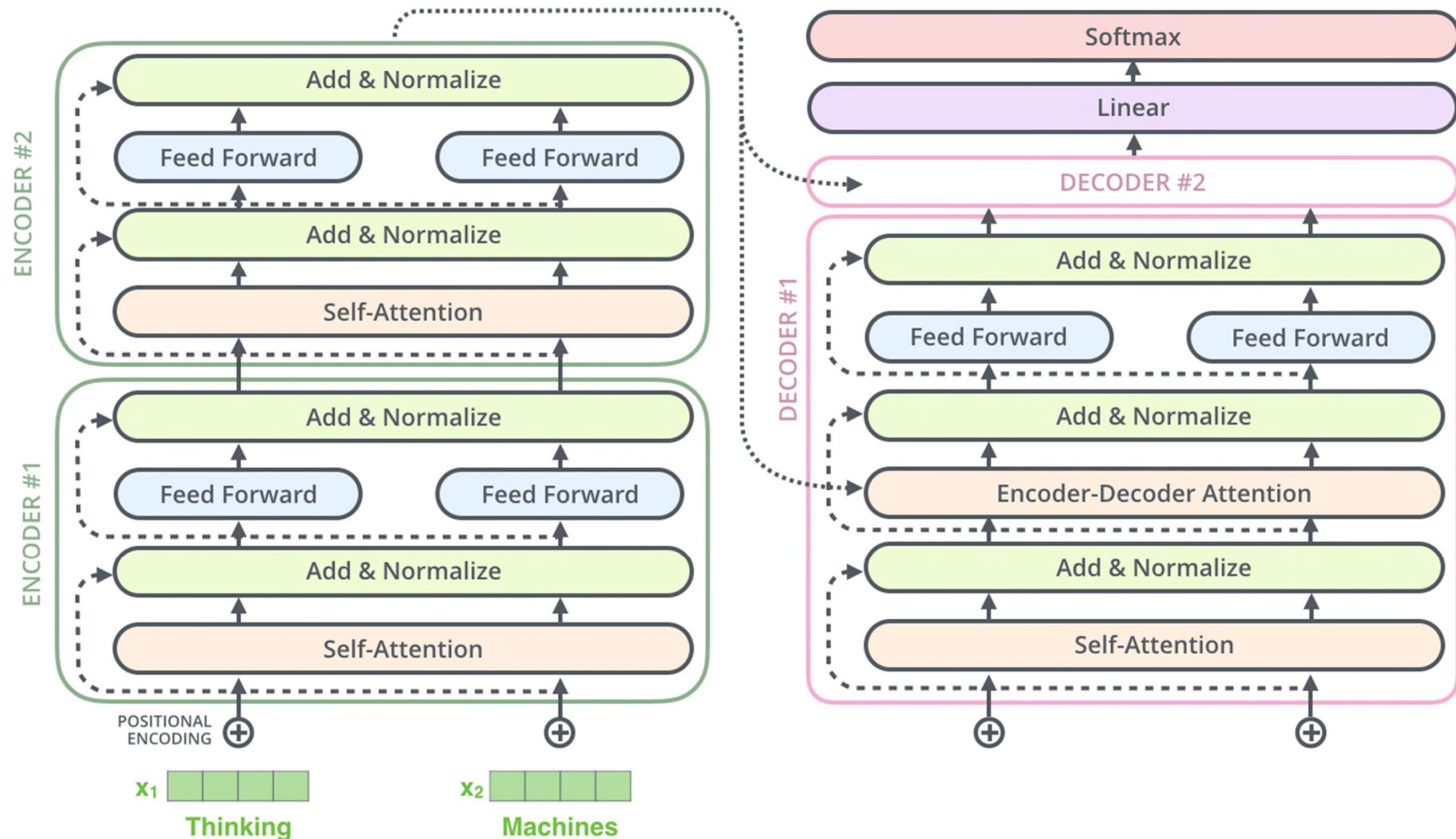


Residual Connection & Layer Norm

- Residual connection leverages the idea of residual learning in Resnet.
- Layer Norm is used to perform normalization.



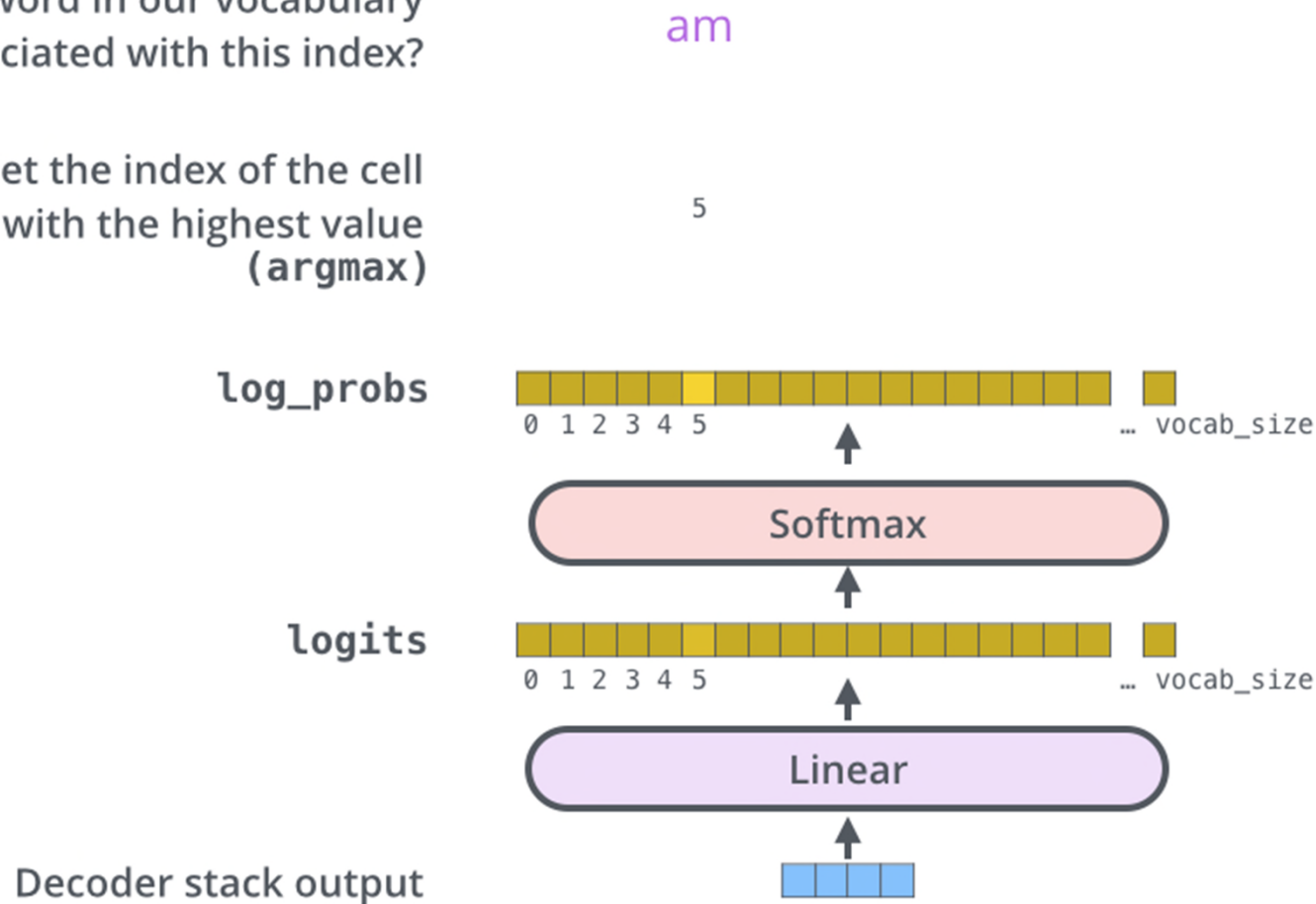
Transformer Encoder + Decoder Architecture



Final Linear and Softmax Layer

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(**argmax**)



Vision Transformer (1)

- A SOTA image classification model based on attention.

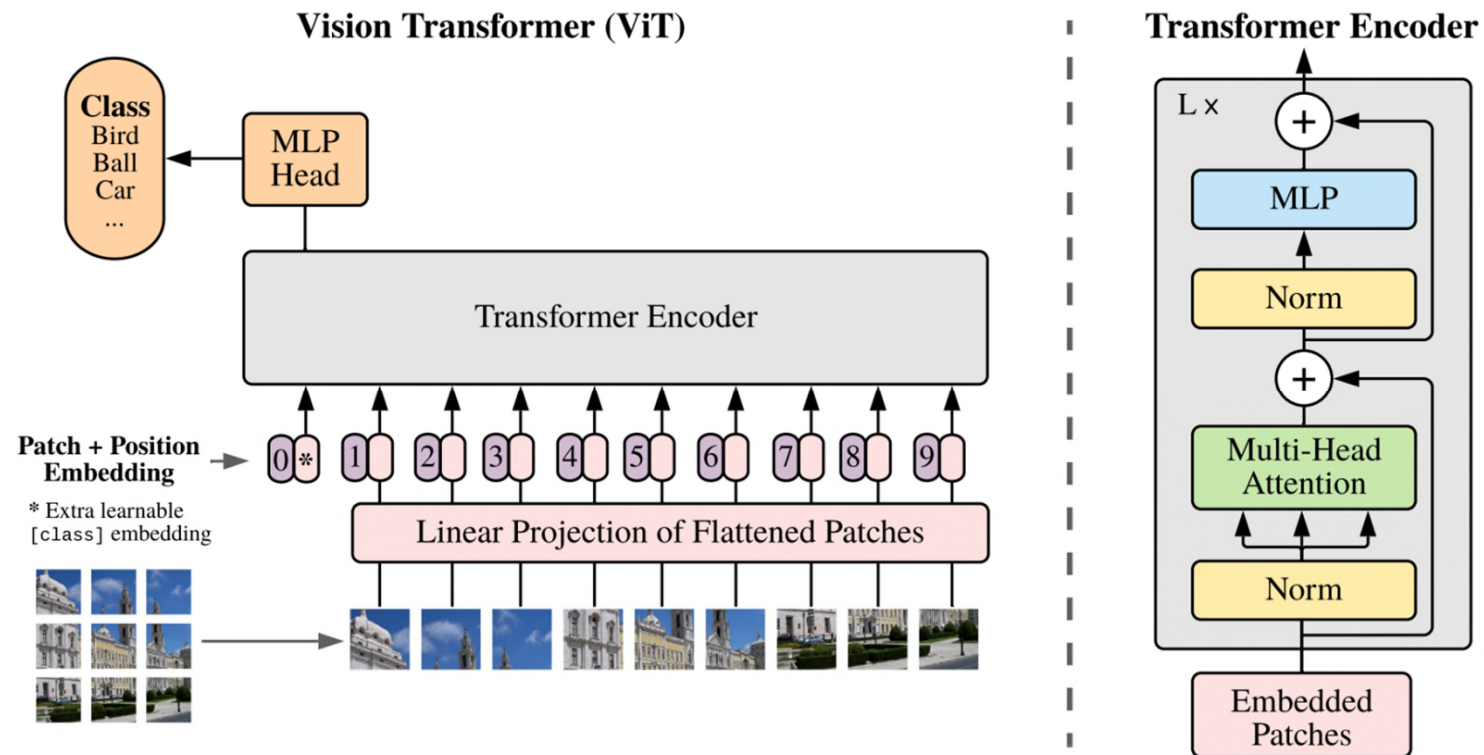


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Vision Transformer (2)

- Key steps of Vision Transformer (ViT):
 - Partition an image into patches / tokens.
 - Flatten the patches / tokens using lexicographical ordering.
 - Generate linear embeddings from the flattened patches / tokens.
 - Introduce an extra learnable class embedding.
 - Add positional embeddings / encoding.
 - Pass the tokens to a transformer encoder.
 - Encoder output of the extra learnable token will pass through a MLP head network for classification.
 - Training will involve using a pretrain model, and finetune it using the target dataset for image classification.

Exercise: Model Comparison

- (b) A user would like to develop an image classification application using a model that can achieve good accuracy and uses attention mechanism when performing classification. He is considering the following 3 candidate models: (i) VGG, (ii) Vision Transformer (ViT), and (iii) Long Short-Term Memory (LSTM). State which model is most likely going to meet the user's need and briefly justify your answer.

(5 Marks)

Solution

Section III Summary

- The section covers the following topics:
 - Attention Concept
 - Transformer Architecture
 - Vision Transformer

Part 4 Summary

- This part covers the following:
 - Convolutional Neural Network (CNN)
 - Recurrent Neural Network (RNN)
 - Long Short-Term Memory (LSTM)
 - Transformer